

# TW128: A Tree-Structured Duplex Authenticated Encryption Mode

Coda Hale

Hale Institute for Bicycle Appreciation

**Abstract.** We present TW128, a nonce-based authenticated encryption mode with associated data built as a two-level tree of keyed duplexes over the KECCAK- $p$ [1600, 12] permutation. A serial root duplex binds the key, nonce, and associated data, encrypts the first message chunk, and aggregates tags from independently keyed leaf duplexes for the remaining chunks. The leaf decomposition is canonical, but the number of leaves evaluated at once is an implementation choice: scalar, SIMD, and multicore implementations produce identical ciphertexts and tags without making the degree of parallelism a mode parameter.

TW128 makes the authentication tag a digest-like commitment to the full  $(K, N, A, M)$  context. In the public permutation model, via an intermediate random oracle and the flat sponge lift, we prove concrete multi-user indistinguishability and derive all-in-one authenticated encryption as the single-user corollary. We also prove that the wrapper returning the tag is a secure hash of the full context—collision-, second-preimage-, and preimage-resistant in the standard hash-function sense. The same tag analysis gives the AEAD-facing commitment guarantees: full-input CMTDs-4 committing security and context discoverability (CDY\*). At the implemented parameters, all stated notions meet a 128-bit security target. Vectorized **amd64** and **arm64** implementations exploit the scalable leaf structure, exceed 10 Gbit/s, and reach 50–75% of hardware-accelerated AES-128-GCM throughput.

**Keywords:** authenticated encryption · committing security · sponge constructions · duplex constructions · Keccak

## 1 Introduction

Authenticated encryption with associated data (AEAD) is often treated in practice as if its tag were a compact digest of the encryption context: a short string that binds the key, nonce, associated data, and message as a hash digest would. Widely deployed AEADs do not generally provide that property. AES-GCM and ChaCha20-Poly1305, for example, are not key-committing: an adversary can craft a single ciphertext that decrypts to valid plaintexts under two or more keys [DGRW18, ADG<sup>+</sup>22]. The cause is structural. Their authenticity rests on algebraic MACs whose outputs can often be made to validate under a second context, whereas a digest is expected to resist both collisions and preimages.

The resulting attacks are not edge cases in the formalism; they are failures of the digest intuition that users and protocols naturally assign to tags. The “invisible salamander” attack defeats Facebook’s attachment franking scheme [DGRW18]; related constructions produce ciphertexts that open to different well-formed files under different keys [ADG<sup>+</sup>22]; partitioning oracles recover passwords from systems built on non-committing AEAD [LGR21]; and context discovery attacks break GCM, CCM, EAX, SIV, and OCB3 by exploiting authentication layers that are not preimage-resistant [MLGR23]. These attacks differ in surface form, but all expose the same mismatch: the tag is used as if it were a binding digest, while the mode only proves ordinary authenticity.

The AEAD commitment literature has captured important parts of this digest model. Compactly committing AEAD was introduced for message franking [GLR17], and the committing hierarchy of [BH22] makes the collision-style requirement explicit: no two distinct contexts should open the same ciphertext and tag. Context discoverability [MLGR23] captures the preimage-facing counterpart: given a target ciphertext, it should be infeasible to find a decrypting context. These notions explain when a ciphertext can have another valid opening, or any valid opening at all, but they are still phrased as AEAD games over ciphertexts. The practitioner’s assumption is more hash-like: the tag itself is treated as a compact fingerprint of the encrypted object. We therefore analyze the tag with hash notions directly, and then recover the AEAD commitment notions as game-specific consequences of the same tag analysis.

TW128 (*TreeWrap*) makes that shift structural: instead of starting with a non-committing AEAD and adding a hash-based commitment transform, we start from the structure of a modern parallel hash and add encryption. KangarooTwelve [BDP<sup>+</sup>16] splits its input into fixed-size chunks, hashes the chunks independently under KECCAK- $p$ [1600, 12], and absorbs their chaining values into a root node; the leaves run in parallel across SIMD lanes and cores, with memory bounded independently of the input length. TW128 adapts that tree into a nonce-based AEAD whose authentication tag is itself a digest of the full  $(K, N, A, M)$  context.

Structurally, TW128 is a two-level tree of keyed duplexes over one permutation. A serial root duplex, initialized with the key  $K$  and nonce  $N$ , absorbs the associated data, encrypts the first message chunk, absorbs a tag from each remaining chunk, and emits the final root tag. Each remaining chunk is processed by an independent leaf duplex keyed by  $(K, N, j)$ , where  $j$  is the chunk index; the leaf encrypts its chunk and returns a leaf tag to the root. Thus every input reaches the root directly or through a leaf tag, while every duplex state remains bounded at 1600 bits. As in KangarooTwelve’s *kangaroo hopping*, a single-chunk message uses only the root duplex and pays no tree overhead (Section 7.4).

TW128 is not merely parallelizable; it is scalable. The mode fixes a canonical leaf decomposition and transcript, but it does not fix how many leaves an implementation evaluates at once. A scalar implementation, an AVX2 implementation, an AVX-512 implementation, a NEON implementation, and a multicore implementation all produce identical ciphertexts and tags. The degree of parallelism is therefore not a mode parameter and is not visible on the wire: it is an implementation choice made at run time to match the host. This separates TW128 from prior duplex-based AEADs with parallel variants, where the parallelism degree is fixed by the mode instance or otherwise shared by the communicating parties.

We prove that the wrapper  $H$  returning the root tag is a secure hash in the sense of [RS04]: collision-, second-preimage-, and preimage-resistant as a function of the full  $(K, N, A, M)$  input. The 256-bit root tag is therefore a compact digest-like commitment to the full context. The familiar AEAD commitment notions are recovered from the same root tag candidate analysis: full-input  $s$ -way CMTDs-4 [BH22] is the collision-facing AEAD consequence, while [MLGR23] context discoverability  $CDY^*$  is the preimage-facing consequence, with an additional hidden-component guessing term. The proofs use the structure of the AEAD games to keep the stated bounds tight.

We analyze TW128 in the public permutation model. We obtain a concrete multi-user indistinguishability bound in the sense of [BT16], with all-in-one authenticated encryption in the sense of [RS06] as its single-user corollary, together with the hash, committing, and context discoverability bounds above. Each bound is proved first against an intermediate random oracle and then transferred to the real permutation by the flat sponge lift of [BDPVA08] (Section B). At the implemented parameters ( $k = c = \tau_{\text{root}} = \tau_{\text{leaf}} = 256$ ), the bounds meet a 128-bit security target for adversaries running up to  $N_{\text{tot}} \leq 2^{63}$  KECCAK- $p$ [1600, 12] calls—on the order of  $2^{71}$  bytes of processed data.

Optimized `amd64` (AVX-512 and AVX2) and `arm64` (NEON with the SHA-3 extension) implementations turn this scalable leaf structure into measured throughput. On contemporary hardware, TW128 exceeds 10 Gbit/s and reaches 50–75% of hardware-accelerated AES-128-GCM (Section 7), while using one permutation for confidentiality, authentication, and commitment.

## 1.1 Our Contributions

- **A scalable tree AEAD mode.** We specify TW128, a nonce-based AEAD with associated data built as a two-level tree of strict [BDPVA11] duplexes over KECCAK- $p[1600, 12]$  (Section 3). The mode fixes the root/leaf transcript but delegates the number of leaves evaluated simultaneously to the implementation, so scalar and vectorized implementations interoperate without changing ciphertexts or tags.
- **Digest-like commitment.** We prove that the wrapper  $H$  returning the root tag is a secure hash of the full  $(K, N, A, M)$  input in the sense of [RS04]: collision-, second-preimage-, and preimage-resistant (Theorems 3 to 5). Thus the authentication tag behaves like the compact digest practitioners often expect an AEAD tag to be.
- **AEAD commitment consequences.** We instantiate the same root tag analysis in the existing AEAD vocabulary. The all-bodies-equal structure of CMTDs-4 gives the full-input  $s$ -way committing bound without the leaf-collision slack of the general hash collision theorem [BH22] (Corollary 3), and the remaining CMT/CMTD notions for  $\ell \in \{1, 3, 4\}$  follow from the committing hierarchy and direct source-game arguments (Corollary 6). The same fixed-target root-preimage analysis, plus hidden-component guessing, gives CDY\* context discoverability security [MLGR23] (Theorem 6).
- **Multi-user and AE security.** We give a concrete multi-user indistinguishability bound [BT16] for  $D$  users (Theorem 2). The  $D = 1$  case yields a concrete all-in-one AE bound [RS06] in the public permutation model (Corollary 1). The proof proceeds through a dedicated TW128 random oracle model and is transferred to the public permutation setting by the [BDPVA08] flat sponge lift.
- **Optimized implementations.** We describe and evaluate vectorized `amd64` (AVX-512 and AVX2) and `arm64` (NEON with the SHA-3 extension) implementations whose batched leaf cores realize the mode’s scalable parallelism in practice (Section 7).

## 1.2 Related Work

**Committing AEAD.** The study of committing authenticated encryption was initiated by [GLR17], who introduced compactly committing AEAD for message franking, and by [DGRW18], who built it from a dedicated primitive (encryptment). [BH22] organized the collision-style notions into the CMT-1/3/4 and CMTD hierarchy and gave the UtC, RtC, and HtE transforms; [CR22] gave the CTX transform, which commits a nonce-based AE scheme to its full context by adding a hash. These transforms are efficient and broadly applicable, but they treat commitment as an external layer. TW128 instead makes the authentication tag itself the hash-like commitment: the root tag is the compact commitment, and the root tag analysis gives both the [BH22] full-input committing consequence and the [MLGR23] discoverability guarantee. Closest in spirit is the recent work of [DHM<sup>+</sup>24], which also builds nonce-based AE natively on SHA-3 functions and obtains CMT-4 from collision resistance; it targets session-based communication through a sequential duplex chain, whereas TW128 is a single scalable tree for standalone AEAD (Section 8.3).

**Sponge and duplex AEADs.** Permutation-based authenticated encryption built on the sponge and duplex constructions of [BDPVA08, BDPVA11] is well established. Keyak [BDP<sup>+</sup>15] uses the Motorist mode over round-reduced KECCAK- $p$  and already admits parallel duplex instances; Ascon [DEMS21] and Xoodyak [DHP<sup>+</sup>20] are serial duplex AEADs over smaller permutations. TW128 shares the single-permutation, constant-state duplex foundation but differs in where parallelism lives. Keyak fixes the degree of parallelism as a mode parameter, while Ascon and Xoodyak expose no mode-level parallelism. In TW128, the decomposition into indexed leaves is fixed by the transcript, but the number of leaves evaluated at once is chosen by the implementation and is invisible to the wire format (Section 8.2).

**Tree hashing.** TW128’s two-level shape is inspired by KangarooTwelve [BDP<sup>+</sup>16], a tree hash over the same permutation. TW128 adopts its choice of permutation and final-node/leaf split and adapts them to authenticated encryption: each leaf chunk is encrypted and tagged independently, leaf tags replace unkeyed chaining values, and the root is keyed by  $(K, N)$ . The construction also inherits KangarooTwelve’s short-message behavior: inputs fitting in one chunk use only the root duplex. Because TW128 has a fixed two-level topology with independently keyed, indexed leaves and phase-separated duplex calls, it does not need KangarooTwelve’s Sakura tree encoding [BDPVA14] to make its transcripts unambiguous (Section 3).

### 1.3 Paper Organization

Section 2 recalls the notation, sponge and duplex constructions, nonce-based AEAD security, committing security [BH22], context discoverability [MLGR23], and the hash notions of [RS04]. Section 3 specifies the TW128 construction and parameters. Section 4 fixes the public permutation games, intermediate random oracle model, and resource accounting. Section 5 states the headline bounds and proves the mu-ind, AE, and lifted hash and committing statements, while Section 6 proves the random oracle hash and committing bounds and gives the CDY\* proof. Section 7 reports implementation performance, and Section 8 compares TW128 to prior AEAD and committing constructions and discusses limitations. The appendices collect structural lemmas, the flat sponge lift, vectorized kernel details, and test vectors.

## 2 Preliminaries

### 2.1 Notation

**Strings.** Write  $\{0, 1\}$  for the binary alphabet. For an integer  $n \geq 0$ ,  $\{0, 1\}^n$  denotes the set of  $n$ -bit strings,  $\{0, 1\}^* = \bigcup_{n \geq 0} \{0, 1\}^n$  the set of all finite-length bit strings, and  $\varepsilon$  the empty string. For  $x \in \{0, 1\}^*$ ,  $|x|$  is the bit length and  $x \parallel y$  is the concatenation; the bitwise XOR of two equal-length strings  $x, y \in \{0, 1\}^n$  is  $x \oplus y$ . A *byte string* is an element of  $(\{0, 1\}^8)^*$ ; its byte length is denoted  $\|x\| = |x|/8$ . For an integer  $0 \leq n < 2^{64}$ ,  $\langle n \rangle_8 \in \{0, 1\}^{64}$  is its eight-byte little-endian encoding. For a finite or infinite bit string  $x$  and  $0 \leq \tau \leq |x|$  when  $x$  is finite,  $x|_\tau \in \{0, 1\}^\tau$  denotes the leading  $\tau$  bits of  $x$ .

**Sampling and probability.** We write  $x \leftarrow \$ S$  for sampling  $x$  uniformly at random from a finite set  $S$ , with implicit coins independent across samples. For an event  $E$  in a probability experiment,  $\Pr[E]$  denotes its probability.

**Sets of maps.** For a set  $S$ ,  $\text{Perm}(S)$  denotes the set of permutations on  $S$ ; for a positive integer  $b$ ,  $\text{Perm}(\{0, 1\}^b)$  is the set of  $b$ -bit permutations.

**Games and advantages.** We write  $\mathcal{A}^{O_1, \dots, O_k}$  for a (possibly randomized) adversary with oracle access to  $O_1, \dots, O_k$ , and  $\Pr[\mathbf{G}^{\mathcal{A}} \Rightarrow 1]$  for the probability that a game  $\mathbf{G}$ —an experiment that runs  $\mathcal{A}$  against specified oracles and returns a Boolean outcome—returns 1. For a distinguishing experiment between two games  $\mathbf{G}_0, \mathbf{G}_1$ , the advantage of  $\mathcal{A}$  against a scheme  $\Pi$  in notion  $\mathbf{X}$  is the absolute difference

$$\text{Adv}_{\Pi}^{\mathbf{X}}(\mathcal{A}) = |\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]|.$$

For a win-event experiment without a hidden challenge bit it is instead  $\Pr[\mathbf{G}^{\mathcal{A}} \text{ wins}]$ , the probability the game returns 1; a bit-guessing experiment with a hidden challenge bit uses the rescaled form  $|2 \Pr[\mathbf{G}^{\mathcal{A}} \text{ wins}] - 1|$ , as in the multi-user notion of Section 2.4.

**Concrete security.** All bounds in this paper are concrete: each advantage is an explicit function of the resource caps introduced in Section 4.3. We make no use of asymptotic or negligibility conventions.

**Reference summary.** Table 1 summarizes the resource caps and loss terms used throughout. The caps are defined formally in Section 4.3, the loss terms in Section 5.1.

**Table 1:** Resource caps and loss terms.

| Symbol                             | Meaning                                                      | Defined |
|------------------------------------|--------------------------------------------------------------|---------|
| $N$                                | Construction interface cost (KECCAK- $p$ [1600, 12] calls)   | § 4.3   |
| $N_{\text{prim}}$                  | Direct primitive-interface queries                           | § 4.3   |
| $N_{\text{tot}}$                   | $N + N_{\text{prim}}$ (total query-sequence cost)            | § 4.3   |
| $q_v, Q_v$                         | Valid verification queries / cap                             | § 4.3   |
| $q_{\text{RO}}, Q_{\text{RO}}$     | Direct random oracle queries / cap (RO-model)                | § 4.3   |
| $n_{\text{leaf}}, N_{\text{leaf}}$ | Distinct construction-generated final leaf transcripts / cap | § 4.3   |
| $\text{Lift}_c(N_{\text{tot}})$    | [BDPVA08] flat sponge loss                                   | § 5.1   |
| $\text{KeyGuess}_k(Q)$             | Ideal system key guess                                       | § 5.1   |
| $\text{RootColl}_{\tau}(Q, s)$     | $s$ -way root-collision                                      | § 5.1   |
| $\text{RootPre}_{\tau}(Q)$         | Root-preimage                                                | § 5.1   |
| $\text{LeafColl}_{\tau}(Q)$        | Known-key leaf-collision                                     | § 5.1   |

## 2.2 Sponges and Duplexes over Keccak- $p$

**The Keccak- $p$  permutation.** The KECCAK- $p$  family of permutations, introduced as part of the Keccak design and standardized within FIPS 202 [Nat15], is parameterized by a state width  $b$  and a round count  $n_r$ ; we write  $\text{KECCAK-}p[b, n_r] \in \text{Perm}(\{0, 1\}^b)$  for the corresponding permutation. TW128 uses KECCAK- $p$ [1600, 12], the same primitive as KangarooTwelve [BDP<sup>+</sup>16], fixed throughout. In the public permutation security model of Section 4.1 and the flat sponge lift of Section B, this permutation is modeled as a uniformly random element of  $\text{Perm}(\{0, 1\}^{1600})$ .

**The sponge construction.** Following [BDPVA08], fix a permutation  $\pi \in \text{Perm}(\{0, 1\}^b)$ , a *sponge-compliant* padding rule  $\text{pad}$ , and a rate  $1 \leq r < b$  with capacity  $c = b - r$ . A padding rule appends  $\text{pad}[r](|M|)$ , a bit string determined by the input length and the rate, to an input  $M$ , extending it to a sequence of  $r$ -bit blocks; it is sponge-compliant [BDPVA11, Definition 1] if  $M \parallel \text{pad}[r](|M|)$  is never empty and, for all  $M \neq M'$  and all  $n \geq 0$ ,  $M \parallel \text{pad}[r](|M|) \neq M' \parallel \text{pad}[r](|M'|) \parallel 0^{nr}$ . The sponge function  $\text{Sponge}[\pi, \text{pad}, r] : \{0, 1\}^* \times \mathbb{N} \rightarrow \{0, 1\}^*$  maps an input  $M$  and an output length  $\ell$  to an  $\ell$ -bit string by absorbing  $M \parallel \text{pad}[r](|M|)$  into the all-zero  $b$ -bit state in  $r$ -bit blocks (each separated by an

application of  $\pi$ ) and squeezing  $\ell$  bits in  $r$ -bit blocks from the resulting state. [BDPVA08, Theorem 2] shows that  $\text{Sponge}[\pi, \text{pad10}^*, r]$ , where the simple rule  $\text{pad10}^*$  appends a single 1 bit followed by the minimum number of 0 bits reaching a multiple of  $r$ , is indistinguishable from a random oracle when  $\pi$  is uniformly random. We bound its distinguishing advantage by  $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$ , derived in Lemma 16 from the exact differentiating probabilities of [BDPVA08, Lemmas 4 and 5].

**The duplex construction.** The *duplex* construction of [BDPVA11] extends the sponge to a stateful interface. A duplex object  $\text{Duplex}[\pi, \text{pad}, r]$  is initialized to the all-zero state. Each call  $\text{duplexing}(\sigma, \ell)$ , with  $\ell \leq r$ , absorbs an input string  $\sigma$  of at most  $\rho_{\max}(\text{pad}, r)$  bits via  $\pi$  and returns the first  $\ell$  bits of the resulting state. Here  $\rho_{\max}(\text{pad}, r) = \min\{x : x + |\text{pad}[r](x)| > r\} - 1$  is the largest input length for which the padded input fits in a single  $r$ -bit block; for the  $\text{pad10}^*1$  rule,  $\rho_{\max}(\text{pad10}^*1, r) = r - 2$  [BDPVA11]. [BDPVA11, Lemma 3] establishes the duplex-to-sponge equality: for any call sequence  $((\sigma_0, \ell_0), \dots, (\sigma_i, \ell_i))$ , the  $i$ -th output equals

$$\text{Sponge}[\pi, \text{pad}, r](\sigma_0 \parallel \text{pad}_0 \parallel \dots \parallel \sigma_i, \ell_i),$$

where  $\text{pad}_j = \text{pad}[r](|\sigma_j|)$  is the padding the duplex appends in its  $j$ -th call; the flattened input carries these interior paddings explicitly, and the sponge appends only the final  $\text{pad}_i$ . The flattening map is injective by [BDPVA11, Lemma 4]: the duplex-call sequence is recoverable from the flattened sponge input.

**TW128 instantiation.** TW128 fixes the padding rule  $\text{pad10}^*1$  and the rate  $r = 1344$ , so the capacity is  $c = 256$  bits. The largest input length per duplex call is therefore  $\rho_{\max}(\text{pad10}^*1, 1344) = 1342$  bits.

**Bridge to the  $H$ -input domain.** The indistinguishability theorem of [BDPVA08] is stated for the simpler  $\text{pad10}^*$  padding over its unpadded  $H$ -input domain, while TW128 uses  $\text{pad10}^*1$  internally. [BDPVA11, Theorem 3] supplies an injective preprocessing bridge  $I[r, r_{\max}]$  between the two domains. For TW128,  $r = r_{\max} = 1344$ , so the bridge reduces to appending  $1 \parallel 0^q$  with  $q$  determined by the input length modulo  $r_{\max}$ ; on the bridge image,  $q$  is recovered as the trailing-zero count. Section 4.2 instantiates this bridge on typed TW128 duplex transcript prefixes as the map  $\text{flat}(\cdot)$  used by the random oracle proofs throughout the paper.

## 2.3 Authenticated Encryption with Associated Data

**Syntax.** A *nonce-based authenticated encryption scheme with associated data* (AEAD scheme) is a pair  $\text{SE} = (\text{Enc}, \text{Dec})$  of deterministic algorithms with associated key space  $\mathcal{K} \subseteq (\{0, 1\}^8)^*$ , nonce space  $\mathcal{N} \subseteq (\{0, 1\}^8)^*$ , associated data space  $\mathcal{A} \subseteq (\{0, 1\}^8)^*$ , message space  $\mathcal{M} \subseteq (\{0, 1\}^8)^*$ , and ciphertext space  $\mathcal{C} \subseteq (\{0, 1\}^8)^*$ . The encryption algorithm  $\text{Enc} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M} \rightarrow \mathcal{C}$  produces a ciphertext  $C = \text{Enc}_K(N, A, M)$ . The decryption algorithm  $\text{Dec} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{C} \rightarrow \mathcal{M} \cup \{\perp\}$  returns either a plaintext or the rejection symbol  $\perp$ . The *tag length in bytes* is the constant  $\|\text{Enc}_K(N, A, M)\| - \|M\|$ , fixed by the scheme. In this generic syntax,  $C$  denotes the full AEAD ciphertext, including any tag material. The TW128 algorithm section (Section 3) writes this same value as  $C \parallel T$ , with  $C$  the ciphertext body and  $T$  the root tag.

**Correctness.** For every  $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$ ,  $\text{Dec}_K(N, A, \text{Enc}_K(N, A, M)) = M$ .

**All-in-one AE.** We use the all-in-one authenticated encryption notion of [RS06], instantiated for nonce-based AE following [BT16]. The experiment grants either the real pair  $(\text{Enc}_K, \text{Ver}_K)$  or the ideal pair  $(\text{Rand}, \text{Replay})$ , where  $\text{Rand}$  returns a fresh uniform byte string of the ciphertext length on each encryption query and  $\text{Replay}$  accepts exactly the tuples  $\text{Rand}$  produced. The advantage  $\text{Adv}_{\text{SE}}^{\text{ae}}(\mathcal{A})$  is the distinguishing advantage between these two worlds over nonce-respecting adversaries. This all-in-one notion is instantiated for TW128 in Section 4.1.

## 2.4 Multi-User Indistinguishability

The multi-user indistinguishability notion of [BT16, Fig. 3] extends the same real-vs-ideal oracle pair to an adversary-selected set of users. The number of users  $u$  is adversary-determined: it is the number of **New** queries the adversary makes, i.e., the final value of the user counter  $v$  in the game below. A challenge bit  $b \leftarrow \{0, 1\}$  selects between the real construction interface (real ciphertexts, real verification) and an ideal interface (uniform strings of matching length, verification that accepts only previously-issued encryption tuples). The adversary also accesses a public primitive interface: the real permutation  $(\pi, \pi^{-1})$  in the real world and the simulator  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$  in the ideal world. The adversary outputs a guess  $b'$  and wins if  $b' = b$ .

**Game and advantage.** The game  $\text{G}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A})$  of [BT16, Fig. 3] samples  $b \leftarrow \{0, 1\}$  and maintains a user counter  $v$ , a set  $V$  of  $(d, N)$  pairs already used by encryption, and a set  $W$  of recorded encryption tuples; its **New**, **Enc**, and **Vf** oracles, together with the public primitive interface—the real permutation when  $b = 1$  and the simulator when  $b = 0$ —are instantiated for TW128 in Section 4.4. We write the user index as  $d$  and the associated data input as  $A$ , where [BT16] writes  $i$  and  $H$ , and take the absolute value of their signed advantage so that

$$\begin{aligned} \text{Adv}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A}) &= |2 \Pr[\text{G}_{\text{SE}}^{\text{mu-ind}}(\mathcal{A}) \text{ returns } 1] - 1| \\ &= |\Pr[\mathcal{A}^{\text{Enc, Vf}, b=1} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Enc, Vf}, b=0} \Rightarrow 1]| \end{aligned}$$

the second equality holding because  $b$  is uniform. Thus  $\text{Adv}_{\text{SE}}^{\text{mu-ind}}$  is the two-game distinguishing advantage between the real  $(\text{Enc}, \text{Ver})$  pair and the ideal  $(\text{Rand}, \text{Replay})$  pair instantiated per user. The single-user case  $u = 1$  recovers the all-in-one AE notion of Section 2.3, augmented with the public primitive interface (the real permutation in the real world and the simulator in the ideal world).

**Per-user nonce uniqueness.** Nonce uniqueness is enforced per user via the set  $V$ : the same nonce may appear under distinct users but not twice under the same user. This is strictly weaker than the global nonce-uniqueness requirement that would force each nonce value  $N$  to appear under at most one user.

## 2.5 Committing and Context Discoverability Notions

[BH22] formalize a family of committing security notions for nonce-based AEAD schemes, parameterized by how much of the encryption input is committed and by whether commitment is established through decryption (the D-notions) or through encryption agreement (the E-notions). We recall their definitions and the  $s$ -way generalization, the complementary *context discoverability* notion of [MLGR23], and then frame the root tag itself as a hash function whose [RS04] security—collision, second-preimage, and preimage resistance—captures the binding and preimage properties expected of a compact commitment. The AEAD committing and discoverability notions are then proved from the same tag analysis in Sections 5 and 6.

**What-is-committed functions.** For  $\ell \in \{1, 3, 4\}$ , the *what-is-committed* function  $\text{WiC}_\ell$  projects an encryption input  $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$  to the portion it should commit:

$$\begin{aligned}\text{WiC}_1(K, N, A, M) &= K, \\ \text{WiC}_3(K, N, A, M) &= (K, N, A), \\ \text{WiC}_4(K, N, A, M) &= (K, N, A, M).\end{aligned}$$

**CMTDs- $\ell$  and CMTs- $\ell$  games.** For  $s \geq 2$ , [BH22, Fig. 5] defines the  $s$ -way committing games on tuples  $(K_i, N_i, A_i, M_i)_{i=1}^s$  with pairwise distinct  $\text{WiC}_\ell$ -images, all entries in the scheme's syntactic domain and none equal to  $\perp$ . The  $s$ -way CMTDs- $\ell$  (decryption) game has the adversary output a ciphertext  $C$  alongside the tuples and returns 1 if  $\text{Dec}_{K_i}(N_i, A_i, C) = M_i$  for every  $i$ ; the  $s$ -way CMTs- $\ell$  (encryption) game has the adversary output the tuples alone and returns 1 if all encryptions  $\text{Enc}_{K_i}(N_i, A_i, M_i)$  are equal. Advantages  $\text{Adv}_{\text{SE}}^{\text{CMTDs-}\ell}(\mathcal{A})$  and  $\text{Adv}_{\text{SE}}^{\text{CMTs-}\ell}(\mathcal{A})$  are the probabilities that the adversary wins the respective game; the parameter  $s$  is suppressed from the notation when clear from context. The two-tuple case  $s = 2$  is the original CMTD- $\ell$ /CMT- $\ell$  pair of [BH22, Fig. 4]. These definitions are instantiated for TW128 in Section 4.1, with the public permutation lift deferred to that section.

**The root tag as a hash.** Many uses of commitment store, transmit, or compare a short tag rather than the full ciphertext, and ask that the short string alone be binding—the compactly committing AEAD goal introduced by [GLR17] for message franking. TW128 ciphertexts are written  $C \parallel T$  with  $T$  the  $\tau_{\text{root}}$ -bit root tag, and we capture this goal directly, by treating the map from the encryption input to the tag as a hash function: write  $H(K, N, A, M)$  for the root tag  $T$  of  $\text{Enc}_K(N, A, M) = C \parallel T$ , the TW128 wrapper that discards the ciphertext body and returns only the tag. Keyed by the permutation, this is a hash family, and we ask that it satisfy the hash-function security notions of [RS04], over the TW128 input domain:

**Collision resistance (Coll).** No adversary finds distinct inputs  $X_1, \dots, X_s$  (any  $s \geq 2$ ) with  $H(X_1) = \dots = H(X_s)$ .

**Always second-preimage resistance (aSec).** For the fixed deployed permutation, given a challenge input  $X$ , no adversary finds  $X' \neq X$  with  $H(X') = H(X)$ .

**Everywhere preimage resistance (ePre).** For a target tag  $T^*$  fixed in advance, no adversary finds  $X$  with  $H(X) = T^*$ .

Their advantages  $\text{Adv}_{\text{TW128}}^{\text{Coll}}$ ,  $\text{Adv}_{\text{TW128}}^{\text{aSec}}$ ,  $\text{Adv}_{\text{TW128}}^{\text{ePre}}$  are the respective success probabilities, instantiated for TW128 in Section 4.1. Coll is the binding property and ePre the preimage property of the tag taken as a compact commitment to  $(K, N, A, M)$ —the two properties a cryptographic hash commitment provides. Of the [RS04] second-preimage notions, collision resistance already implies the everywhere and average-case forms (eSec and Sec), so we prove the always-form aSec, the one it does not imply, and with a sharper bound. The [BH22] committing notions are the collision-facing AEAD projection of the same tag analysis: a winning opening gives distinct inputs sharing a tag, while the dedicated CMTDs-4 proof uses the all-bodies-equal game structure to avoid the leaf-collision slack of the general hash collision theorem (Sections 5.2 and 6).

**Relations.** [BH22, Figs. 4 and 5] record the implications among the notions. For the D-notions we use:

$$\text{CMTDs-4} \Rightarrow \text{CMTDs-}\ell \text{ for } \ell \in \{1, 3\}$$

(both immediate: pairwise distinct keys ( $\ell = 1$ ) or pairwise distinct  $(K, N, A)$  triples ( $\ell = 3$ ) are *a fortiori* pairwise distinct full tuples, so every CMTDs- $\ell$  winner is a CMTDs-4 winner). The E-notions are bounded directly in Corollary 6 by the same final-root candidate-collision argument, rather than through an encryption reduction.

**Context discoverability.** Where committing security forbids two contexts from colliding on one ciphertext, *context discoverability security* [MLGR23] forbids an adversary from *finding* a decrypting context for a target ciphertext: it is to committing security what preimage resistance is to collision resistance. [MLGR23, Fig. 5] formalize it as the game  $\text{CDY}[\text{ts}, S]$ , in which a query-independent context selector  $S$ —a randomized algorithm taking no input—produces a target ciphertext together with the context components named by a target specifier  $\text{ts} \subseteq \{k, n, a\}$ , and the adversary, given the revealed components, supplies the remaining components of a decryption context  $(K, N, A)$  and wins if the target ciphertext decrypts under it without error. The *restricted* notion  $\text{CDY}^*[\text{ts}]$  requires success against every query-independent selector  $S$ ; equivalently, an adversary’s restricted advantage is no larger than its success probability against any fixed selector. Its three per-component variants  $\text{CDY}_k^*$ ,  $\text{CDY}_n^*$ ,  $\text{CDY}_a^*$  pin two of the three context components and have the adversary supply the third ( $\text{ts} = \{n, a\}, \{k, a\}, \{k, n\}$  respectively). In every nontrivial variant the selector fixes the *entire* target ciphertext; the full-reveal case  $\text{ts} = \{k, n, a\}$  hides no context component and is excluded from Theorem 6. [MLGR23] show that schemes which delegate authenticity to a non-preimage-resistant MAC, including CCM, EAX, SIV, GCM, and OCB3, admit efficient context discovery attacks; the preimage character of the notion is what the root tag preimage analysis addresses.

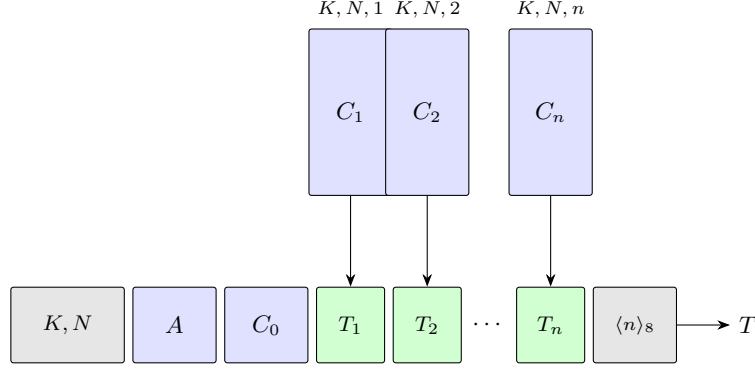
The preimage resistance of  $H$  (ePre, above) is the tag-level preimage notion that the [MLGR23] context discoverability notion is the ciphertext-level counterpart of; the two are distinct, and TW128 is shown to satisfy both (Theorems 5 and 6).

### 3 The TW128 Construction

#### 3.1 Design Overview

TW128 is a nonce-based authenticated encryption scheme with associated data, built as a two-level tree of duplex objects over the KECCAK- $p[1600, 12]$  permutation. A single *root* duplex, initialized with the key  $K$  and nonce  $N$ , absorbs the associated data  $A$  when it is nonempty, emits keystream for the first plaintext chunk  $M_0$  of at most  $\beta$  bytes, and absorbs the resulting ciphertext chunk  $C_0$ ; when  $A$  is empty the associated data phase is elided and the initialization call itself emits the initial  $M_0$  keystream; the remaining plaintext is partitioned into *leaf* chunks  $M_1, \dots, M_n$ , each handled by an independent leaf duplex initialized with  $(K, N, j)$ . The leaf duplex emits the keystream for  $M_j$ , absorbs the resulting ciphertext chunk  $C_j$ , and produces a  $\pi_{\text{leaf}}$ -bit leaf tag  $T_j$ . When  $n \geq 1$  the leaf tags, together with the eight-byte little-endian leaf count  $\langle n \rangle_8$ , are absorbed into the root duplex, which emits a single  $\tau_{\text{root}}$ -bit root tag  $T$ ; when  $n = 0$  the aggregation phase is elided and the closing message phase call of the root chunk emits  $T$  directly, mirroring the way a leaf duplex emits its leaf tag. The construction is depicted in Figure 1.

Three design properties motivate the tree shape. First, the leaf duplexes operate on disjoint state and consume disjoint inputs once  $(K, N)$  is fixed, so leaves  $1, \dots, n$  can be evaluated in parallel. The transcript fixes the leaf decomposition and order, but not the number of leaves an implementation evaluates at once; scalar, SIMD, and multicore implementations therefore produce the same ciphertexts and tags. Section 7 quantifies the resulting throughput on AMD64 and ARM64 hardware. Second, every duplex operates on a bounded 1600-bit state regardless of message length, and the leaf tags are aggregated as a stream rather than buffered in full (Section 3.5), so the working memory is constant



**Figure 1:** Schematic of TW128 for  $n \geq 1$  leaves. The root duplex (bottom row) is initialized with  $(K, N)$  and absorbs in turn the associated data  $A$ , the root ciphertext chunk  $C_0$ , each leaf tag  $T_1, \dots, T_n$ , and the eight-byte little-endian leaf count  $\langle n \rangle_8$ ; it emits the root tag  $T$ . Each leaf duplex (top row) is initialized with  $(K, N, j)$ , emits keystream for plaintext chunk  $M_j$ , absorbs the resulting ciphertext chunk  $C_j$ , and produces  $T_j$ . Leaves can be evaluated in parallel. The ciphertext chunks  $C_0, C_1, \dots, C_n$  are obtained by XORing the keystream emitted by the corresponding duplex with the plaintext chunks  $M_0, M_1, \dots, M_n$ . When  $n = 0$  the diagram reduces to the bottom row without any  $T_j$  boxes or aggregation block: the aggregation phase is elided and the closing message phase call on  $C_0$  emits the root tag  $T$  directly. When the associated data is empty the  $A$  block is absent: the associated data phase is elided and the initialization call emits the initial  $M_0$  keystream directly.

in  $\|M\|$  at a fixed degree of parallelism, scaling only with the number of leaves evaluated simultaneously. Third, the explicit aggregation of leaf tags through a single deterministic root transcript is the AEAD analogue of KangarooTwelve’s final-node hashing of per-chunk chaining values. In TW128, the chaining values are the leaf tags and the final digest is the root tag. This makes the root tag the natural hash point for the full  $(K, N, A, M)$  context; the formal hash, committing, and discoverability analyses appear in Sections 5 and 6.

### 3.2 Parameters and Domain

Table 2 lists the user-visible and derived scheme parameters. The implementation uses the same byte length for the root tag and each leaf tag, but the proofs of Sections 5 and 6 keep  $\tau_{\text{root}}$  and  $\tau_{\text{leaf}}$  as separate symbols.

The bound  $M_{\text{max}} = 8183 \cdot 2^{64}$  bytes is the largest plaintext whose leaf index fits in  $\langle \cdot \rangle_8$ : writing  $\text{leaves}(M) = \max(0, \lceil \|M\|/\beta \rceil - 1)$ , every supported plaintext satisfies  $0 \leq \text{leaves}(M) \leq 2^{64} - 1$ , so leaf indices lie in  $\{1, \dots, 2^{64} - 1\}$ . The associated data bound  $A_{\text{max}}$  is a fixed scheme parameter taken equal to  $M_{\text{max}}$  unless stated otherwise. Inputs exceeding these bounds are outside the scheme domain; every game in Section 4 rejects domain-violating queries.

The derived per-call absorption bound  $\rho$  follows the [BDPVA11] analysis: with  $r = 1344$  and  $\text{pad10}^*1$  padding,  $\rho_{\text{max}}(\text{pad10}^*1, r) = r - 2 = 1342$  bits. The byte-aligned data field shares this allotment with the four-bit phase suffix of Section 3.4, so  $\rho$  is the largest integer satisfying  $8\rho + 4 \leq 1342$ , giving  $\rho = 167$  data bytes (and  $8\rho + 4 = 1340 \leq 1342$ ).

### 3.3 Parameter Choice Rationale

The four free parameters  $c$ ,  $k$ ,  $\tau_{\text{root}} = \tau_{\text{leaf}}$ , and  $\beta$  (the rate  $r = b - c$  is then determined by the capacity) are chosen to give 128-bit single-key security against every term of the

**Table 2:** TW128 parameters.

| Parameter            | Value                     | Description                                             |
|----------------------|---------------------------|---------------------------------------------------------|
| $k$                  | 256                       | Key length (bits)                                       |
| $\nu$                | 256                       | Nonce length (bits)                                     |
| $\tau_{\text{root}}$ | 256                       | Root tag length (bits)                                  |
| $\tau_{\text{leaf}}$ | 256                       | Leaf tag length (bits)                                  |
| $\beta$              | 8183                      | Plaintext chunk size (bytes)                            |
| $M_{\text{max}}$     | $8183 \cdot 2^{64}$ bytes | Maximum plaintext length                                |
| $A_{\text{max}}$     | $8183 \cdot 2^{64}$ bytes | Maximum associated data length                          |
| $b$                  | 1600                      | Permutation state width (bits)                          |
| $n_r$                | 12                        | Permutation rounds                                      |
| $r$                  | 1344                      | Sponge rate (bits)                                      |
| $c$                  | 256                       | Sponge capacity (bits)                                  |
| $\rho$               | 167                       | Maximum byte-aligned absorption per duplex call (bytes) |

concrete bounds derived in Section 5.3 while keeping per-byte cost low; the nonce width  $\nu$  is fixed separately, by the headroom the wide permutation affords. Specifically:

- $c = 256$ . The dominant single-key term is the [BDPVA08] flat sponge loss

$$\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1},$$

which reaches the  $2^{-128}$  threshold at  $N_{\text{tot}} \approx 2^{(c-127)/2} \approx 2^{64.5}$  KECCAK- $p[1600, 12]$  calls; the headline cap of  $N_{\text{tot}} \leq 2^{63}$  used in Section 5.4 keeps this term at most  $2^{-131} + 2^{-194}$ , and hence below  $2^{-130}$ . Setting  $c = 256$  thereby delivers 128-bit single-key security on this term. Under the mu-ind bound of Section 5.2 the per-user security margin degrades by  $\log D$  bits in  $D$  users; the concrete  $D$ -indexed bound appears in Section 5.4.

- $r = 1344$ . With state width  $b = 1600$  fixed by the KECCAK- $p[1600, 12]$  choice,  $r = b - c = 1344$  is the largest rate compatible with the chosen capacity. The 12-round KECCAK- $p[1600, 12]$  permutation matches KangarooTwelve’s choice [BDP<sup>+</sup>16]; TW128 adopts that established tree-hashing parameter rather than introducing a new reduced-round setting.
- $\tau_{\text{root}} = \tau_{\text{leaf}} = 256$ . Matching the capacity ensures that the per-tag collision terms in the authenticity and committing bounds (Section 5.3) are dominated by the capacity birthday rather than the tag length.
- $k = 256$ . Matching the capacity ensures that the per-user key guess

$$\text{KeyGuess}_k(N_{\text{prim}}) = N_{\text{prim}}/2^k$$

remains below the capacity birthday at the target  $D$  of Section 5.4.

- $\beta = 8183$ . The chunk size is set to  $49\rho = 49 \cdot 167$  bytes, the largest multiple of the per-call absorption bound  $\rho$  not exceeding  $2^{13}$  bytes. Taking  $\beta$  to be an exact multiple of  $\rho$  makes every full leaf split into exactly 49 fully utilized message duplex calls with no short final block, so a full leaf costs one initialization duplex call plus 49 message duplex calls. This amortizes the fixed per-leaf initialization cost over a large chunk while keeping the leaf state working set ( $8183 < 2^{13}$  bytes) small enough for L1 cache; Section 7.3 reports the resulting cycles-per-byte figures.

- $\nu = 256$ . The nonce width is not constrained by the concrete bounds, which assume a nonce-respecting adversary; it is set generously because the wide permutation makes a wide nonce nearly free. The root initialization absorbs  $p_{\text{root}} \parallel K \parallel N$ , 560 bits in all, in a single duplex call, comfortably within the 1336-bit ( $\rho = 167$ -byte) data field (Section 3.2), so widening the nonce to 256 bits adds neither a permutation call nor any per-message cost. The width also removes any practical concern over random nonce collisions: a uniformly random 256-bit nonce repeats over  $q$  messages with probability at most  $\binom{q}{2}/2^{256}$ , negligible at the work cap of Section 5.4. Deployments that do not require a full-width random nonce lose nothing by the choice: a narrower nonce—a 96-bit packet counter, say—is carried in the fixed  $\nu$ -bit field by zero-padding, so a single mode serves both random nonce and counter-based settings with no change of parameters.

### 3.4 Domain Separation

Distinct duplex instances and distinct phases within a duplex are kept separate at the transcript level by two mechanisms. First, the initialization strings carry a 6-byte ASCII prefix and, for leaves, an eight-byte little-endian chunk index:

$$p_{\text{root}} = \text{"TW128R"} \in \{0, 1\}^{48},$$

$$p_{\text{leaf}} = \text{"TW128L"} \in \{0, 1\}^{48}.$$

The root duplex's first absorbed string is  $p_{\text{root}} \parallel K \parallel N$ , and the  $j$ -th leaf duplex's first absorbed string is  $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$ . Together with the fixed  $k$ -bit and  $\nu$ -bit widths of  $K$  and  $N$ , these prefixes separate root from leaf transcripts and distinguish leaves with different chunk indices.

Second, every byte-aligned absorption carries a four-bit phase suffix. We identify each suffix with the four-bit string given by its Table 3 value written least significant bit first, so that a duplex call absorbing a byte-aligned block  $\sigma$  under a phase suffix  $\sigma_{\text{ph}}$  feeds the duplex interface of Section 2.2 the input  $\sigma \parallel \sigma_{\text{ph}}$  before `pad10*1` padding. The seven phase suffixes are summarized in Table 3; they correspond to initialization, non-final and final associated data blocks, non-final and final message blocks, and non-final and final aggregation blocks. The seven values are pairwise distinct, so any two transcripts that differ in the phase of any block also differ in the flattened sponge input.

**Table 3:** Phase suffixes (4 bits each, appended LSB-first).

| Name                    | Hex | Use                                        |
|-------------------------|-----|--------------------------------------------|
| $\sigma_{\text{init}}$  | 0xC | End of initialization (the sole init call) |
| $\sigma_{\text{ad}}^-$  | 0x9 | Non-final associated data block            |
| $\sigma_{\text{ad}}^+$  | 0xD | Final associated data block                |
| $\sigma_{\text{msg}}^-$ | 0xA | Non-final message block                    |
| $\sigma_{\text{msg}}^+$ | 0xE | Final message block                        |
| $\sigma_{\text{agg}}^-$ | 0xB | Non-final aggregation block                |
| $\sigma_{\text{agg}}^+$ | 0xF | Final aggregation block (emits root tag)   |

The combined effect of the initialization prefixes and the phase suffixes is that the flattened sponge input of any TW128 duplex call uniquely determines the duplex instance, the leaf index when applicable, the phase, and the duplex-call position within that phase. Definition 1 (Well-formed TW128 transcript prefixes), Lemma 4 (Transcript Injectivity), and its structural corollaries formalize this parsing fact for the proofs of Sections 5 and 6 and Corollary 6.

### 3.5 Encryption

Algorithm 2 specifies  $\text{Enc}_K(N, A, M)$  in terms of two helper procedures, **ABSORB** and **STREAMENC**, defined in Algorithm 1. These helpers sit below the tree layer: only Algorithms 2 and 3 split messages or ciphertexts into  $\beta$ -byte root and leaf chunks, while the helpers split an already chosen byte string into  $\rho$ -byte duplex blocks. **ABSORB** drives a duplex through a sequence of such blocks with non-final  $\sigma^-$  and final  $\sigma^+$  suffix tags, returning the output of the final call; the empty-input case yields a single empty duplex block carrying the  $\sigma^+$  tag. The streaming helpers use the same empty-input convention, so a zero-length message string still performs one final message phase duplex call and returns a zero-length body. Algorithm 2 invokes **ABSORB** for the associated data phase only when  $A \neq \varepsilon$ ; when  $A = \varepsilon$  the phase is elided and the root  $\sigma_{\text{init}}$  call emits the initial  $M_0$  keystream directly. **STREAMENC** encrypts a plaintext byte string one duplex block at a time, XORing each block with the current keystream, absorbing the resulting ciphertext duplex block back into the duplex with the appropriate phase tag, and using the duplex's output as the next keystream. This is the duplex-level presentation of the overwrite mechanism of [BDPVA11, §6.2 and Algorithm 5]: when the current rate prefix emitted as keystream is  $Z$ , XOR-absorbing  $m \oplus Z$  has the same update on the message bytes as overwriting that prefix with  $m$ , with the phase tag still carried as part of the duplex input. The decryption helper **STREAMDEC** is identical to **STREAMENC** with the roles of the plaintext and ciphertext duplex blocks exchanged: it takes ciphertext duplex blocks  $c_i$  as input, recovers each plaintext duplex block as  $m_i = c_i \oplus Z$ , absorbs the same  $c_i$  under the same phase suffix, and returns  $(m_1 \parallel \dots \parallel m_t, Z')$ ; we omit its pseudocode. In both helpers, each duplex call absorbs a duplex block followed by its phase suffix— $\sigma^-$  for a non-final block and  $\sigma^+$  for the final block—and requests the output bits the caller consumes next:  $\ell$  (possibly zero) on the final call, and the next duplex block's keystream length on non-final calls. Each call is the duplex interface of Section 2.2 applied to the suffixed input of Section 3.4.

The encryption procedure (Algorithm 2) instantiates the root duplex and obtains the initial keystream for the root chunk  $M_0$ : when  $A \neq \varepsilon$  it absorbs the associated data  $A$ , and when  $A = \varepsilon$  it elides the associated data phase and takes the keystream from the  $\sigma_{\text{init}}$  call itself. It then encrypts  $M_0$  while absorbing the resulting ciphertext chunk  $C_0$ . When  $n = 0$  the closing  $\sigma_{\text{msg}}^+$  call of this root chunk requests  $\tau_{\text{root}}$  output bits, which are the root tag  $T$ , and the aggregation phase is elided. When  $n \geq 1$  it independently encrypts each leaf chunk  $M_j$  while absorbing  $C_j$ , producing the leaf tag  $T_j$ ; the leaf tags together with  $\langle n \rangle_8$  are then absorbed into the root duplex, and the final  $\sigma_{\text{agg}}^+$  call emits the root tag  $T$ . The ciphertext is the concatenation  $C_0 \parallel C_1 \parallel \dots \parallel C_n$ . Thus this section writes  $C$  for the ciphertext body and  $C \parallel T$  for the full AEAD ciphertext returned by  $\text{Enc}_K(N, A, M)$ .

The initial keystream request for a message string  $X$  is  $8 \min(\|X\|, \rho)$ , the bit length of its first duplex block. For the root chunk  $M_0$ , that value is emitted by the closing  $\sigma_{\text{ad}}^+$  call when  $A \neq \varepsilon$ , and by the root  $\sigma_{\text{init}}$  call itself when  $A = \varepsilon$  and the associated data phase is elided. Each leaf's  $\sigma_{\text{init}}$  call likewise emits  $8 \min(\|M_j\|, \rho)$  keystream bits for the first duplex block of  $M_j$ , so the empty associated data root  $\sigma_{\text{init}}$  call plays exactly the role a leaf  $\sigma_{\text{init}}$  call already plays. When the corresponding message string is empty, the requested keystream length is zero and the next message phase duplex call absorbs the empty ciphertext duplex block carrying  $\sigma_{\text{msg}}^+$ . The final  $\sigma_{\text{msg}}^+$  call in the root chunk requests  $\tau_{\text{root}}$  output bits when  $n = 0$ , namely the root tag  $T$  emitted by the elided-aggregation path, and zero output bits when  $n \geq 1$  because the next duplex operation on the root is then the first aggregation block; the corresponding call on a leaf requests  $\tau_{\text{leaf}}$  bits, namely the leaf tag. When  $n \geq 1$  the final  $\sigma_{\text{agg}}^+$  call requests  $\tau_{\text{root}}$  bits, which is the root tag  $T$ .

The leaf loop is annotated **in parallel** to make explicit that the  $n$  leaf computations are mutually independent; Section 7.1 discusses the SIMD realizations. The annotation is an implementation freedom, not a mode parameter: evaluating leaves serially, in SIMD batches, or across cores leaves the transcript and wire format unchanged.

---

**Algorithm 1** TW128 helper procedures.

---

```

1: procedure ABSORB( $D, s, \sigma^-, \sigma^+, \ell$ )
2:   Partition  $s$  into duplex blocks  $s_1, \dots, s_t$  of exactly  $\rho$  bytes each, except that the
   last block may be shorter.
3:   Require  $t \geq 1$ ; the empty case yields the single empty duplex block  $s_1 = \varepsilon$ .
4:   for  $i \leftarrow 1$  to  $t - 1$  do
5:      $D.\text{duplexing}(s_i \parallel \sigma^-, 0)$ 
6:   end for
7:   return  $D.\text{duplexing}(s_t \parallel \sigma^+, \ell)$ 
8: end procedure

9: procedure STREAMENC( $D, m, Z, \sigma^-, \sigma^+, \ell$ )
10:  Partition  $m$  into duplex blocks  $m_1, \dots, m_t$  of exactly  $\rho$  bytes each, except that the
   last block may be shorter.
11:  Require  $t \geq 1$ ; if  $m = \varepsilon$ , set  $t = 1$  and  $m_1 = \varepsilon$ .
12:  for  $i \leftarrow 1$  to  $t - 1$  do
13:     $c_i \leftarrow m_i \oplus Z$ 
14:     $Z \leftarrow D.\text{duplexing}(c_i \parallel \sigma^-, 8\|m_{i+1}\|)$ 
15:  end for
16:   $c_t \leftarrow m_t \oplus Z$ 
17:   $Z' \leftarrow D.\text{duplexing}(c_t \parallel \sigma^+, \ell)$ 
18:  return  $(c_1 \parallel \dots \parallel c_t, Z')$ 
19: end procedure

```

---

531 **Streaming aggregation.** The string  $G = T_1 \parallel \dots \parallel T_n$  in Algorithms 2 and 3 names the  
532 leaf tags only for specification. The closing ABSORB consumes  $G \parallel \langle n \rangle_8$  as  $\rho$ -byte duplex  
533 blocks in leaf order and advances the root state after each, so an implementation need  
534 not hold  $G$  in full. It can absorb leaf tags in order as they become available, or buffer  
535 completed out-of-order leaves until the next in-order tag or SIMD batch can be absorbed,  
536 keeping only a bounded number of tags at a time. With the bounded 1600-bit state  
537 of every duplex, this keeps the working memory constant in  $\|M\|$  for a fixed degree of  
538 parallelism, growing only with the number of leaves evaluated at once rather than with  
539 the message length; Section 7.1 describes the batched realization.

### 540 3.6 Decryption

541 Algorithm 3 specifies  $\text{Dec}_K(N, A, C \parallel T)$ . The procedure mirrors Enc at the duplex block  
542 level: each ciphertext duplex block  $c_i$  is absorbed into the appropriate duplex (root or leaf)  
543 under the corresponding  $\sigma_{\text{msg}}^-$  or  $\sigma_{\text{msg}}^+$  suffix, and the plaintext duplex block is recovered as  
544  $m_i = c_i \oplus Z^{(i)}$ , writing  $Z^{(i)}$  for the value the running keystream variable  $Z$  of STREAMDEC  
545 holds when duplex block  $i$  is processed ( $i < t$  inside the loop,  $i = t$  after it), the same value  
546 the encryption procedure would have produced. The procedure absorbs ciphertext bytes  
547 before the final tag comparison, so the verification transcript is determined by  $(K, N, A, C)$   
548 for every in-domain ciphertext, accepting or not. This property is used by the direct  
549 mu-ind proof to argue that encryption-first and verification-first transcript classes are well  
550 defined regardless of the tag-comparison outcome.

551 The  $n = 0$  and empty-body cases mirror their encryption counterparts (Section 3.5):  
552 for  $n = 0$  the aggregation phase is elided and the closing  $\sigma_{\text{msg}}^+$  call of the root chunk emits  
553 the candidate tag  $T'$  directly, and an empty body still performs the single empty  $\sigma_{\text{msg}}^+$  call  
554 of Algorithm 1, requesting  $\tau_{\text{root}}$  output bits.

**Algorithm 2**  $\text{Enc}_K(N, A, M)$ .**Require:**  $K \in \{0, 1\}^k$ ,  $N \in \{0, 1\}^\nu$ ,  $A$  with  $\|A\| \leq A_{\max}$ ,  $M$  with  $\|M\| \leq M_{\max}$ .**Ensure:**  $C \parallel T$  with  $\|C\| = \|M\|$  and  $T \in \{0, 1\}^{\tau_{\text{root}}}$ .

```

1:  $M_0 \leftarrow$  first  $\min(\|M\|, \beta)$  bytes of  $M$  (possibly  $\varepsilon$ ).
2: Partition the remaining suffix into chunks  $M_1, \dots, M_n$  of exactly  $\beta$  bytes each, except
   that the last chunk may be shorter but nonempty, so  $n = \text{leaves}(M)$ .
3:  $D_{\text{root}} \leftarrow$  new Duplex
4: if  $A = \varepsilon$  then
5:    $Z \leftarrow D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 8 \min(\|M_0\|, \rho))$ 
6: else
7:    $D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 0)$ 
8:    $Z \leftarrow \text{ABSORB}(D_{\text{root}}, A, \sigma_{\text{ad}}^-, \sigma_{\text{ad}}^+, 8 \min(\|M_0\|, \rho))$ 
9: end if
10: if  $n = 0$  then
11:    $(C_0, T) \leftarrow \text{STREAMENC}(D_{\text{root}}, M_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{root}})$ 
12:   return  $C_0 \parallel T$ 
13: end if
14:  $(C_0, \_) \leftarrow \text{STREAMENC}(D_{\text{root}}, M_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, 0)$ 
15: for  $j \leftarrow 1$  to  $n$  in parallel do
16:    $D_j \leftarrow$  new Duplex
17:    $Z_j \leftarrow D_j.\text{duplexing}(p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8 \parallel \sigma_{\text{init}}, 8 \min(\|M_j\|, \rho))$ 
18:    $(C_j, T_j) \leftarrow \text{STREAMENC}(D_j, M_j, Z_j, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{leaf}})$ 
19: end for
20:  $G \leftarrow T_1 \parallel \dots \parallel T_n$ .
21:  $T \leftarrow \text{ABSORB}(D_{\text{root}}, G \parallel \langle n \rangle_8, \sigma_{\text{agg}}^-, \sigma_{\text{agg}}^+, \tau_{\text{root}})$ 
22: return  $C_0 \parallel C_1 \parallel \dots \parallel C_n \parallel T$ 

```

**Correctness.** For every  $K \in \{0, 1\}^k$ ,  $N \in \{0, 1\}^\nu$ ,  $A$  with  $\|A\| \leq A_{\max}$ , and  $M$  with  $\|M\| \leq M_{\max}$ ,  $\text{Dec}_K(N, A, \text{Enc}_K(N, A, M)) = M$ . This follows by inspection: every duplex call in  $\text{Dec}_K$  receives the same input as the corresponding call in  $\text{Enc}_K$  (each ciphertext duplex block is absorbed under the same phase tag in both directions), so the duplex states and the recomputed root tag  $T'$  exactly match. The constant-time comparison  $T' = T$  succeeds, and the recovered  $m_i = c_i \oplus Z^{(i)} = (m_i \oplus Z^{(i)}) \oplus Z^{(i)} = m_i$  at every position.

## 4 Security Model

This section fixes the security model used throughout the paper. The scheme syntax and the authenticated encryption and committing security definitions of Section 2 are instantiated here for TW128 in the lifted public permutation model (Section 4.1), and we define the intermediate random oracle world through which every bound is first proved (Section 4.2). Section 4.3 fixes the resource accounting that all later bounds reference, and Section 4.4 instantiates the multi-user indistinguishability game for TW128.

### 4.1 Public Permutation Games

The lifted AE-style public permutation advantages below compare a real execution, with sampled  $\pi \leftarrow \$ \text{Perm}(\{0, 1\}^{1600})$  and direct primitive interface  $(\pi, \pi^{-1})$ , to a simulator-ideal execution. In the simulator-ideal execution, the construction interface is the ideal interface specified by the game and the public primitive interface is  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ . The simulator  $\mathcal{S}^{\text{RO}_{\text{flat}}}$  together with its inverse  $\mathcal{S}^{-1, \text{RO}_{\text{flat}}}$  is the public primitive simulator fixed in Section B; both directions share an internal lazy table for the random oracle  $\text{RO}_{\text{flat}}$  of Section 4.2. The

---

**Algorithm 3**  $\text{Dec}_K(N, A, C \parallel T)$ .

---

**Require:**  $K, N, A$  as in Algorithm 2, ciphertext  $C$  with  $\|C\| \leq M_{\max}$ , candidate tag  $T \in \{0, 1\}^{\tau_{\text{root}}}$ .

**Ensure:**  $M$  with  $\|M\| = \|C\|$ , or  $\perp$ .

```

1:  $C_0 \leftarrow$  first  $\min(\|C\|, \beta)$  bytes of  $C$  (possibly  $\varepsilon$ ).
2: Partition the remaining suffix into chunks  $C_1, \dots, C_n$  of exactly  $\beta$  bytes each, except
   that the last chunk may be shorter but nonempty, so  $n = \text{leaves}(C)$ .
3:  $D_{\text{root}} \leftarrow$  new Duplex
4: if  $A = \varepsilon$  then
5:    $Z \leftarrow D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 8 \min(\|C_0\|, \rho))$ 
6: else
7:    $D_{\text{root}}.\text{duplexing}(p_{\text{root}} \parallel K \parallel N \parallel \sigma_{\text{init}}, 0)$ 
8:    $Z \leftarrow \text{ABSORB}(D_{\text{root}}, A, \sigma_{\text{ad}}^-, \sigma_{\text{ad}}^+, 8 \min(\|C_0\|, \rho))$ 
9: end if
10: if  $n = 0$  then
11:    $(M_0, T') \leftarrow \text{STREAMDEC}(D_{\text{root}}, C_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{root}})$ 
12:   return  $M_0$  if  $T' = T$  (compared in constant time), else  $\perp$ .
13: end if
14:  $(M_0, \_) \leftarrow \text{STREAMDEC}(D_{\text{root}}, C_0, Z, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, 0)$ 
15: for  $j \leftarrow 1$  to  $n$  in parallel do
16:    $D_j \leftarrow$  new Duplex
17:    $Z_j \leftarrow D_j.\text{duplexing}(p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8 \parallel \sigma_{\text{init}}, 8 \min(\|C_j\|, \rho))$ 
18:    $(M_j, T_j) \leftarrow \text{STREAMDEC}(D_j, C_j, Z_j, \sigma_{\text{msg}}^-, \sigma_{\text{msg}}^+, \tau_{\text{leaf}})$ 
19: end for
20:  $G \leftarrow T_1 \parallel \dots \parallel T_n$ .
21:  $T' \leftarrow \text{ABSORB}(D_{\text{root}}, G \parallel \langle n \rangle_8, \sigma_{\text{agg}}^-, \sigma_{\text{agg}}^+, \tau_{\text{root}})$ 
22: if  $T' = T$  (compared in constant time) then
23:   return  $M_0 \parallel M_1 \parallel \dots \parallel M_n$ 
24: else
25:   return  $\perp$ 
26: end if

```

---

575 committing security games remain one-world real public permutation success probabilities;  
 576 the simulator appears for them only inside the reductions of Section B.

577 In the real all-in-one AE game, the challenger additionally samples a key  $K \leftarrow$   
 578  $\{0, 1\}^k$  and exposes the construction oracles  $\text{Enc}_K[\pi]$  (the TW128 encryption algorithm  
 579 of Algorithm 2 with the sampled  $\pi$ ) and  $\text{Ver}_K[\pi]$ , defined by  $\text{Ver}_K[\pi](N, A, C, T) = 1$   
 580 if  $\text{Dec}_K[\pi](N, A, C \parallel T) \neq \perp$  and 0 otherwise. The committing security games have no  
 581 challenger-sampled key.

582 **Validity convention.** All construction interface inputs are checked against TW128's public  
 583 syntax and domain before any construction computation or construction-internal primitive  
 584 lookup is performed. An encryption input is valid when  $N \in \{0, 1\}^\nu$ ,  $A$  is a byte string with  
 585  $\|A\| \leq A_{\max}$ , and  $M$  is a byte string with  $\|M\| \leq M_{\max}$ . A verification or decryption input  
 586 is valid when  $N \in \{0, 1\}^\nu$ ,  $A$  is a byte string with  $\|A\| \leq A_{\max}$ , and the full ciphertext  
 587 parses as  $C \parallel T$  with  $\|C\| \leq M_{\max}$  and  $T \in \{0, 1\}^{\tau_{\text{root}}}$ . In the hash and committing games,  
 588 each submitted input  $(K, N, A, M)$  must be a valid encryption input including  $K \in \{0, 1\}^k$ ,  
 589 and a submitted CMTDs ciphertext must be a valid verification/decryption ciphertext.  
 590 Invalid encryption queries are rejected and not recorded, invalid verification queries return  
 591 0, and invalid committing-game outputs make the game return 0. Such rejected inputs  
 592 contribute no construction cost and create no construction transcripts. Throughout, an

593 *accepted* encryption query is one that passes this validity check and any game-specific  
 594 nonce or user check.

595 **All-in-one AE.** The all-in-one authenticated encryption game [RS06] replaces the con-  
 596 struction interface by the ideal pair (Rand, Replay). On each accepted encryption query  
 597  $(N, A, M)$ , Rand samples a uniformly random byte string of length  $\|M\| + \tau_{\text{root}}/8$ , parses  
 598 it as  $C \parallel T$  with  $T \in \{0, 1\}^{\tau_{\text{root}}}$ , records  $(N, A, C, T)$ , and returns  $C \parallel T$ . The oracle  
 599 Replay returns 1 exactly on recorded tuples  $(N, A, C, T)$  and 0 otherwise. The adversary is  
 600 nonce-respecting on encryption queries.

$$601 \quad \text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) = \left| \Pr[\mathcal{A}^{\text{Enc}_K[\pi], \text{Ver}_K[\pi], \pi, \pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{Rand, Replay}, \mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}}} \Rightarrow 1] \right|.$$

602 This lifted all-in-one AE advantage is the single-user case of the multi-user real-vs-ideal  
 603 notion of Section 2.4; Corollary 1 therefore derives it from the direct mu-ind bound of  
 604 Theorem 2.

605 **Hash games.** The collision, second-preimage, and preimage games for the wrapper H  
 606 (Section 2.5) have no challenger-sampled key; all inputs are adversary-supplied. In the  
 607  $s$ -way collision game the adversary outputs  $s$  inputs, and the challenger returns 1 iff they  
 608 are pairwise distinct, in the TW128 domain, and share a root tag:

$$609 \quad \begin{aligned} & \pi \leftarrow \$ \text{Perm}(\{0, 1\}^{1600}), \quad (K_i, N_i, A_i, M_i)_{i=1}^s \leftarrow \$ \mathcal{A}^{\pi, \pi^{-1}}, \\ 610 \quad & \text{return 0 unless the inputs are pairwise distinct and in the TW128 domain,} \\ 611 \quad & \text{return } [\text{H}(K_1, N_1, A_1, M_1) = \dots = \text{H}(K_s, N_s, A_s, M_s)], \end{aligned}$$

612 with advantage  $\text{Adv}_{\text{TW128}}^{\text{Coll}}(\mathcal{A}) = \Pr[\text{the game returns 1}]$ ; no nonce uniqueness is imposed.  
 613 The always second-preimage (aSec) game samples a challenge input  $X$  uniformly from the  
 614 TW128 domain, gives it to  $\mathcal{A}$ , and returns 1 iff  $\mathcal{A}$  outputs  $X' \neq X$  in the domain with  
 615  $\text{H}(X') = \text{H}(X)$ , with advantage  $\text{Adv}_{\text{TW128}}^{\text{aSec}}$ ; following [RS04] the always-notion concerns  
 616 the fixed permutation, here the deployed instance the public permutation model represents  
 617 by a random  $\pi$ . The preimage game fixes a target tag  $T^* \in \{0, 1\}^{\tau_{\text{root}}}$  before the primitive  
 618 sampling and returns 1 iff  $\mathcal{A}(T^*)$  outputs an input  $X$  in the domain with  $\text{H}(X) = T^*$ , the  
 619 advantage  $\text{Adv}_{\text{TW128}}^{\text{ePre}}$  taken as the worst case over query-independent targets.

620 **Committing games.** The [BH22] committing games are instantiated as in Section 2.5,  
 621 with no challenger-sampled key. The  $s$ -way CMTDs- $\ell$  (decryption) game has the adversary  
 622 output a ciphertext  $C \parallel T$  and  $s$  tuples  $(K_i, N_i, A_i, M_i)$  with pairwise distinct  $\text{WiC}_\ell$ -images,  
 623 and returns 1 iff  $\text{Dec}_{K_i}[\pi](N_i, A_i, C \parallel T) = M_i$  for every  $i$ , with the eager length rejection  
 624 justified by Algorithm 3 ( $\text{Dec}_K(N, A, C)$  returns  $\perp$  or a message of length  $\|C\|$ ); its  
 625 advantage is  $\text{Adv}_{\text{TW128}}^{\text{CMTDs-}\ell}(\mathcal{A})$ . The CMTs- $\ell$  (encryption) game has the adversary output  
 626 the  $s$  tuples alone and returns 1 iff the encryptions  $\text{Enc}_{K_i}[\pi](N_i, A_i, M_i)$  are all equal; its  
 627 advantage is  $\text{Adv}_{\text{TW128}}^{\text{CMTs-}\ell}(\mathcal{A})$ . A CMTDs-4 or CMTs-4 winner's tuples share a common  
 628 tag, hence are a collision of H, so Corollary 3 bounds CMTDs-4 through Theorem 7 and  
 629 Corollary 6 forwards the remaining [BH22] notions to the same root-collision bound.

630 **Context discoverability.** The [MLGR23, Fig. 5] context discoverability game (Section 2.5)  
 631 is a ciphertext-level preimage notion, distinct from the preimage resistance of H above. It  
 632 is parameterized by a target specifier  $\text{ts}$  and a query-independent context selector  $S$  that  
 633 produces a target ciphertext  $C^*$  and reveals the  $\text{ts}$ -named components  $\text{rev}$  of a decryption

context; there is no challenger-sampled key.

```

634  $\pi \leftarrow \$ \text{Perm}(\{0, 1\}^{1600}), \quad (C^*, \text{rev}) \leftarrow \$ S[\pi],$ 
635  $(K, N, A) \leftarrow \$ \mathcal{A}^{\pi, \pi^{-1}}(C^*, \text{rev}),$ 
636
637 return 0 if  $C^*$  is not a valid TW128 ciphertext, if  $(K, N, A)$  is outside the domain,
638     or if  $(K, N, A)$  disagrees with  $\text{rev}$  on its ts-components,
639 return  $[\text{Dec}_K[\pi](N, A, C^*) \neq \perp]$ .
```

The restricted advantage  $\text{Adv}_{\text{TW128}}^{\text{CDY}}(\mathcal{A})$  is measured against every query-independent selector  $S$ , matching  $\text{CDY}^*[\text{ts}]$ ; in particular, it is no larger than the success probability against any fixed selector. At  $\text{ts} = \{n, a\}, \{k, a\}, \{k, n\}$ , this gives the per-component variants  $\text{CDY}_k^*, \text{CDY}_n^*, \text{CDY}_a^*$ . Because the selector holds a hidden context, the TW128 bound (Theorem 6) carries a key-test term that the keyless preimage game of  $\mathbf{H}$  does not. Section B lifts the hash and committing bounds of Section 6 to the public permutation statements of Section 5.2.

## 4.2 The TW128 Random Oracle Model

The proofs of Section 6 and Corollary 6, together with the headline reductions of Section 5.2, are carried out through an intermediate random oracle model whose construction interfaces use the [BDPVA11] bridge of Section 2.2 to address a single random oracle. The flat sponge lift of Section B then converts the resulting bounds to the public permutation form.

The model has a single random oracle  $\text{RO}_{\text{flat}} : \{0, 1\}^* \rightarrow \{0, 1\}^\infty$  over the unpadded  $H$ -input domain of the [BDPVA08] simple-padded sponge interface, with lazy sampling: each distinct input receives an independent infinite bit string, and repeated queries return the same string. A direct adversary query is finite-output: on input  $(Y, \ell)$  with finite  $\ell$ , the oracle returns the first  $\ell$  bits of the lazy-sampled value for  $Y$ .

For a well-formed TW128 duplex transcript prefix  $X$  in the language of Definition 1, at one of the construction transcript lookup points formalized by Lemma 6 and Table 9, let  $\text{flat}(X) = I[1344, 1344](\text{raw\_flat}(X))$  be the bridged image of the native flattened input under the [BDPVA11, Theorem 3] preprocessing map of Section 2.2, and define

$$\text{RF}(X) = \text{RO}_{\text{flat}}(\text{flat}(X)).$$

The construction interfaces  $\text{Enc}_K^{\text{RO}}, \text{Dec}_K^{\text{RO}},$  and  $\text{Ver}_K^{\text{RO}}$  are the TW128 algorithms of Section 3 with every root or leaf duplex output replaced by the corresponding requested projection of  $\text{RF}(X)$ ; the projection length is the duplexing call's output parameter  $\ell$ , which may be zero. The bridge and the typed-restriction convention are formalized in Lemma 6; the direct-query key-hit predicate ignores the adversary's requested output length and is formalized as **KeyedTWOut** in Lemma 6.

The random oracle model games are the construction interface variants of Sections 4.1 and 4.4: the primitive interface is replaced by direct adversary access to  $\text{RO}_{\text{flat}}$ , and each construction oracle uses the corresponding  $\text{RO}$  algorithm, the ideal oracles (**Rand**, **Replay**) being unchanged. Under this substitution  $\text{Adv}_{\text{RO}}^{\text{ae}}$  is the all-in-one advantage of Section 4.1 with no simulator on the ideal side; the keyless hash advantages  $\text{Adv}_{\text{RO}}^{\text{Coll}}(\mathcal{B}), \text{Adv}_{\text{RO}}^{\text{aSec}}(\mathcal{B}), \text{Adv}_{\text{RO}}^{\text{ePre}}(\mathcal{B})$  of  $\mathbf{H}$ , the committing  $\text{Adv}_{\text{RO}}^{\text{CMTDs-4}}(\mathcal{B})$ , and, for  $\ell \in \{1, 3, 4\}$ ,  $\text{Adv}_{\text{RO}}^{\text{CMTs-}\ell}(\mathcal{B})$  are the probabilities that the corresponding random oracle hash and committing games return 1, the CMTs challenger computing its deterministic encryption checks with  $\text{Enc}^{\text{RO}}$ . For mu-ind,  $\text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B})$  denotes the Section 4.4 game with the same **New** interface and per-user nonce discipline—user  $d$  using  $(\text{Enc}_{K_d}^{\text{RO}}, \text{Ver}_{K_d}^{\text{RO}})$  when  $b = 1$  and (**Rand**, **Replay**) when  $b = 0$ —equivalently, in the two-game notation of Section 2.4,

$$\text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}) = |\Pr[\mathcal{B}^{\text{Real}^{\text{mu}}, \text{RO}_{\text{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\text{Ideal}^{\text{mu}}, \text{RO}_{\text{flat}}} \Rightarrow 1]|,$$

where the superscripts name the entire multi-user construction interface in the corresponding world. Adversary queries to  $\text{RO}_{\text{flat}}$  are finite-output and counted in  $q_{\text{RO}}(\mathcal{B})$  of Section 4.3. Construction-internal  $\text{RF}(\cdot)$  lookups — those made by  $\text{Enc}_K^{\text{RO}}$ ,  $\text{Dec}_K^{\text{RO}}$ ,  $\text{Ver}_K^{\text{RO}}$ , or a committing challenger — are not.

### 4.3 Resources

Adversary resources are measured by fixed, execution-uniform caps on the number and size of oracle queries. An execution-uniform cap is one that holds on every execution path, including over adversary randomness, oracle randomness, and adaptive query choices. Proofs may also introduce local execution-uniform caps, such as  $q_e$  for the number of accepted encryption queries, solely to index a finite hybrid sequence. Such local caps are part of the adversary’s fixed query bounds, but they are not listed as headline resource parameters unless they affect the final bound.

**$N$ :** the construction interface cost, measured as the number of  $\text{KECCAK-}p[1600, 12]$  calls that the adversary’s construction interface queries induce in the real TW128 computation after the validity convention above has been applied. In ideal games, the same accounting applies to the corresponding real TW128 computation on the valid query, even when an ideal oracle answers by table lookup or without running the construction.  $N$  counts at per-block granularity, summing each root initialization, associated data block, root message block, leaf initialization, leaf message block, and aggregation block across the entire query sequence. Valid AE-style verification computations are counted whether they accept or reject. In the hash, committing, and discoverability games there is no adversary construction oracle: for the H collision game,  $N$  counts the  $s$  deterministic encryptions on the submitted inputs (and the one or two encryptions of the second-preimage and preimage games); for CMTDs- $\ell$ ,  $N$  counts only the deterministic decryption checks performed by the challenger on the common ciphertext; for the context discoverability game,  $N$  counts the worst-case selector’s encryption producing  $C^*$  together with the single deterministic decryption check the challenger performs on  $C^*$ ; while for the CMTs- $\ell$  source games of Corollary 6,  $N$  counts only the deterministic encryption checks actually used by the challenger to compare the  $s$  ciphertexts.

**$N_{\text{prim}}$ :** the number of direct adversary queries to the primitive interface (the pair  $(\pi, \pi^{-1})$  in public permutation games; the simulator’s  $\text{RO}_{\text{flat}}$  table is internal and is not exposed to the application adversary in those games).

**$N_{\text{tot}} = N + N_{\text{prim}}$ :** the total query-sequence cost. The flat sponge lift of Section B is applied at  $N_{\text{tot}}$  because the indistinguishability replacement sees both the construction-internal sponge calls counted by  $N$  and the adversary’s direct primitive-interface queries counted by  $N_{\text{prim}}$  in one chronological public primitive transcript.

**$q_v(\mathcal{A})$ ,  $Q_v$ :** the number of verification queries with valid inputs, and a cap on it. Used in the verification terms of the direct mu-ind and AE bounds of Section 5.2. Every valid verification query contributes at least one verification/decryption construction call in the  $N$  accounting, so  $q_v(\mathcal{A}) \leq N$ .

**$q_{\text{RO}}(\mathcal{B})$ ,  $Q_{\text{RO}}$ :** the number of direct finite-output queries to  $\text{RO}_{\text{flat}}$  by a random oracle model adversary  $\mathcal{B}$ , and a cap on it. Repeated direct queries on the same  $Y$  are counted again. Construction-internal  $\text{RF}(\cdot)$  lookups are not counted. After the derived-adversary construction,  $q_{\text{RO}}$  counts only simulator-induced  $\text{RO}_{\text{flat}}$  calls from the adversary’s direct primitive-interface queries, not the construction calls already counted in  $N$  for the flat sponge lift.

$n_{\text{leaf}}(\mathcal{B})$ ,  $N_{\text{leaf}}$ : the number of distinct construction-generated final leaf transcripts in the execution of  $\mathcal{B}$ , and a cap on it. Distinct final leaf transcripts are generated by valid encryption computations, by valid AE-style verification computations (whether they accept or reject), and by hash, committing, and discoverability challenger construction checks that pass the early validity checks.

For a byte string  $X$ ,

$$\text{leaves}(X) = \max(0, \lceil \|X\|/\beta \rceil - 1)$$

counts the leaf chunks induced by a body byte string of length  $\|X\|$ . Thus  $n_{\text{leaf}}$  is bounded by the sum of  $\text{leaves}(X)$  over the valid construction computations that create final leaf transcripts, where  $X$  is the plaintext body for encryption computations and the submitted ciphertext body for verification and committing or discoverability decryption checks. Each distinct final leaf transcript contains a final leaf duplex call counted in  $N$ , so  $n_{\text{leaf}}(\mathcal{B}) \leq N$  unconditionally. The ratio  $n_{\text{leaf}}/N$  is largest when most processed bytes lie in leaf chunks, where each full leaf contributes  $1 + \beta/\rho = 50$  duplex calls to  $N$  per final leaf transcript.

All theorems are stated for adversaries whose execution-uniform caps hold at the stated bounds.

#### 4.4 Multi-User Setting

This subsection instantiates the mu-ind game of [BT16] (Section 2.4) for TW128 in the lifted public permutation model, using the same ideal-side simulator convention as the single-user AE-style games of Section 4.1. The challenger samples a challenge bit  $b \leftarrow \{0, 1\}$ . If  $b = 1$ , it samples a permutation  $\pi \leftarrow \text{Perm}(\{0, 1\}^{1600})$  and exposes  $(\pi, \pi^{-1})$  as the primitive interface; if  $b = 0$ , it exposes  $(\mathcal{S}^{\text{RO}_{\text{nat}}}, \mathcal{S}^{-1, \text{RO}_{\text{nat}}})$  as the primitive interface. The adversary  $\mathcal{A}$  creates users on demand via **New**, and the game maintains an active-user counter  $v$ . We write  $D$  for an execution-uniform cap on the number of **New** calls. Equivalently, the game may sample independent keys  $K_1, \dots, K_D$  at the start of the experiment and let the  $d$ -th **New** call activate  $K_d$ ; inactive indices are never valid users.

The mu-ind construction oracles inherit the validity convention of Section 4.1. The encryption oracle on input  $(d, N, A, M)$  rejects, without recording, if  $(N, A, M)$  is outside the TW128 encryption domain, if  $d \notin \{1, \dots, v\}$ , or if  $(d, N) \in V$ ; otherwise it sets  $C = \text{Enc}_{K_d}[\pi](N, A, M)$  when  $b = 1$ , and samples a uniformly random byte string  $C$  of length  $\|M\| + \tau_{\text{root}}/8$  when  $b = 0$ . The game records  $(d, N)$  in  $V$  and  $(d, N, A, C)$  in  $W$ , and returns  $C$ . The verification oracle on input  $(d, N, A, C)$  rejects if  $d \notin \{1, \dots, v\}$  or if  $(N, A, C)$  is not a valid TW128 verification/decryption input; if  $(d, N, A, C) \in W$  it returns **true**; if  $b = 0$  it returns **false**; otherwise it returns  $[\text{Dec}_{K_d}[\pi](N, A, C) \neq \perp]$ . Nonce uniqueness is enforced per user by the set  $V$ .

After querying these oracles,  $\mathcal{A}$  outputs  $b' \in \{0, 1\}$ . The mu-ind advantage of  $\mathcal{A}$  against TW128 is

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) = |2 \Pr[b' = b] - 1|.$$

Resource caps split per potential user  $d \leq D$ :  $N^{(d)}$ ,  $q_v^{(d)}$ ,  $n_{\text{leaf}}^{(d)}$  are the per-user counts under Section 4.3, with inactive users contributing zero. The global totals  $N = \sum_{d=1}^D N^{(d)}$ ,  $q_v = \sum_{d=1}^D q_v^{(d)}$ ,  $n_{\text{leaf}} = \sum_{d=1}^D n_{\text{leaf}}^{(d)}$  appear in the final bound:  $N$  through the lift term  $\text{Lift}_c(N_{\text{tot}})$ , and  $n_{\text{leaf}}$  and  $q_v$  through the leaf-collision and verification-tag terms.  $N_{\text{prim}}$  remains a single shared count over the primitive interface.

The mu-ind treatment applies only to indistinguishability. The committing security games already give the adversary control of all  $s$  keys, so the  $s$ -way CMTDs- $\ell$  statements of Section 4.1 are themselves the multi-key committing statement and need no separate mu-cmt notion.

## 5 Main Results

This section states and instantiates TW128’s concrete security bounds. We first collect the recurring loss terms (Section 5.1). We then present the headline theorem sequence, including the random oracle bounds used by the lift (Section 5.2), give the concrete TW128-parameter forms (Sections 5.3 and 5.4), and close with a discussion of the bounds’ tightness (Section 5.5).

### 5.1 Loss Terms

Five closed-form expressions recur across the headline bounds of Section 5.2— $\text{Lift}_c$  throughout, with  $\text{KeyGuess}_k$ ,  $\text{RootColl}_\tau$ ,  $\text{RootPre}_\tau$ , and  $\text{LeafColl}_\tau$  in some. We define them here, ahead of the theorems that use them.

**Flat sponge loss.** The flat sponge differentiating loss is

$$\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}.$$

For  $N_{\text{tot}} < 2^c$  it upper-bounds the [BDPVA08] sponge-differentiating advantage; Lemma 16 carries out the derivation from the [BDPVA08, Lemmas 4 and 5] product bounds through the Weierstrass product inequality. For  $N_{\text{tot}} \geq 2^c$ ,  $\text{Lift}_c(N_{\text{tot}}) \geq 1$  and the advantage bound is trivial without any appeal to indifferenciability, and already at  $N_{\text{tot}} \geq 2^{c/2}$  the bound exceeds  $1/2$  and provides no meaningful security.

**Key-guessing term.** The ideal system key-guessing term of [BDPVA11] is

$$\text{KeyGuess}_k(Q) = Q/2^k.$$

It accounts for direct  $\text{RO}_{\text{flat}}$  queries whose inputs satisfy the decoder  $\text{KeyedTWOut}$  of Table 9. Each such query is an adaptive test of the decoded candidate key against the hidden key  $K$ , independent of the query’s requested output length. Lemma 9 bounds the probability that any of  $Q$  direct queries tests the right key by  $Q/2^k$ .

**Root-collision term.** The  $s$ -way root tag multi-collision term is

$$\text{RootColl}_\tau(Q, s) = \binom{Q+s}{s} / 2^{\tau(s-1)},$$

with the abbreviation  $\text{RootColl}_{\tau_{\text{root}}}(Q, s) = \text{RootColl}_\tau(Q, s)$  at  $\tau = \tau_{\text{root}}$ . The  $+s$  is candidate-sequence slack for the  $s$  final root transcript inputs generated by the committing challenger after the adversary outputs its tuples; these lookups are construction-internal and are not added to  $q_{\text{RO}}(\mathcal{B})$ . In the public permutation CMTDs-4 and direct CMTs- $\ell$  bounds, the committing proof builds a chronological candidate sequence and bounds only the event that some  $s$  distinct candidates have equal root tag projections. The flat sponge lift of Section B gives  $Q = N_{\text{prim}}$ : only the adversary’s  $N_{\text{prim}}$  direct primitive queries can induce simulator-side  $\text{RO}_{\text{flat}}$  queries that contribute to that sequence.

**Root-preimage term.** The root tag preimage term is

$$\text{RootPre}_\tau(Q) = (Q+1)/2^\tau,$$

with the abbreviation  $\text{RootPre}_{\tau_{\text{root}}}(Q) = \text{RootPre}_\tau(Q)$  at  $\tau = \tau_{\text{root}}$ . It is the preimage analogue of  $\text{RootColl}_\tau$ : a single query-independent target tag replaces the  $s$ -way internal collision, the  $+s$  candidate-sequence slack collapses to the  $+1$  of the single discoverability challenger decryption check, and the exponent drops from  $\tau(s-1)$  to  $\tau$ . As in the root-collision term, the flat sponge lift gives  $Q = N_{\text{prim}}$ . The term bounds the query-independent target of context discoverability (Theorem 6).

815 **Leaf-collision term.** The known-key leaf tag collision term is

$$816 \quad \text{LeafColl}_\tau(Q) = \binom{Q}{2} / 2^\tau.$$

817 It is the pairwise birthday bound on  $\tau$ -bit leaf tag projections used by the collision-resistance  
 818 and always second-preimage theorems (Theorem 3, Theorem 4). Unlike the hidden-key  
 819 leaf collision term in the direct mu-ind and AE bounds, the compact committing game has  
 820 adversary-supplied keys, so direct primitive queries can pre-read candidate leaf tag points.  
 821 The random oracle proof therefore counts both direct leaf tag candidates and challenger-  
 822 generated final leaf transcripts; after the flat sponge lift this gives  $Q = N_{\text{prim}} + N_{\text{leaf}}$ .

## 823 5.2 Headline Theorems

824 TW128 delivers its headline results in the lifted public permutation model of Section 4.1:  
 825 multi-user indistinguishability [BT16], all-in-one AE security [RS06] as its single-user  
 826 corollary, and the commitment security of the root tag (Section 2.5). The last is stated  
 827 as the [RS04] hash security of the wrapper H—collision, second-preimage, and preimage  
 828 resistance—together with the [BH22] CMTDs-4 committing notion and the [MLGR23]  
 829 CDY\* notion. For the AE-style notions the ideal side exposes the BDPV simulator; the  
 830 committing, hash, and discoverability bounds remain real public permutation success  
 831 probabilities. Theorems 1 to 6 and Corollaries 1 and 3 state these bounds in terms of the  
 832 loss terms  $\text{Lift}_c$ ,  $\text{KeyGuess}_k$ ,  $\text{RootColl}_\tau$ ,  $\text{LeafColl}_\tau$ , and  $\text{RootPre}_\tau$  of Section 5.1.

833 The real-vs-ideal results come first: a random oracle mu-ind theorem, its public  
 834 permutation lift, and the all-in-one AE corollary obtained by setting the mu-ind user cap  
 835 to  $D = 1$ . The independent hash, committing, and discoverability results follow afterward.  
 836 The CMTDs-4, collision, second-preimage, and preimage random oracle bounds, together  
 837 with the proof of the context-discoverability theorem, are in Section 6. All random oracle  
 838 adversaries in this subsection are flat-RO adversaries (Section 4.2) with the displayed  
 839 resources, and the flat sponge lift of Section B instantiates each random oracle bound at  
 840  $q_{\text{RO}} = N_{\text{prim}}$  when transferring to the public permutation setting.

841 **Theorem 1** (Random Oracle Multi-User Indistinguishability). *For every flat-RO mu-ind*  
 842 *adversary  $\mathcal{B}$  with execution-uniform user cap  $D$  and resource counts of Section 4.4,*

$$843 \quad \text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}) \leq \frac{D(D-1)}{2^{k+1}} + D \cdot \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B})-1)}{2^{\tau_{\text{leaf}}+1}} + \frac{q_v(\mathcal{B})}{2^{\tau_{\text{root}}}}.$$

844 *Proof.* Use the pre-sampled-key formulation of Section 4.4: sample keys  $K_1, \dots, K_D$  at  
 845 the start of the experiment, and let the  $d$ -th New call activate  $K_d$ , unused keys being  
 846 ignored. Let  $\text{coll}$  denote the event that two of the pre-sampled keys  $K_1, \dots, K_D$  collide;  
 847  $\Pr[\text{coll}] \leq \binom{D}{2} / 2^k = D(D-1) / 2^{k+1}$  by union bound. In the random oracle model, use a  
 848 standard hybrid over the  $D$  users:  $H_d$  answers users  $1, \dots, d$  with (Rand, Replay) and users  
 849  $d+1, \dots, D$  with the real construction interface, so  $H_0$  is the real mu-ind game and  $H_D$   
 850 is the ideal one. Let  $H_d^*$  be  $H_d$  with a fixed output bit on  $\text{coll}$ . Since  $\text{coll}$  has the same  
 851 probability in all hybrids,

$$852 \quad \text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}) \leq \Pr[\text{coll}] + |\Pr[\mathcal{B}^{H_0^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_D^*} \Rightarrow 1]|.$$

853 By the triangle inequality, it remains to bound each adjacent stopped hybrid. For each  $d$ ,  
 854 Lemma 14 gives

$$855 \quad |\Pr[\mathcal{B}^{H_{d-1}^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^*} \Rightarrow 1]| \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)}-1)}{2^{\tau_{\text{leaf}}+1}} + \frac{q_v^{(d)}}{2^{\tau_{\text{root}}}}.$$

Summing across  $d = 1, \dots, D$  gives  $D \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B}))$  for the key-test contribution. The leaf-collision contribution satisfies

$$\sum_d \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)}{2^{\tau_{\text{leaf}} + 1}} = \frac{1}{2^{\tau_{\text{leaf}}}} \sum_d \binom{n_{\text{leaf}}^{(d)}}{2} \leq \frac{1}{2^{\tau_{\text{leaf}}}} \left( \sum_d n_{\text{leaf}}^{(d)} \right) = \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\text{leaf}} + 1}},$$

because the unordered pairs drawn within the per-user blocks are a subset of all unordered pairs among  $\sum_d n_{\text{leaf}}^{(d)} = n_{\text{leaf}}(\mathcal{B})$  items. The verification-tag contribution satisfies

$$\sum_d q_v^{(d)} / 2^{\tau_{\text{root}}} = q_v(\mathcal{B}) / 2^{\tau_{\text{root}}}.$$

Combining these estimates with the coll penalty gives the stated RO bound.  $\square$

**Theorem 2** (Multi-User Indistinguishability). *For every mu-ind adversary  $\mathcal{A}$  against TW128 with execution-uniform user cap  $D$  and the global resources of Section 4.4,*

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \frac{D(D-1)}{2^{k+1}} + D \cdot \text{KeyGuess}_k(N_{\text{prim}}) + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{Q_v}{2^{\tau_{\text{root}}}}.$$

*Proof.* By the Lift Application corollary (Corollary 8) for the mu-ind game, there is a random oracle mu-ind adversary  $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$  with  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ ,  $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$ ,  $q_v(\mathcal{B}) \leq Q_v$ , and the per-user construction-side caps of Section 4.4 such that

$$\text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{Adv}_{\text{RO}}^{\text{mu-ind}}(\mathcal{B}).$$

Theorem 1 bounds the random oracle term. Substituting  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ ,  $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$ , and  $q_v(\mathcal{B}) \leq Q_v$  gives the stated bound.  $\square$

**Corollary 1** (All-in-One AE Security). *For every nonce-respecting AE adversary  $\mathcal{A}$  against TW128 with the resources of Section 4.3,*

$$\text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{KeyGuess}_k(N_{\text{prim}}) + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{Q_v}{2^{\tau_{\text{root}}}}.$$

*Proof.* Build a mu-ind adversary  $\mathcal{A}^\mu$  that makes one New query, runs  $\mathcal{A}$ , and forwards each encryption and verification query of  $\mathcal{A}$  to user 1. The primitive interface is forwarded unchanged. The real and ideal views of  $\mathcal{A}$  in the all-in-one AE experiment are exactly the real and ideal views of  $\mathcal{A}^\mu$  in the mu-ind experiment with  $D = 1$ , and the resource counts are unchanged. Applying Theorem 2 with  $D = 1$  eliminates the key-collision term and gives the stated bound.  $\square$

**Hash and committing random oracle bounds.** The remaining public permutation statements use the following random oracle bounds for the root tag wrapper and committing games.

**CMTDs-4 bound.** The flat-RO CMTDs-4 theorem of Section 6 is the direct root-collision bound

$$\text{Adv}_{\text{RO}}^{\text{CMTDs-4}}(\mathcal{B}) \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s) = \binom{q_{\text{RO}}(\mathcal{B}) + s}{s} / 2^{\tau_{\text{root}}(s-1)},$$

with no key-guessing or leaf-collision term. The keys are adversary-supplied, so a direct query on an input accepted by the keyed transcript decoder  $\text{KeyedTWOut}$  is one of the adversary's visible  $\text{RO}_{\text{flat}}$  queries and is counted in  $q_{\text{RO}}(\mathcal{B})$ ; leaf tag collisions cannot merge the final root transcripts whose tags are compared, since the common ciphertext body makes pairwise distinct full inputs yield pairwise distinct  $(K, N, A)$  triples; by Opening Triple Separation (Proposition 1) these reach distinct final root transcripts before any leaf tag is aggregated.

**Collision bound.** The flat-RO collision theorem of Section 6 is

$$\text{Adv}_{\text{RO}}^{\text{Coll}}(\mathcal{B}) \leq \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})) + \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s).$$

The root-collision part is the same final-root candidate argument as CMTDs-4. The additional known-key leaf-collision term is needed because colliding inputs may use different ciphertext bodies; absent such a leaf tag collision, pairwise distinct inputs force pairwise distinct final root transcripts before the common tag is compared.

**Second-preimage bound.** The flat-RO always second-preimage theorem of Section 6 is

$$\text{Adv}_{\text{RO}}^{\text{aSec}}(\mathcal{B}) \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B})) + \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})).$$

The challenge tag is a fixed second-preimage target, so the candidate-collision argument becomes a candidate-preimage argument against the designated challenge candidate; the leaf-collision term is the same adversary-supplied-key accounting as the collision bound.

**Preimage bound.** The flat-RO preimage theorem of Section 6 is the direct root-preimage bound

$$\text{Adv}_{\text{RO}}^{\text{ePre}}(\mathcal{B}) \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B})) = (q_{\text{RO}}(\mathcal{B}) + 1)/2^{\tau_{\text{root}}},$$

again with no key-guessing term, by the same adversary-supplied-key accounting, and no leaf-collision term, since a single submitted input admits no pairwise leaf tag collision case. With a single context, the candidate-collision argument becomes a candidate-preimage argument against the query-independent target tag and no triple-separation is invoked.

Section B carries out the lift in two hops: Hop 1 (Lemma 16) replaces the real permutation by the [BDPVA08] public primitive simulator and the flat-RO construction interface at cost at most  $\text{Lift}_c(N_{\text{tot}})$ , with  $N_{\text{tot}} = N + N_{\text{prim}}$ , and Hop 2 (Lemma 17) couples the result, without loss, to a derived flat-RO adversary  $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$  whose simulator answers induce at most one  $\text{RO}_{\text{flat}}$  call each (Lemma 15), so  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ . The random oracle bounds of Section 5.2 are therefore instantiated at  $N_{\text{prim}}$  rather than  $N_{\text{tot}}$ ; the committing, hash, and discoverability challengers' encryption and decryption checks are construction-internal and counted in  $N$ .

**Public permutation hash, committing, and discoverability bounds.** Applying the same lift gives the remaining headline bounds.

**Theorem 3** (Collision Resistance of H). *For every  $s \geq 2$  and every adversary  $\mathcal{A}$  against the  $s$ -way multi-collision resistance of H in the sense of [BH22], with the resources of Section 4.3,*

$$\text{Adv}_{\text{TW128}}^{\text{Coll}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{LeafColl}_{\tau_{\text{leaf}}}(N_{\text{prim}} + N_{\text{leaf}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s).$$

*Proof.* By the Lift Application corollary (Corollary 8) for the collision game, the derived random oracle adversary  $\mathcal{B}$  has  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$  and  $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$ . The random oracle bound of Theorem 8 is nondecreasing in both  $q_{\text{RO}}(\mathcal{B})$  and  $n_{\text{leaf}}(\mathcal{B})$ , so the monotone form of Corollary 8 gives the stated bound. The  $s$  colliding inputs may differ in their messages, hence in their ciphertext bodies; the  $\text{LeafColl}$  term accounts for the only way two distinct inputs sharing a context  $(K, N, A)$  can reach the same final root transcript—a collision among the leaf tags it absorbs—while distinct contexts already reach distinct final root transcripts by Opening Triple Separation (Proposition 1).  $\square$

**Corollary 2** (Two-Way Collision Resistance of H). *Specializing Theorem 3 to  $s = 2$  gives the classical two-way collision resistance of [RS04]: for every collision adversary  $\mathcal{A}$  against H with the resources of Section 4.3,*

$$\text{Adv}_{\text{TW128}}^{\text{Coll}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{LeafColl}_{\tau_{\text{leaf}}}(N_{\text{prim}} + N_{\text{leaf}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, 2),$$

938 with  $\text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, 2) = \binom{N_{\text{prim}}+2}{2}/2^{\tau_{\text{root}}}$ .

939 *Proof.* The  $s = 2$  case of Theorem 3; [BH22] note that  $s$ -way multi-collision resistance at  
940  $s = 2$  is the classical collision resistance of [RS04].  $\square$

941 **Theorem 4** (Always Second-Preimage Resistance of H). *For every aSec adversary  $\mathcal{A}$*   
942 *against H with the resources of Section 4.3,*

$$943 \quad \text{Adv}_{\text{TW128}}^{\text{aSec}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootPre}_{\tau_{\text{root}}}(N_{\text{prim}}) + \text{LeafColl}_{\tau_{\text{leaf}}}(N_{\text{prim}} + N_{\text{leaf}}).$$

944 *Proof.* By the Lift Application corollary (Corollary 8) for the aSec game, the derived  
945 random oracle adversary  $\mathcal{B}$  has  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$  and  $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$ ; the random oracle  
946 bound of Theorem 10 is nondecreasing in both, so the monotone form of Corollary 8  
947 gives the stated bound. Collision resistance (Theorem 3) already implies eSec and hence  
948 Sec in the sense of [RS04], both with the looser root-collision term; the always-notion  
949 is the one [RS04] shows collision resistance does not imply. Here the second preimage  
950 is sought against the single fixed tag  $H(X)$  of the challenge input  $X$ , yielding the linear  
951 root-preimage term.  $\square$

952 **Corollary 3** (CMTDs-4 Committing Security). *For every  $s \geq 2$  and every  $s$ -way CMTDs-4*  
953 *adversary  $\mathcal{A}$  against TW128 with the resources of Section 4.3,*

$$954 \quad \text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s) = \text{Lift}_c(N_{\text{tot}}) + \frac{\binom{N_{\text{prim}}+s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

955 *Proof.* A CMTDs-4 winner exhibits  $s$  distinct inputs  $(K_i, N_i, A_i, M_i)$  whose encryptions  
956 share one common ciphertext  $C \parallel T$ ; in particular they share the tag  $T$ , so they are an  
957  $s$ -way collision of H, and  $\text{Adv}_{\text{TW128}}^{\text{CMTDs-4}} \leq \text{Adv}_{\text{TW128}}^{\text{Coll}}$ . For the displayed no-LeafColl bound,  
958 use the dedicated root-collision derivation of Theorem 7 for this all-bodies-equal case,  
959 lifted by Corollary 8: the single shared body  $C$  forces the triples  $(K_i, N_i, A_i)$  pairwise  
960 distinct—by Opening Triple Separation (Proposition 1)—so a leaf tag collision cannot  
961 merge two openings and no LeafColl term arises.  $\square$

962 **Theorem 5** (Everywhere Preimage Resistance of H). *For every target tag  $T^* \in \{0, 1\}^{\tau_{\text{root}}}$*   
963 *fixed in advance and every preimage adversary  $\mathcal{A}$  against H with the resources of Section 4.3,*

$$964 \quad \text{Adv}_{\text{TW128}}^{\text{ePre}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootPre}_{\tau_{\text{root}}}(N_{\text{prim}}) = \text{Lift}_c(N_{\text{tot}}) + \frac{N_{\text{prim}} + 1}{2^{\tau_{\text{root}}}}.$$

965 *Proof.* By the Lift Application corollary (Corollary 8) for the preimage game—whose  
966 target  $T^*$  is fixed before the primitive sampling—the derived random oracle adversary  
967  $\mathcal{B}$  has  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ , and the random oracle bound of Theorem 9 is nondecreasing in  
968  $q_{\text{RO}}(\mathcal{B})$ , so the monotone form of Corollary 8 gives the stated bound. With one fixed target  
969 tag, Theorem 9 applies a candidate-preimage bound against that target and drops the  
970 exponent from  $\tau_{\text{root}}(s - 1)$  to  $\tau_{\text{root}}$ .  $\square$

971 **Theorem 6** (Context Discoverability (CDY\*) of [MLGR23]). *For every target specifier*  
972  *$\mathbf{ts} \subsetneq \{k, n, a\}$ —so that at least one context component stays hidden—and every adversary  $\mathcal{A}$*   
973 *against TW128, the restricted CDY\*[ts] advantage of [MLGR23, Fig. 5]—and in particular*  
974 *each per-component variant  $\text{CDY}_k^*$ ,  $\text{CDY}_n^*$ ,  $\text{CDY}_a^*$ —satisfies*

$$975 \quad \text{Adv}_{\text{TW128}}^{\text{CDY}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootPre}_{\tau_{\text{root}}}(N_{\text{prim}}) + \text{KeyGuess}_k(N_{\text{prim}}).$$

976 The proof is given in Section 6.

977 **Corollary 4** (Root Tag as a Compact Commitment). *The wrapper  $H$  is a secure hash*  
 978 *of the full encryption context  $(K, N, A, M)$  in the sense of [RS04]: collision-resistant*  
 979 *(Theorem 3), second-preimage-resistant (Theorem 4), and preimage-resistant (Theorem 5).*  
 980 *At the implemented parameters all three advantages are at most  $2^{-128}$  (Corollary 5),*  
 981 *so the single  $\tau_{\text{root}}$ -bit root tag is a binding, preimage-resistant compact commitment to*  
 982  *$(K, N, A, M)$ , native to the mode.*

983 *Proof.* Collision resistance is Theorem 3, second-preimage resistance Theorem 4, and  
 984 preimage resistance Theorem 5; all three quantify the adversary's success at the same  $\tau_{\text{root}}$ -  
 985 bit projection of the final root transcript, and Corollary 5 evaluates them at the implemented  
 986 parameters. Binding (collision resistance) is the property a compact commitment must  
 987 have, and preimage resistance certifies that the tag alone reveals no opening; together they  
 988 are the properties of a hash-based commitment.  $\square$

### 989 5.3 Concrete TW128 Bounds

990 The implemented parameters of Section 3.2 fix

$$991 \quad k = c = \tau_{\text{root}} = \tau_{\text{leaf}} = 256.$$

992 Substituting into Corollaries 1 and 3 and Theorems 2 to 6 and using the random oracle  
 993 bounds of Section 5.2 gives the following closed-form bounds.

#### All-in-one AE.

$$994 \quad \text{Adv}_{\text{TW128}}^{\text{ae}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{N_{\text{prim}} + Q_v}{2^{256}} + \frac{N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{257}}.$$

#### Multi-user indistinguishability.

$$995 \quad \text{Adv}_{\text{TW128}}^{\text{mu-ind}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{DN_{\text{prim}} + Q_v}{2^{256}} + \frac{D(D - 1) + N_{\text{leaf}}(N_{\text{leaf}} - 1)}{2^{257}}.$$

996  **$s$ -way collision resistance of H.** For every  $s \geq 2$ ,

$$997 \quad \text{Adv}_{\text{TW128}}^{\text{Coll}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{\binom{N_{\text{prim}} + N_{\text{leaf}}}{2}}{2^{256}} + \frac{\binom{N_{\text{prim}} + s}{s}}{2^{256(s-1)}}.$$

998 For  $s = 2$ , this is the ordinary two-input collision bound:

$$999 \quad \begin{aligned} \text{Adv}_{\text{TW128}}^{\text{Coll}}(\mathcal{A}) &\leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{(N_{\text{prim}} + N_{\text{leaf}})(N_{\text{prim}} + N_{\text{leaf}} - 1)}{2^{257}} \\ &\quad + \frac{(N_{\text{prim}} + 1)(N_{\text{prim}} + 2)}{2^{257}}. \end{aligned}$$

1000 **Always second-preimage resistance of H.** The root-preimage contribution is linear in  
 1001 the primitive-query budget:

$$1002 \quad \text{Adv}_{\text{TW128}}^{\text{aSec}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{N_{\text{prim}} + 1}{2^{256}} + \frac{\binom{N_{\text{prim}} + N_{\text{leaf}}}{2}}{2^{256}}.$$

1003  **$s$ -way CMTDs-4 committing security.** For every  $s \geq 2$ , the all-bodies-equal special case  
 1004 drops the leaf tag term:

$$1005 \quad \text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{\binom{N_{\text{prim}} + s}{s}}{2^{256(s-1)}}.$$

**Table 4:** Concrete TW128 security at the work cap  $N_{\text{tot}} \leq 2^{63}$  (with  $D \leq 2^{63}$  for mu-ind), under  $k = c = \tau_{\text{root}} = \tau_{\text{leaf}} = 256$  and the resource consequences  $N_{\text{prim}}, Q_v, N_{\text{leaf}} \leq N_{\text{tot}}$  and  $N_{\text{prim}} + N_{\text{leaf}} \leq N_{\text{tot}}$  of Section 4.3. The committing rows hold for every  $s \geq 2$ ; larger  $s$  only shrinks the root-collision term.

| Notion                   | Dominant scale                         | At the cap      |
|--------------------------|----------------------------------------|-----------------|
| All-in-one AE            | $\text{Lift}_c(N_{\text{tot}})$        | $\leq 2^{-128}$ |
| Multi-user ind.          | $D \text{KeyGuess}_k(N_{\text{prim}})$ | $\leq 2^{-128}$ |
| H collision (Coll)       | $\text{Lift}_c(N_{\text{tot}})$        | $\leq 2^{-128}$ |
| H second-preimage (aSec) | $\text{Lift}_c(N_{\text{tot}})$        | $\leq 2^{-128}$ |
| H preimage (ePre)        | $\text{Lift}_c(N_{\text{tot}})$        | $\leq 2^{-128}$ |
| CMTDs-4 [BH22]           | $\text{Lift}_c(N_{\text{tot}})$        | $\leq 2^{-128}$ |
| CDY [MLGR23]             | $\text{Lift}_c(N_{\text{tot}})$        | $\leq 2^{-128}$ |

1006 *Remark 1* (Two-Key CMTDs-4 Specialization). Setting  $s = 2$  recovers the ordinary two-key  
 1007 CMTDs-4 statement of [BH22]:

$$1008 \text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{\binom{N_{\text{prim}}+2}{2}}{2^{256}} = \frac{N_{\text{tot}}(N_{\text{tot}} + 1) + (N_{\text{prim}} + 1)(N_{\text{prim}} + 2)}{2^{257}}.$$

#### Everywhere preimage resistance of H.

$$1009 \text{Adv}_{\text{TW128}}^{\text{ePre}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{N_{\text{prim}} + 1}{2^{256}}.$$

#### Context discoverability ([MLGR23]).

$$1010 \text{Adv}_{\text{TW128}}^{\text{CDY}}(\mathcal{A}) \leq \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{257}} + \frac{2N_{\text{prim}} + 1}{2^{256}}.$$

## 1011 5.4 Concrete Security Claim

1012 **Corollary 5** (Concrete TW128 Security). *Under the implemented parameters  $k = c =$   
 1013  $\tau_{\text{root}} = \tau_{\text{leaf}} = 256$ , every adversary with work cap  $N_{\text{tot}} \leq 2^{63}$ —and, in the multi-user case,  
 1014 user cap  $D \leq 2^{63}$ —has advantage at most  $2^{-128}$  in each of TW128’s notions: multi-user  
 1015 indistinguishability (Theorem 2), all-in-one AE (Corollary 1), the [RS04] hash security  
 1016 of H— $s$ -way collision resistance for every  $s \geq 2$ , always second-preimage resistance, and  
 1017 preimage resistance (Theorems 3 to 5)—the [BH22] CMTDs-4 committing bound it yields  
 1018 (Corollary 3), and [MLGR23] CDY\* (Theorem 6).*

1019 *Proof.* The claim instantiates the closed forms of Section 5.3 under the resource conse-  
 1020 quences of Section 4.3:  $N_{\text{prim}} \leq N_{\text{tot}}$  by definition, and the pathwise bounds  $q_v \leq N$  and  
 1021  $n_{\text{leaf}} \leq N$  let the resource caps  $Q_v$  and  $N_{\text{leaf}}$  be taken at most  $N_{\text{tot}}$ , so no separate top-level  
 1022 cap on  $Q_v$  or  $N_{\text{leaf}}$  is required; because  $N_{\text{tot}} = N + N_{\text{prim}}$  counts construction-internal  
 1023 calls and direct queries disjointly, the same bound  $n_{\text{leaf}} \leq N$  yields the joint constraint  
 1024  $N_{\text{prim}} + N_{\text{leaf}} \leq N_{\text{tot}}$ . Only the mu-ind user cap  $D$  remains an independent hypothesis,  
 1025 since created-but-idle users need not contribute to  $N$ . At the headline work cap  $N_{\text{tot}} \leq 2^{63}$   
 1026 (and  $D \leq 2^{63}$  for mu-ind) every bound is below  $2^{-128}$ , with the flat sponge loss  $\text{Lift}_c(N_{\text{tot}})$   
 1027 the dominant scale in all but the multi-user case (Table 4).

1028 Term by term at the cap, with  $N_{\text{prim}}, Q_v, N_{\text{leaf}} \leq N_{\text{tot}}$ : the flat sponge loss  $\text{Lift}_c(N_{\text{tot}})$   
 1029 and any leaf tag birthday term  $N_{\text{leaf}}(N_{\text{leaf}} - 1)/2^{257}$  are each at most one  $2^{-131}$ -sized  
 1030 term; the AE linear key-guess and verification contributions  $(N_{\text{prim}} + Q_v)/2^{256}$  are at  
 1031 most  $2 \cdot 2^{-193}$ ; the root-collision term  $\text{RootColl}_{256}(N_{\text{prim}}, s) = \binom{N_{\text{prim}}+s}{s}/2^{256(s-1)}$  likewise

matches  $\text{Lift}_c$  as a  $2^{-131}$ -sized term in its two-key ( $s = 2$ ) specialization and is smaller still for  $s \geq 3$  under the cap; the known-key  $\text{LeafColl}_{256}(N_{\text{prim}} + N_{\text{leaf}})$ , which the joint cap  $N_{\text{prim}} + N_{\text{leaf}} \leq N_{\text{tot}}$  bounds by  $\binom{N_{\text{tot}}}{2}/2^{256}$ , is likewise  $\leq 2^{-131}$ ; and the root-preimage term  $(N_{\text{prim}} + 1)/2^{256}$  is at most  $2 \cdot 2^{-193}$ . The single-user totals are therefore dominated by  $\text{Lift}_c$  and stay below  $2^{-128}$ . The multi-user bound is the exception: its hybrid-induced term  $D \cdot \text{KeyGuess}_k(N_{\text{prim}}) \leq DN_{\text{tot}}/2^{256}$  reaches  $\approx 2\text{Lift}_c$  at the upper end and dominates—staying at most  $\text{Lift}_c$  once  $D \leq N_{\text{tot}}(N_{\text{tot}} + 1)/(2N_{\text{prim}})$  (for  $N_{\text{prim}} > 0$ ; when  $N_{\text{prim}} = 0$  the hybrid-induced term vanishes), typical of any practical deployment—and the total there is at most  $5 \cdot 2^{-131} + 2 \cdot 2^{-194} < 8 \cdot 2^{-131} = 2^{-128}$ . Expressed in bytes,  $N_{\text{tot}} \leq 2^{63}$  corresponds to at most  $\rho \cdot 2^{63} = 167 \cdot 2^{63} < 2^{71}$  bytes of plaintext, associated data, ciphertext, and aggregation through the duplex (Section 3.2), well beyond any realistic deployment lifetime.  $\square$

## 5.5 Discussion of the Bounds

The bounds of Section 5.3 aggregate the loss terms of Section 5.1, whose sizes at the work cap Table 4 records and whose dominance Section 5.4 discusses. We comment on their tightness.

### Tightness.

- $\text{KeyGuess}_k(Q) = Q/2^k$ . Tight at constants: a direct random-guess attack on a  $k$ -bit key matches the per-query success probability  $1/2^k$ . The multi-user bound's dominant term  $D \text{KeyGuess}_k(N_{\text{prim}}) = DN_{\text{prim}}/2^k$  is likewise tight at the leading constant: against the  $D$  independent user keys each direct guess matches some key with probability  $D/2^k$ , so  $N_{\text{prim}}$  guesses recover a user key—and distinguish—with probability  $\approx DN_{\text{prim}}/2^k$ , matching the bound.
- $\text{RootColl}_\tau(Q, s) = \binom{Q+s}{s}/2^{\tau(s-1)}$ . Tight up to lower-order terms: it is the standard  $s$ -way multi-collision bound on a  $\tau$ -bit projection of  $\text{RO}_{\text{flat}}$ , and a generic birthday-style adversary against the random oracle matches the leading-order term. It is the root term of the collision bound (H Coll, Theorem 3). Its  $N_{\text{prim}}$  counts every primitive query, though by Output-Point Separation (Corollary 7) only those whose induced  $\text{RO}_{\text{flat}}$  call lands at a root tag output point ( $\mathcal{R}_{\text{tag}}$ , or  $\mathcal{R}_{\text{msg}}^+$  for single-chunk ciphertexts) can enter the candidate sequence; a class-filtered count would be no larger, and potentially smaller, but the headline keeps the  $N_{\text{prim}}$  form for uniformity with Corollary 1 and Theorem 2, the birthday threshold at  $\tau_{\text{root}} = 256$  being unaffected.
- $\text{RootPre}_\tau(Q) = (Q + 1)/2^\tau$ . Tight at constants: a generic preimage search against a fixed  $\tau$ -bit target matches the per-query  $2^{-\tau}$  rate. It is the root term of the preimage (ePre), always second-preimage (aSec), and discoverability (CDY\*) bounds, where the single fixed target yields a linear term.
- *Leaf tag collision*  $N_{\text{leaf}}(N_{\text{leaf}} - 1)/2^{\tau_{\text{leaf}}+1}$ . Tight for verification security: by reusing a nonce, an adversary detects a leaf tag birthday collision through the matching root tags and splices the colliding leaf chunk into a fresh ciphertext that still verifies, forging at the leading  $N_{\text{leaf}}^2/2^{\tau_{\text{leaf}}+1}$  rate. The mu-ind and all-in-one AE bounds carry the term through the verification side of their real-vs-ideal coupling; their nonce-respecting encryption rules this attack out, so there it is inherited slack.
- *Known-key leaf collision*  $\text{LeafColl}_\tau(Q)$ . The same birthday attack applies when keys are adversary-supplied and direct queries can target leaf tag points; the collision and always second-preimage proofs use  $Q = N_{\text{prim}} + N_{\text{leaf}}$ .

- *Flat sponge lift.* The term  $\text{Lift}_c(N_{\text{tot}}) = N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$  dominates the random-permutation differentiating bound  $f_P(N_{\text{tot}})$  of [BDPVA08, Lemma 5] via Lemma 16. The remaining slack is the unimproved tightness of the [BDPVA08] simulator itself, which is an open problem on the sponge side and not specific to TW128.

Cross-scheme bound-level comparisons against prior sponge AEADs and the committing AEAD line of [BH22] are deferred to Sections 8.1 to 8.3.

## 6 Hash Security of the Root Tag

The tag-level results of this section share one spine. The Root tag Candidate Sequence lemma (Lemma 1) collects the root tag inputs reached by a keyless game into a chronological candidate sequence, and the Root Tag Hash Bound (Lemma 2) turns a collision, preimage, or second-preimage among those candidates into the matching adaptive bound of Section A, controlling the chance that distinct inputs land on the same  $\tau_{\text{root}}$ -bit root tag projection. The four random oracle theorems—CMTDs-4 (Theorem 7), collision (Theorem 8), preimage (Theorem 9), and always second-preimage (Theorem 10)—are then short instantiations, differing only in which case of the bound they invoke and in whether a leaf tag collision term is needed when colliding inputs may use different ciphertext bodies. The context-discoverability proof of Theorem 6 uses the same fixed-target root-preimage case, together with hidden-component key-test accounting. Section B transfers each instance to the public permutation setting, Section 5.2 records the concrete bounds, and Corollary 6 forwards the remaining [BH22] committing notions to the same root-collision bound.

The instantiations rest on two structural separation facts, recorded first: distinct contexts  $(K, N, A)$  reach distinct final root transcripts regardless of the leaf tags they aggregate (Opening Triple Separation, Proposition 1), and—absent a collision among those leaf tags—so do distinct full inputs (Final-Root Input Separation, Proposition 2).

**Proposition 1** (Opening Triple Separation). *If two TW128 root computations have distinct  $(K, N, A)$  triples, then their bridged final root transcript inputs under  $\text{flat}(\cdot)$  are distinct, regardless of whether each computation is an encryption or a decryption check on a common ciphertext.*

*Proof.* Suppose for contradiction that  $\text{flat}(R_i) = \text{flat}(R_j)$ . Since the bridge  $\text{flat}(\cdot)$  is injective on well-formed transcript prefixes (Lemma 6), the native flattened inputs of the two final-root prefixes are equal. Then Lemma 4 recovers the full duplex-call sequence and reads off the pair  $(\sigma, \sigma_{\text{ph}})$  for each call. The seven 4-bit phase suffixes are pairwise distinct, so the  $\sigma_{\text{init}}$ ,  $\sigma_{\text{ad}}^{\pm}$ ,  $\sigma_{\text{msg}}^{\pm}$ , and (when present)  $\sigma_{\text{agg}}^{\pm}$  calls in the recovered sequence are unambiguously identified. The aggregation phase is present exactly when  $n \geq 1$ , and  $n = \text{leaves}(C)$  is determined by the common ciphertext, so the two recovered sequences either both carry an aggregation phase or both omit it; either way the recovery of  $(K, N, A)$  below uses only the  $\sigma_{\text{init}}$  and AD calls, which precede any message or aggregation block.

Equality of the recovered sequences forces equality of the  $\sigma_{\text{init}}$  call, whose  $\sigma$  is  $p_{\text{root}} \| K \| N$  with fixed-width  $K$  and  $N$ , hence  $K_i = K_j$  and  $N_i = N_j$ . It also forces equality of the recovered AD subsequences. By the AD bullet of Lemma 4 the associated data phase contributes a  $\sigma_{\text{ad}}^+$ -tagged block exactly when the associated data is nonempty, so the recovered sequences agree on the presence of the AD phase. If exactly one of  $A_i, A_j$  were empty the recovered sequences would differ in the presence of a  $\sigma_{\text{ad}}^+$  call, contradicting their equality; hence either both are empty, giving  $A_i = A_j = \varepsilon$ , or both are nonempty, in which case the phase-recovery sub-claim of Lemma 4 inverts the  $\rho$ -byte-block partition map on the AD byte string and equal AD  $\sigma$ -block sequences imply  $A_i = A_j$ . In all cases  $(K_i, N_i, A_i) = (K_j, N_j, A_j)$ , contradicting the hypothesis of distinct triples.  $\square$

Triple Separation handles distinct contexts; when inputs share a context but differ in their messages, distinctness of the final root transcripts needs the absence of a leaf tag collision, recorded next.

**Proposition 2** (Final-Root Input Separation). *Let  $\text{bad}_{\text{leaf}}^*$  be the event that two distinct construction-generated final leaf transcripts receive the same  $\tau_{\text{leaf}}$ -bit RF projection. On  $\neg\text{bad}_{\text{leaf}}^*$ , if two TW128 construction computations reach final root transcripts with  $\text{flat}(R) = \text{flat}(R')$ , then they share the same context  $(K, N, A)$  and the same full ciphertext  $C$ . Equivalently, on  $\neg\text{bad}_{\text{leaf}}^*$  computations with distinct  $(K, N, A, C)$  reach distinct bridged final root transcripts.*

*Proof.* By bridge injectivity (Lemma 6),  $\text{flat}(R) = \text{flat}(R')$  gives equal native flattened final-root inputs. Lemma 4 then recovers the same root initialization  $p_{\text{root}} \parallel K \parallel N$ —hence the same  $K$  and  $N$ —the same associated data blocks—hence the same  $A$ —the same root ciphertext chunk, and the same leaf count and ordered leaf tag sequence. Under  $\neg\text{bad}_{\text{leaf}}^*$ , Leaf-Transcript Recovery (Lemma 5) identifies the equal leaf tag sequences with the same final leaf transcripts, hence the same leaf ciphertext bodies; with the common root ciphertext chunk this gives the same full  $C$ .  $\square$

**Lemma 1** (Root tag Candidate Sequence). *Consider a random oracle game of Section 4.2 in which the adversary  $\mathcal{B}$  makes direct  $\text{RO}_{\text{flat}}$  queries and the challenger runs  $m \geq 1$  deterministic encryption or decryption checks, the  $i$ -th reaching a final root transcript  $R_i$  whose root tag is read by a single  $\text{RO}_{\text{flat}}$  lookup at  $\text{flat}(R_i)$ . Every execution then admits a chronological sequence of root tag inputs such that*

1. *each distinct direct query whose input  $Y$  decodes under KeyedTWOut to a root tag output-point class— $\mathcal{R}_{\text{msg}}^+$  (the final root message block, which carries the root tag only on the no-leaf path) or  $\mathcal{R}_{\text{tag}}$  (the aggregated root tag)—is appended as a candidate before its  $\text{RO}_{\text{flat}}$  lookup, and contributes at most one candidate; multi-chunk computations also reach  $\mathcal{R}_{\text{msg}}^+$  with zero requested output before aggregation, so this predicate over-approximates the root tag candidate set, which only loosens the count;*
2. *if the challenger reaches its checks, each  $\text{flat}(R_i)$  is in the sequence, appended—if not already present—immediately before the  $i$ -th root tag answer is sampled;*
3. *no candidate's  $\text{RO}_{\text{flat}}$  value is read before its append step, so the sequence satisfies the predictability and unrevealed-value premises of Lemma 8; and*
4. *its length is at most  $q_{\text{RO}}(\mathcal{B}) + m$ .*

*Proof.* The committing, discoverability, and hash (collision, second-preimage, and preimage) games are keyless: all keys are adversary-supplied, so a direct query to a keyed transcript is one of  $\mathcal{B}$ 's visible  $\text{RO}_{\text{flat}}$  queries counted in  $q_{\text{RO}}(\mathcal{B})$ , and there is no  $\text{bad}_{\text{key}}$  event. No construction oracle runs while  $\mathcal{B}$  issues its direct queries, so each direct-query candidate is first read no earlier than its append step. The append predicate depends on  $Y$  alone through the exact decoder of Lemma 6, which ignores the requested output length, and by Output-Point Separation (Corollary 7) each input decodes to at most one output-point class; hence each distinct direct query contributes at most one candidate, giving (1).

If the challenger reaches its  $m$  checks, append  $\text{flat}(R_i)$  immediately before the  $i$ -th root tag answer is sampled, unless it is already present. The intermediate  $\text{RO}_{\text{flat}}$  lookups of check  $i$  lie at non-root tag output points, so by Corollary 7 they do not read  $\text{flat}(R_i)$ . If  $\text{flat}(R_i)$  was touched earlier—by a direct query or by an earlier check  $j < i$ —then Lemma 6 and Corollary 7 make that earlier access a final-root lookup at the same address, which was therefore already appended, so the input is in the sequence and is not appended again. Otherwise its value is still unread when appended. Either way no candidate's value is read before its append step, giving (2) and (3). At most  $q_{\text{RO}}(\mathcal{B})$  direct candidates and  $m$  challenger candidates are appended, giving (4).  $\square$

**Lemma 2** (Root Tag Hash Bound). *Consider a keyless TW128 random oracle game (Section 4.2) in which, after  $\mathcal{B}$ 's direct  $\text{RO}_{\text{flat}}$  queries, the challenger runs deterministic encryption or decryption checks reaching final root transcripts, with win event  $W$ . By the Root tag Candidate Sequence lemma (Lemma 1) the root tag inputs form a chronological candidate sequence meeting the premises of Lemma 8 with  $\tau = \tau_{\text{root}}$ . Writing  $q_{\text{RO}} = q_{\text{RO}}(\mathcal{B})$ :*

1. *if  $W$  forces  $s$  checks to reach pairwise distinct bridged final root transcripts carrying a common root tag, then, with  $m = s$  checks,  $\Pr[W] \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}, s)$ ;*
2. *if  $W$  forces a single check ( $m = 1$ ) to carry a root tag equal to a target fixed independently of  $\text{RO}_{\text{flat}}$ , then  $\Pr[W] \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}})$ ;*
3. *if a designated check reaches  $R_X$  whose root tag is exposed before  $\mathcal{B}$  outputs, and  $W$  forces a single further check ( $m = 1$  besides  $R_X$ ) to reach a transcript  $R' \neq R_X$  carrying  $R_X$ 's root tag, then  $\Pr[W] \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}})$ .*

*Proof.* In each case the candidate sequence has the stated length and meets the premises of Lemma 8. (1) The  $s$  distinct candidates carrying a common  $\tau_{\text{root}}$ -projection are an  $s$ -way collision; the collision case with  $L = q_{\text{RO}} + s$  gives  $\binom{q_{\text{RO}}+s}{s} \cdot 2^{-\tau_{\text{root}}(s-1)} = \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}, s)$ . (2) The candidate hitting the fixed target is a preimage; the preimage case with  $L = q_{\text{RO}} + 1$  gives  $(q_{\text{RO}} + 1) \cdot 2^{-\tau_{\text{root}}} = \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}})$ . (3) Prepend  $\text{flat}(R_X)$  as the designated candidate, its value the exposed root tag; the further candidate colliding with it on its  $\tau_{\text{root}}$ -projection is a second-preimage, and the second-preimage corollary with  $L = q_{\text{RO}} + 2$  gives  $(q_{\text{RO}} + 1) \cdot 2^{-\tau_{\text{root}}} = \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}})$ .  $\square$

**Theorem 7** (Random Oracle CMTDs-4). *For every  $s \geq 2$  and every  $s$ -way CMTDs-4 adversary  $\mathcal{B}$  in the TW128 random oracle model (Section 4.2),*

$$\text{Adv}_{\text{RO}}^{\text{CMTDs-4}}(\mathcal{B}) \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s) = \frac{\binom{q_{\text{RO}}(\mathcal{B})+s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

*Proof.* Run the random oracle CMTDs-4 game. After rejecting on any failed syntax, message-length, or pairwise-full-input check, the challenger performs  $s$  deterministic decryption checks  $\text{Dec}_{K_i}^{\text{RO}}(N_i, A_i, C \| T)$  on the one common body  $C \| T$ , reaching  $R_1, \dots, R_s$ . On a winning execution the tuples  $(K_i, N_i, A_i, M_i)$  are pairwise distinct, and since all checks decrypt the common body, determinism of  $\text{Dec}$  makes the triples  $(K_i, N_i, A_i)$  pairwise distinct—equal triples would return equal messages, hence equal tuples. By Opening Triple Separation (Proposition 1) the candidates  $\text{flat}(R_1), \dots, \text{flat}(R_s)$  are pairwise distinct (leaf tag collisions cannot merge them), and every accepting check carries the common root tag  $T$ . The Root Tag Hash Bound (Lemma 2, part 1) gives  $\text{Adv}_{\text{RO}}^{\text{CMTDs-4}}(\mathcal{B}) \leq \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s)$ .  $\square$

**Remaining committing notions.** The hierarchy and  $s$ -way extension of the [BH22] committing notions are given in [BH22, §3, Figs. 4–5, and App. A]. We use those relations for the D-notions and a direct source-game argument for the CMTs- $\ell$  notions to preserve the resource split of Section 4.3.

**Corollary 6** (Committing-Notion Root-Collision Bound). *For every  $s \geq 2$ , every*

$$X \in \{\text{CMTDs-1}, \text{CMTDs-3}, \text{CMTDs-4}, \text{CMTs-1}, \text{CMTs-3}, \text{CMTs-4}\},$$

*and every  $s$ -way  $X$ -adversary  $\mathcal{A}$  against TW128,*

$$\text{Adv}_{\text{TW128}}^X(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{RootColl}_{\tau_{\text{root}}}(N_{\text{prim}}, s) = \text{Lift}_c(N_{\text{tot}}) + \frac{\binom{N_{\text{prim}}+s}{s}}{2^{\tau_{\text{root}}(s-1)}}.$$

*Proof.* CMTDs-4 is bounded directly by Corollary 3. For  $\ell \in \{1, 3\}$ , the  $s$ -way hierarchy of [BH22, §3, Fig. 5 and App. A] gives  $\text{Adv}_{\text{TW128}}^{\text{CMTDs-}\ell}(\mathcal{A}) \leq \text{Adv}_{\text{TW128}}^{\text{CMTDs-4}}(\mathcal{A})$  for the same output distribution, so the resource counts are unchanged.

The E-notions are bounded by the Root Tag Hash Bound, with the challenger's  $s$  deterministic *encryption* checks supplying the candidate final-root transcripts (Lemma 1 builds the candidate sequence for encryption or decryption checks alike). On a winning execution all  $s$  encryptions are equal, hence of equal length and with one common root tag  $T$ ; pairwise  $\text{WiC}_\ell$ -distinctness forces pairwise distinct triples—for  $\ell = 4$  via determinism, since equal triples with equal ciphertexts would decrypt to equal messages—so Proposition 1 makes the  $s$  final-root candidates distinct while their  $\tau_{\text{root}}$ -bit projections all equal  $T$ . By the Lift Application corollary (Corollary 8) for the  $X$  game, the derived random oracle adversary  $\mathcal{B}$  has  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ ; the Root Tag Hash Bound (Lemma 2, part 1) then bounds its win by  $\text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s)$  in the random oracle model, which the lift transfers to the public permutation setting at  $N_{\text{prim}}$ ; the encryption checks are construction-internal work counted in  $N$ , not in  $N_{\text{prim}}$ .  $\square$

**Theorem 8** (Random Oracle Collision Resistance of H). *For every  $s \geq 2$  and every  $s$ -way collision adversary  $\mathcal{B}$  in the TW128 random oracle model,*

$$\text{Adv}_{\text{RO}}^{\text{Coll}}(\mathcal{B}) \leq \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})) + \text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s).$$

*Proof.* Run the random oracle collision game. Let  $\text{bad}_{\text{leaf}}^*$  be the event that two distinct construction-generated final leaf transcripts receive the same  $\tau_{\text{leaf}}$ -bit RF projection; by the adversary-supplied-key bound of Lemma 10,  $\Pr[\text{bad}_{\text{leaf}}^*] \leq \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B}))$ , so it remains to bound the win under  $\neg \text{bad}_{\text{leaf}}^*$ . After rejecting unless the  $s$  output inputs are pairwise distinct and in the TW128 domain, the challenger performs  $s$  deterministic encryptions  $\text{Enc}_{K_i}^{\text{RO}}(N_i, A_i, M_i)$ , reaching  $R_1, \dots, R_s$ , all carrying a common root tag  $T$  on a win. Pairwise distinct tuples  $(K_i, N_i, A_i, M_i)$  have pairwise distinct  $(K_i, N_i, A_i, C_i)$ : two tuples differing in their context already differ in  $(K, N, A)$ , while two sharing a context but differing in the message yield distinct ciphertexts, since for a fixed context encryption maps the message to the ciphertext bijectively. So on  $\neg \text{bad}_{\text{leaf}}^*$  Final-Root Input Separation (Proposition 2) makes  $\text{flat}(R_1), \dots, \text{flat}(R_s)$  pairwise distinct. The Root Tag Hash Bound (Lemma 2, part 1) bounds the win under  $\neg \text{bad}_{\text{leaf}}^*$  by  $\text{RootColl}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}), s)$ ; adding  $\Pr[\text{bad}_{\text{leaf}}^*]$  gives the theorem.  $\square$

**Theorem 9** (Random Oracle Everywhere Preimage Resistance of H). *For every query-independent target tag  $T^* \in \{0, 1\}^{\tau_{\text{root}}}$  and every preimage adversary  $\mathcal{B}$  in the TW128 random oracle model (Section 4.2),*

$$\text{Adv}_{\text{RO}}^{\text{ePre}}(\mathcal{B}) \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B})) = \frac{q_{\text{RO}}(\mathcal{B}) + 1}{2^{\tau_{\text{root}}}}.$$

*Proof.* Run the random oracle preimage game for the query-independent target  $T^*$ , which is independent of  $\text{RO}_{\text{flat}}$ . After rejecting unless the output input  $X = (K, N, A, M)$  is in the TW128 domain, the challenger performs one encryption  $\text{Enc}_K^{\text{RO}}(N, A, M)$ , reaching a final root transcript  $R$ . A win means  $H(X) = T^*$ , i.e. the root tag  $\lfloor \text{RO}_{\text{flat}}(\text{flat}(R)) \rfloor_{\tau_{\text{root}}}$  at  $R$  equals the fixed target  $T^*$ . The Root Tag Hash Bound (Lemma 2, part 2, with target  $T^*$ ) gives  $\text{Adv}_{\text{RO}}^{\text{ePre}}(\mathcal{B}) \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}))$ .  $\square$

### Context discoverability.

*Proof of Theorem 6.* Full reveal  $\text{ts} = \{k, n, a\}$  hides no component and is won trivially by any scheme, so the restriction to  $\text{ts} \subsetneq \{k, n, a\}$  excludes no meaningful case. The restricted advantage requires  $\mathcal{A}$  to win against every query-independent selector, so it is at most the success probability against the selector  $S^*$  that samples a fresh context  $(K_0, N_0, A_0)$ —each

of  $K_0$ ,  $N_0$ , and  $A_0$  uniform on a 256-bit space—encrypts a message under it to obtain the target ciphertext  $C^* = C \parallel T^*$ , and reveals only the  $\mathbf{ts}$ -components. By the Lift Application corollary (Corollary 8), the derived random oracle adversary  $\mathcal{B}$  has  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$ ; it remains to bound the random oracle success against  $S^*$ . Here  $T^* = \lfloor \text{RF}(R_0) \rfloor_{\tau_{\text{root}}}$  is the root tag of the selector’s final root transcript  $R_0$  under  $(K_0, N_0, A_0)$ , and a winning  $\mathcal{A}$  outputs a context reaching a final root transcript  $R$  with  $\text{RF}(R)$  projecting to  $T^*$ .

Let  $Z_0$  be any context component hidden by  $\mathbf{ts}$ —one exists because  $\mathbf{ts} \subsetneq \{k, n, a\}$ —which  $\mathcal{A}$  must itself supply (for the named variants, the key for  $\mathbf{ts} \in \{\emptyset, \{n, a\}\}$ , the nonce for  $\mathbf{ts} = \{k, a\}$ , the associated data for  $\mathbf{ts} = \{k, n\}$ ), and which  $S^*$  sampled uniformly on  $\{0, 1\}^w$  with  $w = 256 \geq k$  and which  $\mathcal{A}$  never sees: the only values revealed to it, the body and tag of  $C^*$ , are RF projections independent of  $Z_0$ . If  $R = R_0$ , then  $\mathcal{A}$  has reconstructed the selector’s transcript, in particular its  $Z_0$ -component, which by Lemma 6 requires a candidate decoding under  $\text{KeyedTWOut}_{Z_0}$  to the hidden  $Z_0$ ; since  $Z_0$  stays uniform over the values no direct query has tested, Lemma 9 (hidden component  $Z_0$ ,  $w = 256$ ) bounds this by  $\text{KeyGuess}_k(N_{\text{prim}})$ . If  $R \neq R_0$ , the win is that  $R$  collides with  $R_0$  on its  $\tau_{\text{root}}$ -bit projection, which the Root tag Candidate Sequence lemma (Lemma 1) and the second-preimage corollary of Lemma 8, with the designated candidate  $\text{flat}(R_0)$ , bound by  $\text{RootPre}_{\tau_{\text{root}}}(N_{\text{prim}})$ . Adding the lift gives the bound, uniformly over  $\mathbf{ts}$ .  $\square$

**Theorem 10** (Random Oracle aSec). *For every aSec adversary  $\mathcal{B}$  in the TW128 random oracle model (Section 4.2),*

$$\text{Adv}_{\text{RO}}^{\text{aSec}}(\mathcal{B}) \leq \text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B})) + \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})).$$

*Proof.* Run the random oracle aSec game: the challenger draws the challenge input  $X$ , encrypts it to obtain  $T = \text{H}(X)$  at a final root transcript  $R_X$ , and gives  $X$  to  $\mathcal{B}$ , which outputs  $X' \neq X$ , winning if  $\text{H}(X') = T$ . Let  $\text{bad}_{\text{leaf}}^*$  be the event that two distinct construction-generated final leaf transcripts receive the same  $\tau_{\text{leaf}}$ -bit RF projection; by Lemma 10,  $\Pr[\text{bad}_{\text{leaf}}^*] \leq \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B}))$ , so it remains to bound the win under  $\neg \text{bad}_{\text{leaf}}^*$ . Encrypting  $X'$  reaches a final root transcript  $R'$ . If  $\text{flat}(R') = \text{flat}(R_X)$  then on  $\neg \text{bad}_{\text{leaf}}^*$  Final-Root Input Separation (Proposition 2) makes  $X$  and  $X'$  share  $(K, N, A, C)$ , hence the same message by determinism, so  $X' = X$ —a contradiction; thus  $R' \neq R_X$ . The challenge root tag  $T = \lfloor \text{RF}(R_X) \rfloor_{\tau_{\text{root}}}$  is exposed to  $\mathcal{B}$ , and a win places  $R'$ ’s root tag at  $T$ . The Root Tag Hash Bound (Lemma 2, part 3, with designated  $R_X$ ) bounds the win under  $\neg \text{bad}_{\text{leaf}}^*$  by  $\text{RootPre}_{\tau_{\text{root}}}(q_{\text{RO}}(\mathcal{B}))$ ; adding  $\Pr[\text{bad}_{\text{leaf}}^*]$  gives the theorem.  $\square$

## 7 Performance Evaluation

### 7.1 Optimized Implementations

We evaluate three implementations of TW128 sharing a common structure: a portable reference written in Go, an **amd64** implementation, and an **arm64** implementation. The two architecture-specific implementations replace only the  $\text{KECCAK-}p[1600, 12]$  permutation and the inner absorb/squeeze loops with hand-written assembly; the framing, domain separation, padding, and tree bookkeeping remain in the shared Go code. The portable reference uses a scalar permutation and a scalar eight-instance loop, and serves both as a correctness oracle for the assembly cores and as the fallback on other architectures.

The tree structure of TW128 exposes two distinct permutation workloads, and both architectures provide a separate core for each. The root duplex—which absorbs the associated data, processes chunk 0, and absorbs the leaf tag aggregation transcript—is inherently sequential, since each of its calls consumes the previous call’s output; it is driven by a *single-duplex* ( $\times 1$ ) core that permutes one 1600-bit state at a time, except for its chunk-0 message phase, which is fused into the first batched leaf call as described below.

The leaf duplexes, by contrast, operate on disjoint state and disjoint inputs (Section 3), so the implementation batches eight complete leaf chunks and runs them in lockstep through an *eight-way* ( $\times 8$ ) core. Because every leaf chunk is exactly  $\beta = 49\rho$  bytes, the eight instances step through an identical sequence of 49 full  $\rho$ -blocks (the first 48 closed with `MSG_MORE`, the last with `MSG_LAST`), so a single shared block position drives all eight. The eight-way state is held lane-major—25 lanes each holding the corresponding 64-bit word of all eight instances—so that the batched cores can address the eight instances of a lane with a single vector load.

Messages are decomposed into eight-chunk batches run on the  $\times 8$  core, and a leftover run of two to seven complete chunks is drained by a narrower kernel: on `amd64` a register-resident kernel processes the remainder in a single call, reading the active chunks in place, and on `arm64` a 2-wide kernel consumes it two chunks at a time. These remainder kernels read each chunk as a sequential stream, so they prefetch as well on the cold tail of a long message as on a cache-resident short one; on the `amd64` host they outperform both repeated single-duplex calls and padding the remainder to a full eight-chunk batch, at every remainder width. Only a single leftover complete chunk, a partial trailing chunk, and the count-aggregation tail fall back to the single-duplex ( $\times 1$ ) core.

Chunk 0 itself runs on the batched kernels rather than serially, a technique we refer to as *lane-0 fusion*. The root duplex’s chunk-0 message phase has the same kernel-visible block schedule as a leaf chunk—the same 49  $\rho$ -blocks with the same closing flags—and differs from a leaf only in its initial state, to which the kernels are indifferent; chunk 0 is also contiguous in the message with the first leaf chunks. The implementation therefore seeds lane 0 of the first batched call with the root state (after initialization and associated data processing) and runs the chunk-0 phase alongside up to seven leaves: in the  $\times 8$  core when the message has at least seven complete leaf chunks, and in the `amd64` remainder kernel or the `arm64` 2-wide kernel below that. Lane 0’s output state is written back to the root duplex, which then absorbs the leaf tags produced by the same call; no serial pass over chunk 0 precedes the leaf work. On `amd64` the assembly kernels handle only the 48 full `MSG_MORE` blocks of each chunk; the final `MSG_LAST` block and the leaf tag extraction are performed in portable Go, shared with the reference path, which keeps the byte-granular tail logic out of assembly. The `arm64` kernels process the complete chunk, extracting the leaf tags and storing the permutation states back in assembly—the writeback that allows the root duplex to occupy lane 0.

**Architecture-specific cores.** On `amd64` the cores are built from AVX-512 (selected once at startup from the `AVX512VL` CPUID feature bit) and AVX2, holding the lane-major state in ZMM or YMM registers and permuting it with no cross-lane shuffles. The steady-state  $\times 8$  kernel transposes each rate block into the lane-major layout with contiguous, prefetcher-friendly loads and stores, while the register-resident kernel that drains a two-to-seven-chunk remainder reads its active chunks in place by masked gather and scatter. On `arm64` the cores use NEON with the Armv8.2 SHA-3 extension (`FEAT_SHA3`, available on Apple Silicon and Neoverse V1/V2 and N2), one instance per 64-bit lane in the single-duplex core and two instances per 128-bit register in the  $\times 8$  and 2-wide kernels. Both batched paths store the permutation state back after the closing block—the writeback that lets the root duplex occupy lane 0. Section C gives the per-instruction detail, including the fused permutation instructions and the remainder-kernel strategies on each architecture.

**Constant time.** All three implementations are constant time with respect to the key, nonce, and plaintext. The permutation and the XOR-based absorb/squeeze contain no secret-dependent branches and no secret-dependent memory or vector indices. The only indexed table is the round-constant array, addressed by the public round number; the AVX-512 chunk transpose uses contiguous loads and stores, and the gather and scatter

that remain—for the partial rate lane and the register-resident remainder kernel—are addressed by the fixed public chunk strides under a mask set by the public chunk count, and the AVX2 remainder kernel’s redirection of inactive instances to its dummy block is likewise selected by the public chunk count. None of these depend on secret data.

## 7.2 Benchmark Methodology

We measure on two hosts, one per architecture. The **amd64** host is a Google Compute Engine **c4-standard-4** instance with an Intel Xeon Platinum 8581C (Emerald Rapids, 2.30 GHz base, AVX-512 including AVX512VL, AVX512\_VBMI2, GFNI, and VAES), two physical cores with simultaneous multithreading enabled, 48 KiB L1d and 32 KiB L1i per core, 2 MiB L2 per core, and a 260 MiB shared L3, running Debian 12 (kernel 6.1) under KVM. The guest does not expose a **cpufreq** interface, so the frequency governor and turbo state are not under our control; we mitigate the resulting variance statistically (below). The **arm64** host is an Apple M4 Pro (10 performance cores and 4 efficiency cores; per performance core, 128 KiB L1i, 64 KiB L1d, and a 16 MiB shared L2) running macOS 26.5.1 (Darwin 25.5), with FEAT\_SHA3 present. macOS exposes no userspace frequency control.

Measurements are collected by a standalone harness (**cmd/cpb**) that locks its goroutine to an OS thread and, for each algorithm, operation, and message length, first calibrates an iteration count that fills at least 100 ms, then records 100 samples, from which it reports the median cycles per byte and median wall-clock throughput together with the interquartile range as a dispersion measure (Table 7). Both metrics are read from the same timed loop, so the cycles-per-byte and throughput figures for a given measurement come from one interleaved run rather than two separately scheduled ones. Each sample reuses the same source and destination buffers, so the reported figures are warm-cache, steady-state rates. The single-threaded measurement leaves the SMT sibling of the **amd64** host idle. We report a sweep of seven message lengths from 1 B to 16 MiB.

The cycle counters differ by architecture, and this affects how the cycles-per-byte figures may be read. On **amd64** the harness reads RDTSC; on this part the time-stamp counter is invariant and ticks at the 2.30 GHz reference frequency, so the reported figures are *reference* cycles per byte and are unaffected by turbo (and convert directly to wall-clock time). On **arm64** the harness reads CNTVCT\_ELO and scales it by the ratio of the peak core frequency to CNTFRQ\_ELO; since Apple exposes no frequency API, the peak is auto-detected by timing a dependent integer-add chain and taking the top observed P-state, yielding *core* cycles per byte. Consequently the cycles-per-byte figures are comparable within an architecture and against the same-host AES-128-GCM baseline, but not directly across architectures; absolute throughput (Table 6) is the cross-platform metric.

The baseline is Go’s AES-128-GCM implementation (**crypto/aes** with **crypto/cipher**). It uses AES-NI and PCLMULQDQ on **amd64**, and the AES and PMULL instructions on **arm64**. Each timed AES-128-GCM call constructs its cipher, so its measurement includes the key schedule and the GHASH key-power precomputation; this mirrors the TW128 measurement, whose one-shot seal and open perform the root duplex initialization (one permutation) inside the timed call. Both directions are measured for each scheme; the two directions track each other to within a few percent across the sweep, with a worst-case divergence of 8% (TW128 on **arm64** at 64 B).

## 7.3 Throughput

Table 5 reports cycles per byte across the message-length sweep and Table 6 the corresponding wall-clock throughput in the bulk regime. The dominant feature of the TW128 rows is the descent produced by the tree-parallel leaf core: the per-byte cost falls as a message lengthens and its chunks occupy more SIMD lanes. A message of at most one chunk has no complete leaf chunk for chunk 0 to share a kernel with, so it runs entirely

**Table 5:** Median cycles per byte (100 samples). The **amd64** figures are RDTSC reference cycles at the 2.30 GHz invariant time-stamp counter; the **arm64** figures are core cycles at the auto-detected peak performance-core frequency. Cycle counts are not directly comparable across architectures (Section 7.2).

|                                                   | Message length |       |       |        |        |       |        |
|---------------------------------------------------|----------------|-------|-------|--------|--------|-------|--------|
|                                                   | 1 B            | 64 B  | 8 KiB | 32 KiB | 64 KiB | 1 MiB | 16 MiB |
| <i>Intel Xeon 8581C (Emerald Rapids), AVX-512</i> |                |       |       |        |        |       |        |
| TW128 encrypt                                     | 590            | 9.27  | 1.89  | 0.78   | 0.40   | 0.39  | 0.40   |
| TW128 decrypt                                     | 615            | 9.72  | 1.90  | 0.75   | 0.40   | 0.39  | 0.39   |
| AES-128-GCM seal                                  | 765            | 13.23 | 0.38  | 0.32   | 0.30   | 0.30  | 0.30   |
| AES-128-GCM open                                  | 785            | 12.79 | 0.37  | 0.31   | 0.30   | 0.29  | 0.29   |
| <i>Intel Xeon 8581C (Emerald Rapids), AVX2</i>    |                |       |       |        |        |       |        |
| TW128 encrypt                                     | 656            | 10.31 | 2.20  | 1.10   | 1.02   | 1.07  | 1.08   |
| TW128 decrypt                                     | 683            | 10.70 | 2.21  | 1.11   | 1.08   | 1.07  | 1.08   |
| <i>Apple M4 Pro, NEON + FEAT_SHA3</i>             |                |       |       |        |        |       |        |
| TW128 encrypt                                     | 620            | 9.77  | 1.95  | 0.91   | 0.88   | 0.88  | 0.89   |
| TW128 decrypt                                     | 667            | 10.59 | 2.00  | 0.89   | 0.87   | 0.87  | 0.87   |
| AES-128-GCM seal                                  | 1044           | 17.28 | 0.54  | 0.45   | 0.44   | 0.43  | 0.44   |
| AES-128-GCM open                                  | 1055           | 17.31 | 0.52  | 0.43   | 0.42   | 0.41  | 0.42   |

on the single-duplex core, and the 8 KiB column sits at that core’s rate. Past one chunk, lane-0 fusion (Section 7.1) keeps every complete chunk on the batched kernels, and the descent tracks lane occupancy: at 32 KiB, chunk 0 and three complete leaves run four-wide in the **amd64** remainder kernels and as two full pairs on **arm64**, and 64 KiB is the first length whose eight complete chunks ( $8\beta \approx 64$  KiB) fill the eight-way ( $\times 8$ ) core. On the **amd64** AVX-512 path TW128 encryption falls  $1.89 \rightarrow 0.78 \rightarrow 0.40$  across the 8, 32, and 64 KiB columns and is already at its bulk rate at 64 KiB, holding at 0.39–0.40 through 16 MiB. The AVX2 path follows the same shape at lower throughput ( $2.20 \rightarrow 1.10$  over the same span, thereafter between 1.02 and 1.08); it runs the eight instances as two groups of four rather than in a single 8-wide register, leaving the AVX-512 core about  $2.7\times$  faster in bulk. The **arm64** curve flattens earliest ( $1.95 \rightarrow 0.91 \rightarrow 0.88 \rightarrow 0.88 \rightarrow 0.89$  cycles per byte from 8 KiB to 16 MiB): the eight-way core itself processes its batch as four two-instance pairs, so the fused 2-wide kernel runs at essentially the same per-chunk rate, and a 32 KiB message—chunk 0 and three leaves as two full pairs—is already within 3% of the 16 MiB floor. In wall-clock terms TW128 reaches 5.77 GB/s on the **amd64** AVX-512 host and 5.04 GB/s on the M4 Pro for 16 MiB messages.

**Dispersion.** Each cell of Tables 5 and 6 is the median of 100 samples; Table 7 summarizes their spread as the interquartile range (IQR) relative to the median. The medians are stable: across the sweep the relative IQR of the TW128 cycles-per-byte measurements is at most 1.6% on the **amd64** AVX-512 path, 2.1% on AVX2, and 1.0% on the M4 Pro, with per-path medians of 1.2%, 1.4%, and 0.6%. The full min–max range is wider on the **amd64** guest—up to 16.5% (AVX-512) and 20.2% (AVX2) at isolated lengths—because its turbo and governor states are not under our control (Section 7.2); these are tail excursions that the IQR excludes, and the M4 Pro, whose frequency is steadier, keeps even its full range within 4.7%. We therefore report medians throughout and treat the IQR as the dispersion measure.

**Table 6:** Measured wall-clock throughput (GB/s, GB =  $10^9$  bytes) from the same `cmd/cpb` run as Table 5, encryption direction; the decryption direction tracks it. The `amd64` AES-128-GCM row is the AVX-512 host and is unaffected by the TW128 core selection.

|                                          | 8 KiB | 32 KiB | 64 KiB | 1 MiB | 16 MiB |
|------------------------------------------|-------|--------|--------|-------|--------|
| <i>Intel Xeon 8581C (Emerald Rapids)</i> |       |        |        |       |        |
| TW128 (AVX-512)                          | 1.22  | 2.95   | 5.78   | 5.94  | 5.77   |
| TW128 (AVX2)                             | 1.05  | 2.09   | 2.25   | 2.15  | 2.14   |
| AES-128-GCM                              | 6.11  | 7.27   | 7.57   | 7.68  | 7.63   |
| <i>Apple M4 Pro</i>                      |       |        |        |       |        |
| TW128 (NEON+SHA3)                        | 2.30  | 4.92   | 5.06   | 5.10  | 5.04   |
| AES-128-GCM                              | 8.25  | 9.89   | 10.18  | 10.33 | 10.09  |

**Table 7:** Cycles-per-byte measurement dispersion for TW128 across the seven-length sweep (seal and open, 100 samples per point), as the interquartile range relative to the median. The final column is the widest single-point min–max range; its size on the `amd64` guest reflects the turbo and governor states we cannot pin (Section 7.2) and is excluded from the IQR as a tail effect.

| Path                | Relative IQR |      | Max   |
|---------------------|--------------|------|-------|
|                     | median       | max  | range |
| Xeon 8581C, AVX-512 | 1.2%         | 1.6% | 16.5% |
| Xeon 8581C, AVX2    | 1.4%         | 2.1% | 20.2% |
| Apple M4 Pro        | 0.6%         | 1.0% | 4.7%  |

## 7.4 Latency

For short messages the cost is set by the serial root duplex and is independent of the parallel leaf machinery. A message of at most  $\rho = 167$  bytes occupies a single rate block and never enters leaf mode, so it costs exactly two permutations—one to initialize the root duplex and one to close the message block and extract the tag. The 1 B and 64 B columns of Table 5 bear this out: per-message latency is nearly flat across them at roughly 590 reference cycles on `amd64` (590 at 1 B versus  $9.27 \times 64 \approx 593$  at 64 B) and 620 core cycles on `arm64` (620 versus  $9.77 \times 64 \approx 625$ ), i.e. about 295 and 310 cycles per KECCAK- $p[1600, 12]$  respectively. Because such messages bypass the leaf core entirely, leaf-level parallelism imposes no latency penalty on them.

The latency/throughput trade-off appears instead in the intermediate regime, which lane-0 fusion has narrowed to roughly the one-to-eight-chunk band. The worst per-byte cost in the sweep is the single-chunk regime at 8 KiB (1.9–2.2 cycles per byte across the three paths), where the message is the chunk-0 phase alone on the single-duplex core. Once the message has complete leaf chunks for chunk 0 to share a kernel with, the cost falls quickly: by 32 KiB TW128 is within a factor of two of its AVX-512 floor and within 3% of its `arm64` floor, and the floor itself is reached once the eight-way core fills at 64 KiB. The root duplex also bounds the tail of a long message: the final tag cannot be produced until chunk 0 has been processed and every leaf tag has been absorbed into the aggregation transcript. That absorption is cheap—each rate block of the root absorbs five 32-byte leaf tags, so a 16 MiB message ( $\approx 2000$  leaves) adds on the order of 400 root permutations against roughly  $10^5$  permutations of leaf work, under 0.5% overhead—so the serial root does not materially cap bulk throughput.

## 7.5 Comparison with Other AEADs

We compare against the one baseline measured under the same harness on the same hosts, the Go standard library’s hardware-accelerated AES-128-GCM (Tables 5 and 6). The harness times TW128 and AES-128-GCM back-to-back at each length and reads both metrics from the same loop, so the cycles-per-byte and throughput comparisons hold under identical conditions and agree. In the bulk regime AES-128-GCM is faster on both architectures, as expected for a primitive with dedicated AES and carryless-multiply instructions: at 16 MiB it runs at 0.30 cycles per byte on `amd64` and 0.44 on `arm64`, against 0.40 and 0.89 for TW128—about  $1.3\times$  and  $2\times$  faster. The wall-clock throughputs show the same margins: TW128 reaches 5.77 and 5.04 GB/s against AES-128-GCM’s 7.63 and 10.09 GB/s. The wider `arm64` gap reflects the strength of Apple’s AES and PMULL units. TW128 attains these rates with a permutation that has no dedicated hardware support, relying only on the SHA-3 extension on `arm64` and on general SIMD on `amd64`.

The ordering reverses for very short messages, where AES-128-GCM’s key schedule and GHASH key-power precomputation dominate: at 1 B AES-128-GCM costs 765 and 1044 cycles on the two hosts against TW128’s 590 and 620. This comparison is setup-bound rather than representative of steady state, since each timed call rebuilds its cipher; it nonetheless reflects the per-message cost an application pays when it does not amortize key setup across messages. In the sub-bulk regime AES-128-GCM is several times faster at one chunk (0.38 versus 1.89 cycles per byte at 8 KiB on `amd64`), but the gap narrows quickly once the fused kernels engage: about  $2.4\times$  at 32 KiB, and the bulk ratio from 64 KiB on. TW128’s intended value relative to AES-128-GCM lies in the tree structure and committing security properties analyzed in Sections 5 and 6 rather than in raw speed.

## 8 Discussion

### 8.1 Comparison Table

Table 8 situates TW128 against representative authenticated encryption schemes along the axes that distinguish it: the underlying primitive, whether the mode is parallelizable, the key, nonce, and tag sizes, associated data support, nonce-misuse resistance, full-input key commitment (CMT-4), and context discoverability security (CDY). The comparison is structural; TW128’s own concrete bounds and the regimes in which each term dominates are treated in Section 5.5, and its measured throughput against hardware-accelerated AES-128-GCM in Section 7.5. The CMT-4 column records full-input commitment: TW128 provides it within the mode, as does Ascon [KSW23], a committing transform supplies it for an arbitrary base scheme, and the remaining listed schemes do not provide it. The CDY column records context discoverability security in the [MLGR23] sense: TW128 provides it within the mode (Theorem 6), and the remaining entries are marked “?” because, to our knowledge, neither a context discovery attack nor a context discoverability security proof has been published for them. Context discoverability is a recent notion [MLGR23], so its absence of entries reflects the state of the literature rather than a negative result; the schemes [MLGR23] break with context discovery attacks (GCM, CCM, EAX, SIV, OCB3) are not among the rows above. The table does not separately track compact (tag-only) commitment, since that property has not been systematically analyzed for the comparison schemes.

### 8.2 Comparison with Prior Sponge AEADs

TW128 belongs to the family of single-permutation duplex AEADs initiated by [BDPVA11]. The dedicated lightweight members of that family, Ascon [DEMS21] and Xoodyak [DHP<sup>+</sup>20], target 128-bit security on small permutations (320 and 384 bits) with

**Table 8:** Structural comparison of TW128 with representative AEADs. Sizes are in bytes; AES-GCM-SIV also admits a 32-byte key. “Misuse-res.” is nonce-misuse resistance in the sense of [RS06]; the CMT-4 column records full-input commitment in the sense of [BH22], abstracting over its decryption (CMTD) and encryption (CMT) forms, of which TW128’s is the decryption form CMTDs-4; CDY is context discoverability in the sense of [MLGR23]. A dash means the entry is inherited from the base scheme, and “?” that no result is established in the literature. TW128’s concrete bounds appear in Section 5.5.

| Scheme                       | Primitive              | Par. | K/N/T    | AD  | Misuse | CMT-4 | CDY |
|------------------------------|------------------------|------|----------|-----|--------|-------|-----|
| TW128                        | KECCAK- $p$ [1600, 12] | yes  | 32/32/32 | yes | no     | yes   | yes |
| Ascon-128a [DEMS21]          | Ascon- $p$ (320 b)     | no   | 16/16/16 | yes | no     | yes   | ?   |
| Xoodoo [DHP <sup>+</sup> 20] | Xoodoo (384 b)         | no   | 16/16/16 | yes | no     | no    | ?   |
| AES-GCM-SIV [GLL17]          | AES                    | yes  | 16/12/16 | yes | yes    | no    | ?   |
| nAE + CTX [CR22]             | any nAE                | —    | —        | yes | —      | yes   | ?   |

a strictly serial duplex: each call depends on the previous one, so there is no mode-level parallelism to exploit. TW128 instead pays for a larger permutation, KECCAK- $p$ [1600, 12], and a 256-bit capacity—the same round count as KangarooTwelve [BDP<sup>+</sup>16]—in exchange for a tree that processes leaf chunks in parallel and for the headroom that yields 128-bit security with a comfortable margin. The cost of this choice is visible in Section 7: on short, single-leaf inputs TW128 is slower per byte than a compact serial duplex would be, and its advantage appears only once a message spans enough leaves to fill the vector units.

TW128 reuses KangarooTwelve’s final-node/leaf split but not its Sakura coding, which renders that scheme’s tree hashing sound: the frame bits and coded child count of a Sakura coding make every node input unambiguously decodable, so that any collision in the hash output entails a collision in the inner function. TW128 adopts no such coding and does not need one. Its tree has a single fixed two-level shape—one root and a leaf count fixed by  $\|C\|$  through  $\text{leaves}(\cdot)$ —so it never distinguishes among a family of admissible topologies, the generality a Sakura coding exists to serve; the leaf count is moreover pinned by absorbing  $\langle n \rangle_8$  before the root tag. The unambiguous parsing such a coding would supply is instead furnished by the per-call domain separation of Section 3.4: the prefixes  $p_{\text{root}}$ ,  $p_{\text{leaf}}$  and the leaf index  $\langle j \rangle_8$  fix the duplex instance, and the seven pairwise-distinct phase suffixes of Table 3 fix the phase and the final/non-final role of every call, so distinct transcripts flatten to distinct sponge inputs (Lemma 4 and Corollary 7); binding the index into each leaf is in fact a finer separation than ordering leaves within an aggregation alone. More to the point, the committing and discoverability bounds of Section 6 do not rest on soundness of a tree coding at all: distinct  $(K, N, A)$  triples already diverge in the root initialization and associated data calls, before any leaf tag is aggregated (Proposition 1), so agreement among leaf tags cannot merge distinct contexts. The leaf encoding enters only the collision bound of Theorem 3, and there only through its explicit leaf tag collision term. A Sakura coding would re-establish parsing facts TW128 already possesses, and would leave these bounds unchanged.

Keyak [BDP<sup>+</sup>15], whose Motorist mode is already parallelizable, is the closest prior duplex AEAD in spirit; Section 1.2 contrasts its wire-visible, encryptor-and-decryptor-shared degree of parallelism with TW128’s run-time leaf choice. Strobe [Ham17] is a serial sponge framework aimed at whole protocols rather than at a standalone AEAD, and like Ascon and Xoodoo it does not parallelize. Committing security is not unique to TW128 in this family: a dedicated analysis of the sponge-based NIST lightweight finalists proves Ascon committing while breaking others [KSW23], and some constructions, such as Strobe, have not been examined in that setting at all. What distinguishes TW128 is that it makes the root tag a secure compact commitment to the full input—collision-, second-preimage-, and preimage-resistant as a hash, which in particular yields the strongest full-input committing notion (CMT-4)—as a proven, designed property of the mode, and additionally

resists [MLGR23] context discovery, established without leaving the single-permutation duplex setting; context discoverability security in particular is not known to hold for the other members of the family.

### 8.3 Comparison with Committing AEAD Constructions

The dominant route to committing AEAD is a generic transform over a non-committing base scheme. [BH22] give the UtC, RtC, and HtE transforms, which add key- or context-commitment to an arbitrary AEAD, and [CR22] give CTX, which makes any nonce-based AE scheme commit to its full context at the cost of a single hash over a short string and with no ciphertext expansion. These transforms are inexpensive and broadly applicable; CTX in particular is nearly free. TW128 differs not by undercutting their cost but by integration: it is committing as the mode itself, with the same permutation and the same tag that already provide confidentiality and integrity, so no separate commitment primitive, key-derivation step, or auxiliary hash is invoked, and the strongest  $s$ -way full-input notion follows from a single root tag collision bound (Section 6). More strongly, Theorem 3 shows that the root tag alone is a compact commitment string even when the ciphertext bodies used as openings differ, up to the additional leaf tag birthday term. The same tag, being preimage-resistant, additionally yields context discoverability security against that fixed tag, which the context-committing transforms are not known to provide. This places TW128 alongside the dedicated committing primitives—the compactly committing AEAD of [GLR17] and the encryption of [DGRW18]—which likewise build commitment in rather than bolt it on, but which were designed for message franking rather than as general nonce-based AEADs. The contrast with transformed conventional schemes is sharpened by the attacks of Section 1.2: base schemes such as GCM and OCB are not committing on their own [CR22], and a transform is the only way to make them so.

Closest to TW128 here is the recent work of [DHM<sup>+</sup>24], which builds nonce-based AE directly on the SHA-3 functions SHAKE and TurboSHAKE—the latter using the same round-reduced KECCAK- $p$  permutation as TW128—and likewise achieves CMT-4 natively, from the collision resistance of the underlying function rather than from a transform. The two designs make different structural choices: [DHM<sup>+</sup>24] targets session-based communication, chaining a stream of messages through intermediate tags and generalizing the keyed duplex into a *duplex cipher*, whereas TW128 is a single two-level tree whose independent leaves expose run-time-selectable parallelism at constant memory (Sections 1.2 and 7.1). Because a session chain processes each message serially, TW128’s principal advantage is throughput on large messages (Section 7): its parallel leaves fill vector units a serial duplex leaves idle, at the cost of the session support [DHM<sup>+</sup>24] provides and TW128 does not. Both demonstrate that committing AE needs no add-on transform once it is built on a well-scrutinized KECCAK- $p$  permutation.

### 8.4 Limitations and Open Problems

**Nonce misuse.** TW128’s AE and mu-ind bounds require nonce-respecting encryption queries, and the mode is not misuse-resistant: a repeated nonce reuses leaf keystream and breaks confidentiality, as for any online duplex stream cipher. Schemes such as AES-GCM-SIV [GLL17], in the deterministic/misuse-resistant lineage of [RS06], tolerate nonce repetition at the price of a second pass. A misuse-resistant variant of TW128 is left open.

**Release of unverified plaintext.** The analysis assumes that plaintext is released only after the tag verifies. TW128 is online and single-pass, so an implementation that streams decrypted leaf output before checking the root tag operates in the release-of-unverified-

1587 plaintext setting of [ABL<sup>+</sup>14], which our bounds do not cover; plaintext-awareness and  
 1588 INT-RUP security for TW128 are unstudied.

1589 **Bound tightness.** Per-term tightness is itemized in Section 5.5; one term is conservative  
 1590 in a way that points to an open problem. The flat sponge differentiating loss  $\text{Lift}_c(N_{\text{tot}}) =$   
 1591  $N_{\text{tot}}(N_{\text{tot}} + 1)/2^{c+1}$  bounds the indistinguishability loss of [BDPVA08], used here as a generic  
 1592 lift from the random oracle model. A direct analysis of the keyed duplex is tighter: the  
 1593 modern keyed-duplex bounds synthesized by [Men23], building on the full-state keyed-  
 1594 duplex line, bound a keyed duplex against an ideal permutation without passing through  
 1595 indistinguishability, and recasting TW128’s leaf and root duplexes in that framework is a  
 1596 promising route to a sharper capacity term. This route is not turnkey, however: the keyed  
 1597 duplex bundles squeezing and absorption into a single call, so its authenticated encryption  
 1598 mode `MonkeySpongeWrap` has the decryptor *overwrite* the rate with each ciphertext block  
 1599 rather than feed back the ciphertext it has just produced. On the final, padded block  
 1600 this overwrite no longer matches the encryptor’s XOR-absorbed padded plaintext outside  
 1601 the message bits, so the decryptor cannot reconstruct the final block of duplex output  
 1602 and recomputes the tag from a divergent state [Men23, Alg. 8]. TW128 sidesteps this  
 1603 by absorbing the ciphertext in both directions (Section 3), so recasting it on the keyed  
 1604 duplex would first require repairing the mode. We retain the indistinguishability lift for its  
 1605 modularity, as the resulting  $\text{Lift}_c$  already meets the 128-bit target (Section 5.5).

1606 **Beyond-birthday and leakage.** All bounds are birthday-type on the 256-bit capacity,  
 1607 delivering the 128-bit target but no more; a beyond-birthday analysis is open. The model  
 1608 is also single-stage and leakage-free: side-channel resistance is addressed only at the  
 1609 implementation level, through the constant-time properties of Section 7.1, and is not part  
 1610 of the provable-security claims.

## 1611 9 Conclusion

1612 TW128 is a two-level tree of keyed duplexes over a single KECCAK- $p$ [1600, 12] permutation  
 1613 that combines mode-level parallelism, constant memory, and commitment security in both  
 1614 its collision and preimage forms. Its concrete public permutation bounds—for multi-user  
 1615 indistinguishability, all-in-one AE as its single-user corollary, the secure-hash security of the  
 1616 root tag wrapper (collision, second-preimage, and preimage resistance) that makes the tag a  
 1617 compact commitment, the  $s$ -way full-input committing notion (CMTDs-4), and [MLGR23]  
 1618 context discoverability (CDY\*)—meet a 128-bit target at the implemented parameters,  
 1619 and vectorized `amd64` and `arm64` implementations turn the tree’s leaf parallelism into  
 1620 multi-gigabyte per second throughput within a small factor of hardware-accelerated AES-  
 1621 128-GCM.

1622 Several questions remain open (Section 8.4): a nonce-misuse-resistant variant, security  
 1623 under the release of unverified plaintext, and sharper bounds—whether by recasting the  
 1624 leaf and root duplexes on the modern keyed-duplex analysis or through a beyond-birthday  
 1625 argument. More broadly, TW128 adds to the growing evidence that a single, well-scrutinized  
 1626 KECCAK- $p$  permutation suffices to build authenticated encryption that is at once fast,  
 1627 parallel, and committing.

## 1628 Acknowledgments

1629 A hearty thank-you to my wife, who remains reasonably mystified at my choice of hobbies.

## References

- [ABL<sup>+</sup>14] Elena Andreeva, Andrey Bogdanov, Atul Luykx, Bart Mennink, Nicky Mouha, and Kan Yasuda. How to securely release unverified plaintext in authenticated encryption. In *Advances in Cryptology – ASIACRYPT 2014*, volume 8873 of *Lecture Notes in Computer Science*, pages 105–125. Springer, 2014.
- [ADG<sup>+</sup>22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3291–3308. USENIX Association, 2022.
- [BDP<sup>+</sup>15] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. CAESAR submission: Keyak v2. CAESAR Competition, Round 3 Candidate, 2015.
- [BDP<sup>+</sup>16] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. KangarooTwelve: Fast hashing based on Keccak-p. *Cryptology ePrint Archive*, Paper 2016/770, 2016.
- [BDPVA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indistinguishability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.
- [BDPVA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the sponge: Single-pass authenticated encryption and other applications. In *Selected Areas in Cryptography – SAC 2011*, volume 7118 of *Lecture Notes in Computer Science*, pages 320–337. Springer, 2011.
- [BDPVA14] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sakura: A flexible coding for tree hashing. In *Applied Cryptography and Network Security – ACNS 2014*, volume 8479 of *Lecture Notes in Computer Science*, pages 217–234. Springer, 2014.
- [BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, volume 13276 of *Lecture Notes in Computer Science*, pages 845–875. Springer, 2022.
- [BT16] Mihir Bellare and Björn Tackmann. The multi-user security of authenticated encryption: AES-GCM in TLS 1.3. In *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *Lecture Notes in Computer Science*, pages 247–276. Springer, 2016.
- [CR22] John Chan and Phillip Rogaway. On committing authenticated encryption. In *Computer Security – ESORICS 2022*, *Lecture Notes in Computer Science*. Springer, 2022.
- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.
- [DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryption. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 155–186. Springer, 2018.

- [DHM<sup>+</sup>24] Joan Daemen, Seth Hoffert, Silvia Mella, Gilles Van Assche, and Ronny Van Keer. Shaking up authenticated encryption. Cryptology ePrint Archive, Paper 2024/1618, 2024.
- [DHP<sup>+</sup>20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodyak, a lightweight cryptographic scheme. *IACR Transactions on Symmetric Cryptology*, 2020(S1):60–87, 2020.
- [GLL17] Shay Gueron, Adam Langley, and Yehuda Lindell. AES-GCM-SIV: Specification and analysis. Cryptology ePrint Archive, Paper 2017/168, 2017.
- [GLR17] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *Advances in Cryptology – CRYPTO 2017*, volume 10403 of *Lecture Notes in Computer Science*, pages 66–97. Springer, 2017.
- [Ham17] Mike Hamburg. The STROBE protocol framework. Cryptology ePrint Archive, Paper 2017/003, 2017.
- [KSW23] Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Committing AE from sponges: Security analysis of the NIST LWC finalists. Cryptology ePrint Archive, Paper 2023/1525, 2023.
- [LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 195–212. USENIX Association, 2021.
- [Men23] Bart Mennink. Understanding the duplex and its security. *IACR Transactions on Symmetric Cryptology*, 2023(2):1–46, 2023.
- [MLGR23] Sanketh Menda, Julia Len, Paul Grubbs, and Thomas Ristenpart. Context discovery and commitment attacks: How to break CCM, EAX, SIV, and more. In *Advances in Cryptology – EUROCRYPT 2023*, volume 14007 of *Lecture Notes in Computer Science*, pages 379–407. Springer, 2023.
- [Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication FIPS PUB 202, U.S. Department of Commerce, August 2015.
- [RS04] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption – FSE 2004*, volume 3017 of *Lecture Notes in Computer Science*, pages 371–388. Springer, 2004.
- [RS06] Phillip Rogaway and Thomas Shrimpton. A provable-security treatment of the key-wrap problem. In *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 373–390. Springer, 2006.

## A Supporting Lemmas

The proof core separates structural decoding from probabilistic accounting. The well-formed transcript language of Definition 1 fixes which flattened queries are valid TW128 construction prefixes. Lemmas 3, 4 and 6 then show how such a query is flattened,

1717 parsed as a typed transcript, assigned an output point, and decoded to a candidate key,  
 1718 when any. Lemma 9 turns those decoded keys into an adaptive key-test bound. The  
 1719 remaining proof interfaces expose the events their consumers must control: encryption-side  
 1720 freshness, verification hidden-key exposure and root tag guessing, leaf tag collisions, and  
 1721 the committing candidate-sequence collision event. Thus the mu-ind and committing proofs  
 1722 only have to derive the relevant local event premise before applying the corresponding  
 1723 accounting argument.

## 1724 A.1 Duplex Flattening

1725 **Lemma 3** (Duplex Flattening). *Every TW128 root or leaf duplex output is equal to a*  
 1726 *[BDPVA11] sponge output on the corresponding flattened duplex-call transcript, and the*  
 1727 *ciphertext-absorbing message phase admits a flattened duplex transcript that is well defined*  
 1728 *for both encryption and decryption.*

1729 *Proof.* Let  $D$  be a duplex object using permutation  $\pi$ , rate  $r$ , and padding  $\text{pad10}^*1$ . For a  
 1730 sequence of duplex calls  $Z_i = D.\text{duplexing}(\sigma_i, \ell_i)$  with each  $\sigma_i$  short enough to fit in one  
 1731 duplex block after padding, [BDPVA11, Lemma 3] gives

$$1732 \quad Z_i = \text{Sponge}[\pi, \text{pad10}^*1, r](\sigma_0 \parallel \text{pad}_0 \parallel \sigma_1 \parallel \text{pad}_1 \parallel \cdots \parallel \sigma_i, \ell_i),$$

1733 where  $\text{pad}_j = \text{pad10}^*1[r](|\sigma_j|)$ , and Lemma 4 says that, for fixed  $\text{pad10}^*1$  and  $r$ , the map  
 1734 from the duplex-call sequence to the flattened sponge input is injective.

1735 In TW128, each duplex call absorbs a byte-aligned block  $\sigma$  followed by a 4-bit phase  
 1736 suffix  $\sigma_{\text{ph}} \in \{\sigma_{\text{init}}, \sigma_{\text{ad}}^-, \dots, \sigma_{\text{agg}}^+\}$  (Section 3.4), so the duplex-call input is  $\sigma \parallel \sigma_{\text{ph}}$  followed  
 1737 by  $\text{pad10}^*1$ . With  $r = 1344$  and  $\rho_{\text{max}}(\text{pad10}^*1, r) = r - 2 = 1342$  bits, the maximum  
 1738 byte-aligned input is  $\rho = 167$  bytes (Section 3.2), giving a worst-case duplex-call input of  
 1739  $8 \cdot 167 + 4 = 1340 \leq 1342$  bits. The two initialization calls are shorter still:  $|p_{\text{root}} \parallel K \parallel N| + 4 =$   
 1740  $8 \cdot (6 + 32 + 32) + 4 = 564$  bits and  $|p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8| + 4 = 8 \cdot (6 + 32 + 32 + 8) + 4 = 628$   
 1741 bits. [BDPVA11, Lemmas 3 and 4] therefore apply independently to every TW128 duplex  
 1742 object.

1743 The root duplex is initialized by absorbing  $p_{\text{root}} \parallel K \parallel N$  under  $\sigma_{\text{init}}$  (Algorithm 2),  
 1744 then absorbs the associated data blocks under  $\sigma_{\text{ad}}^- / \sigma_{\text{ad}}^+$ , the root ciphertext blocks in the  
 1745 message phase under  $\sigma_{\text{msg}}^- / \sigma_{\text{msg}}^+$ , and the leaf tag aggregation blocks under  $\sigma_{\text{agg}}^- / \sigma_{\text{agg}}^+$ . By  
 1746 the duplex-to-sponge equality, every root duplex output is the sponge function evaluated  
 1747 on the root transcript prefix accumulated up to the relevant output point. The same holds  
 1748 for each leaf duplex, initialized by  $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$  and absorbing only its own ciphertext  
 1749 blocks in the message phase. In decryption and verification, Algorithm 3 absorbs the  
 1750 submitted ciphertext blocks before the root tag comparison, so the same flattened message  
 1751 phase transcript is defined whether the comparison accepts or rejects.  $\square$

## 1752 A.2 Well-formed Transcript Prefixes

1753 For a duplex-call prefix  $((\sigma_0, \sigma_{\text{ph},0}), \dots, (\sigma_i, \sigma_{\text{ph},i}))$ , write

$$1754 \quad \text{Flat}((\sigma_0, \sigma_{\text{ph},0}), \dots, (\sigma_i, \sigma_{\text{ph},i})) = \left\| \bigg|_{j=0}^{i-1} (\sigma_j \parallel \sigma_{\text{ph},j} \parallel \text{pad}_j) \parallel \sigma_i \parallel \sigma_{\text{ph},i} \right.$$

1755 The current call's padding  $\text{pad}_i$  is not part of  $\text{Flat}$ ; it is applied internally by the sponge  
 1756 function. Equivalently, after the  $i$ -th call the duplex state is the sponge state after absorbing  
 1757  $\text{Flat}(\cdot) \parallel \text{pad}_i$ .

1758 **Definition 1** (Well-Formed TW128 Transcript Prefixes). A typed TW128 transcript prefix  
 1759 is *well formed* if it is a prefix, ending at a construction transcript lookup point, of one of  
 1760 the following two call languages.

- A root transcript starts with  $(p_{\text{root}} \parallel K \parallel N, \sigma_{\text{init}})$ , with fixed-width fields  $K \in \{0, 1\}^k$  and  $N \in \{0, 1\}^\nu$ . It then absorbs the associated data byte string (only when the associated data is nonempty; the phase is otherwise elided, Section 3.5), the root ciphertext byte string, and the aggregation byte string (only when  $n \geq 1$ ; when  $n = 0$  the aggregation phase is elided, Section 3.5, and the root tag is emitted by the final  $\sigma_{\text{msg}}^+$  message call), in that order. Each present phase byte string is partitioned by the block rule of Algorithm 1: the empty string gives one empty final block, and a nonempty string gives zero or more  $\rho$ -byte non-final blocks followed by one final block. The phases use respectively  $\sigma_{\text{ad}}^-/\sigma_{\text{ad}}^+$ ,  $\sigma_{\text{msg}}^-/\sigma_{\text{msg}}^+$ , and  $\sigma_{\text{agg}}^-/\sigma_{\text{agg}}^+$ .
- A leaf- $j$  transcript starts with  $(p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8, \sigma_{\text{init}})$ , where  $1 \leq j < 2^{64}$  and the fields have fixed width. It then absorbs the leaf ciphertext byte string under  $\sigma_{\text{msg}}^-/\sigma_{\text{msg}}^+$ , using the same block rule.

The message byte string in this language is always the ciphertext byte string absorbed by the construction: encryption obtains it by XORing the plaintext with the current keystream, while decryption and verification use the submitted ciphertext. Thus a verification transcript is well formed for every in-domain ciphertext before the tag comparison is made; acceptance is not part of the transcript language.

The root aggregation byte string has the form  $T_1 \parallel \dots \parallel T_n \parallel \langle n \rangle_8$ , where each  $T_j$  has fixed byte length  $\tau_{\text{leaf}}/8$ . The trailing fixed-width field therefore gives a unique parse of the leaf tag sequence. The output-point class of a well-formed counted prefix is the class in Table 9 determined by its duplex instance and final-call suffix.

Name the tag-emitting transcript families:

$\mathcal{R}_{\text{tag}}$  = root transcript prefix whose sponge output gives the final root tag,

$\mathcal{L}_{\text{tag}}(j)$  = leaf- $j$  transcript prefix whose sponge output gives the leaf tag.

The final root tag is emitted by the  $\sigma_{\text{agg}}^+$  aggregation call (the  $\mathcal{R}_{\text{tag}}$  family) when  $n \geq 1$ , and by the closing  $\sigma_{\text{msg}}^+$  message call (an  $\mathcal{R}_{\text{msg}}^+$  prefix, in the notation of Corollary 7) when  $n = 0$  and the aggregation phase is elided. Since  $n = \text{leaves}(C)$  is fixed by  $\|C\|$ , every computation on a given ciphertext emits its tag at the one of these two output points selected by  $\|C\|$ . Intermediate construction transcript lookup points (init, non-final AD, non-final message, and non-final aggregation) are also typed prefixes covered by Lemma 6; the exact keyed direct-query decoder names them in Table 9.

### A.3 Transcript Injectivity

**Lemma 4** (Transcript Injectivity). *Well-formed TW128 flattened duplex-call transcript prefixes are injectively encoded and separated by duplex instance (root vs leaf), leaf index  $j$ , phase suffix, and duplex-call order. For final-root prefixes, equality of flattened inputs recovers the same root initialization, AD blocks, root ciphertext blocks, aggregation input, and leaf tag sequence.*

*Proof.* By Lemma 3, every well-formed output point can be written as a [BDPVA11] flattened sponge input for the corresponding duplex-call prefix, namely Flat as defined above. The deterministic current-call  $\text{pad10}^*1$  is applied internally by the sponge function and is not part of the input to which [BDPVA11, Lemma 4] is applied. That lemma recovers the duplex-call sequence from the flattened sponge input; in TW128, each call input is  $\sigma \parallel \sigma_{\text{ph}}$  with byte-aligned  $\sigma$  and fixed 4-bit suffix  $\sigma_{\text{ph}}$ , so recovering the call input recovers the pair  $(\sigma, \sigma_{\text{ph}})$ .

Definition 1 fixes the root and leaf initialization fields, the admissible phase order, and the final-call suffixes. The distinct prefixes  $p_{\text{root}}$  and  $p_{\text{leaf}}$  separate root from leaf transcripts; the fixed-width leaf field  $\langle j \rangle_8$  separates leaves; and the seven suffix values of

Table 3 separate phases and non-final/final calls. Thus equality of flattened inputs forces equality of the recovered instance type, key, nonce, leaf index when present, phase suffixes, and duplex-call order.

**Phase-recovery sub-claim.** For each present phase  $P \in \{\text{ad}, \text{msg}, \text{agg}\}$  (the ad phase is present exactly when the associated data is nonempty; see Section 3.5), TW128 absorbs the phase’s input byte string  $s_P$  by partitioning into  $\rho$ -byte blocks (with the empty case yielding a single empty block, per Section 3.5) and feeding each block to the duplex with suffix  $\sigma_P^-$  (non-final) or  $\sigma_P^+$  (final). Lemma 3 and [BDPVA11, Lemma 4] recover the corresponding  $(\sigma, \sigma_{\text{ph}})$  sequence from the flattened input. Because the partition map is deterministic and uniquely invertible on its image (the empty case yielding the single empty block carrying  $\sigma_P^+$ ), concatenating the recovered  $\sigma$  blocks reconstructs  $s_P$  exactly. The map from  $s_P$  to the phase’s  $(\sigma, \sigma_{\text{ph}})$  sequence is therefore injective.

Applied phase by phase to a well-formed final-root transcript:

- AD. The associated data phase contributes a  $\sigma_{\text{ad}}^+$ -tagged block exactly when the associated data is nonempty; an empty associated data elides the phase and contributes no block. When the phase is present the sub-claim recovers the AD byte string. Thus distinct associated data strings  $A_i \neq A_j$  are separated either by the presence versus absence of the AD phase (when exactly one is empty) or, when both are nonempty, by their distinct AD  $\sigma$ -block sequences.
- MSG. Distinct absorbed root or leaf ciphertext byte strings produce distinct MSG  $\sigma$ -block sequences; the message language of Definition 1 is ciphertext-absorbing for encryption and decryption alike.
- AGG. The aggregation phase is present exactly when  $n \geq 1$  (Section 3.5). When it is present, the sub-claim applied to the aggregation byte string  $T_1 \parallel \dots \parallel T_n \parallel \langle n \rangle_8$  recovers that byte string. The final eight bytes give  $\langle n \rangle_8$ , and each leaf tag has fixed byte length  $\tau_{\text{leaf}}/8$ , so the byte string has a unique parse as a leaf tag sequence. Two root aggregation encodings are therefore equal exactly when their leaf tag sequences are equal. When  $n = 0$  the aggregation phase is absent and the final-root transcript ends at the recovered  $\sigma_{\text{msg}}^+$  root message call; the MSG bullet recovers the root ciphertext byte string, and the absence of any  $\sigma_{\text{agg}}^-/\sigma_{\text{agg}}^+$  call distinguishes such a prefix from every  $n \geq 1$  final-root prefix.

Therefore the recovered final-root transcript determines the root initialization, AD blocks, root ciphertext blocks, and, when  $n \geq 1$ , the aggregation byte string and leaf tag sequence.  $\square$

**Lemma 5** (Leaf-Transcript Recovery). *Let two construction-generated final-root prefixes have equal flattened inputs, and suppose that no two distinct construction-generated final leaf transcripts produce the same leaf tag. Then equality of flattened inputs recovers the same leaf transcript sequence, and therefore the same full absorbed ciphertext.*

*Proof.* By Transcript Injectivity (Lemma 4), the two equal flattened final-root inputs recover the same root initialization, AD blocks, root ciphertext blocks, and, when  $n \geq 1$ , the same aggregation byte string and leaf tag sequence; the trailing  $\langle n \rangle_8$  field fixes the common leaf count  $n$ . When  $n = 0$  no leaf is present and the recovered root ciphertext blocks already constitute the full absorbed ciphertext. When  $n \geq 1$ , the aggregation byte string parses uniquely as the leaf tag sequence  $T_1 \parallel \dots \parallel T_n$ ; under the hypothesis that distinct construction-generated final leaf transcripts receive distinct leaf tags, equality of the two leaf tag sequences forces, position by position, equality of the corresponding final leaf transcripts, hence of their absorbed leaf ciphertext bodies. Together with the common root ciphertext blocks this recovers the same full absorbed ciphertext.  $\square$

## A.4 Bridge and Typed Decoding

**Lemma 6** (Bridge and Typed Decoding). *The random oracle games used in Sections 5 and 6 and Corollary 6 are ordinary random oracle games on a single random oracle  $\text{RO}_{\text{flat}} : \{0, 1\}^* \rightarrow \{0, 1\}^\infty$  over the unpadded [BDPVA08]  $H$ -input domain (Section 4.2), with TW128 root and leaf duplex outputs answered by the typed restriction*

$$\text{RF}(X) = \text{RO}_{\text{flat}}(\text{flat}(X)), \quad \text{flat}(X) = I[r, r_{\max}](\text{raw\_flat}(X)),$$

for  $r = r_{\max} = 1344$ . Here  $\text{raw\_flat}(X)$  is the native flattened input  $\text{Flat}(\cdot)$  of Section A.2 applied to the duplex-call sequence of  $X$ , and  $I[r, r_{\max}]$  is the [BDPVA11] Theorem 3 preprocessing map. The induced map  $X \mapsto \text{flat}(X)$  is injective on well-formed typed TW128 duplex transcript prefixes. The exact decoder  $\text{KeyedTWOut}$  of Definition 2 then maps any direct-query input to either  $\perp$  or a unique candidate key and counted output-point class.

*Proof.* The bridge  $I[r, r_{\max}]$  is described by the proof of [BDPVA11, Theorem 3] in three steps: (1) pad the input with  $\text{pad10}^*1[r]$ ; (2) for each  $r$ -bit block, append  $r_{\max} - r$  zero bits; (3) strip the trailing  $\text{pad10}^*$  from the result. The  $\text{pad10}^*1$  padding in step (1) appends a 1, then  $q = (-|\text{raw\_flat}(X)| - 2) \bmod r$  zero bits, then a closing 1; the  $\text{pad10}^*$ -unpadding in step (3) strips the  $r_{\max} - r$  trailing zero bits introduced by step (2) together with the closing 1 of step (1). For TW128 with  $r = r_{\max}$ , step (2) appends an empty string and step (3) strips zero trailing zeros and the closing 1, so the net effect of steps (1) and (3) is to map a native flattened input  $U = \text{raw\_flat}(X)$  to  $U \parallel 1 \parallel 0^q$ . Any string in this image has a unique inverse:  $q$  is the number of trailing zero bits, the preceding bit is the inserted 1, and deleting this suffix recovers  $U$ .

Injectivity of  $\text{flat}$  on well-formed typed TW128 prefixes then follows from Lemma 4: distinct well-formed typed prefixes have distinct  $\text{raw\_flat}(\cdot)$  images, and the bridge is injective on that image. Direct adversary queries on inputs outside the bridged TW128 well-formed transcript language still count in  $q_{\text{RO}}(\mathcal{B})$  (Section 4.3) but cannot trigger  $\text{bad}_{\text{key}}$ .

Counted transcript lookup points are the well-formed prefixes of Definition 1 whose classes are listed in Table 9: root or leaf initialization, root AD blocks, root and leaf message blocks, root aggregation blocks, and the final root tag point. An input accepted by  $\text{KeyedTWOut}$  therefore fixes a unique candidate key—by the injectivity just shown and the fixed-width key field of Lemma 4—and a unique output-point class, by Output-Point Separation (Corollary 7); inputs outside this well-formed transcript language decode to none.  $\square$

The bridge injectivity established here underlies Opening Triple Separation (Proposition 1, Section 6): distinct  $(K, N, A)$  triples yield distinct bridged final root transcripts.

**Corollary 7** (Output-Point Separation). *Every well-formed TW128 typed transcript prefix at a counted construction transcript lookup point belongs to exactly one of the following output-point classes, identified by the recovered duplex instance and the 4-bit suffix of its final call:*

- $\mathcal{R}_{\text{init}}(\text{root duplex, suffix } \sigma_{\text{init}})$ ,
- $\mathcal{L}_{\text{init}}(j) (\text{leaf-}j \text{ duplex, suffix } \sigma_{\text{init}})$ ,
- $\mathcal{R}_{\text{ad}}^- / \mathcal{R}_{\text{ad}}^+ (\text{root duplex, suffix } \sigma_{\text{ad}}^- \text{ or } \sigma_{\text{ad}}^+)$ ,
- $\mathcal{R}_{\text{msg}}^- / \mathcal{R}_{\text{msg}}^+ (\text{root duplex, suffix } \sigma_{\text{msg}}^- \text{ or } \sigma_{\text{msg}}^+)$ ,
- $\mathcal{L}_{\text{msg}}^-(j) (\text{leaf-}j \text{ duplex, suffix } \sigma_{\text{msg}}^-)$ ,
- $\mathcal{L}_{\text{tag}}(j) (\text{leaf-}j \text{ duplex, suffix } \sigma_{\text{msg}}^+)$ ,

- $\mathcal{R}_{\text{agg}}(\text{suffix } \sigma_{\text{agg}}^-)$ ,
- $\mathcal{R}_{\text{tag}}(\text{root duplex, suffix } \sigma_{\text{agg}}^+)$ .

For any two such prefixes whose bridged flattened inputs are equal, the separation of Lemma 4 recovers the same duplex instance, leaf index  $j$  when present,  $(K, N)$  pair, output-point class, and duplex-call sequence. Equivalently, any two distinct counted transcript prefixes already differ in one of these recovered fields, and hence in their flattened inputs.

*Proof.* Lemma 6 recovers equality of native flattened inputs from equality of bridged inputs, after which Lemma 4 recovers the duplex-call sequence and its per-call  $(\sigma, \sigma_{\text{ph}})$  pairs. The first call's  $\sigma$  is  $p_{\text{root}} \parallel K \parallel N$  for root instances and  $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$  for leaf instances, with fixed-width fields, so it fixes the instance type (by  $p_{\text{root}}$  vs  $p_{\text{leaf}}$ ), the  $(K, N)$  pair, and the leaf index  $j$ ; within each instance type the phase suffixes are pairwise distinct, so the recovered instance type together with the final-call suffix fixes the output-point class, and each prefix therefore lies in exactly one class. Conversely, two distinct well-formed prefixes in the same class on the same instance differ in some recovered call's  $\sigma$  or in the sequence length, hence in their flattened inputs.  $\square$

**Definition 2** (Exact Keyed Decoder). For a direct query to  $\text{RO}_{\text{flat}}$  with input  $Y$  and finite length  $\ell$ , define  $\text{KeyedTWOOut}(Y)$  as a partial map from  $\{0, 1\}^*$  to  $\{0, 1\}^k \times \mathcal{C}_{\text{out}}$ . The map ignores  $\ell$ . It returns  $(K, \mathcal{C})$  iff there exists a typed TW128 duplex-call prefix  $X$  such that  $Y = \text{flat}(X)$ , the raw flattened prefix of  $X$  parses as a well-formed TW128 transcript prefix in the class  $\mathcal{C}$  listed in Table 9, and the first call of  $X$  contains the candidate key  $K$  in the fixed-width root field  $p_{\text{root}} \parallel K \parallel N$  or leaf field  $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$ . If no such prefix exists,  $\text{KeyedTWOOut}(Y) = \perp$ . In particular, inputs with the right final suffix but a malformed phase order, noncanonical block partition, malformed aggregation string, or out-of-range leaf index are outside the well-formed language and decode to  $\perp$ . A non- $\perp$  decoding fixes not only the candidate key  $K$  but the candidate context components present in the decoded prefix: the nonce  $N$  occupies the remaining fixed-width init-field slot, and the associated data  $A$  is read off the recovered associated data blocks. We use the key projection  $(K, \mathcal{C})$  throughout; when a game hides a single context component  $Z \in \{K, N, A\}$ , the same decoding returns that component's candidate value, which we denote  $\text{KeyedTWOOut}_Z(Y)$ .

**Table 9:** Counted keyed transcript classes for KeyedTWOOut.

| Class $\mathcal{C}$             | Instance | Final call / role                                             |
|---------------------------------|----------|---------------------------------------------------------------|
| $\mathcal{R}_{\text{init}}$     | root     | $\sigma_{\text{init}}$ , root initialization                  |
| $\mathcal{R}_{\text{ad}}^-$     | root     | $\sigma_{\text{ad}}^-$ , non-final AD block                   |
| $\mathcal{R}_{\text{ad}}^+$     | root     | $\sigma_{\text{ad}}^+$ , final AD block                       |
| $\mathcal{R}_{\text{msg}}^-$    | root     | $\sigma_{\text{msg}}^-$ , non-final root message block        |
| $\mathcal{R}_{\text{msg}}^+$    | root     | $\sigma_{\text{msg}}^+$ , final root message block            |
| $\mathcal{R}_{\text{agg}}^-$    | root     | $\sigma_{\text{agg}}^-$ , non-final aggregation block         |
| $\mathcal{R}_{\text{tag}}$      | root     | $\sigma_{\text{agg}}^+$ , final aggregation / root tag        |
| $\mathcal{L}_{\text{init}}(j)$  | leaf $j$ | $\sigma_{\text{init}}$ , leaf initialization                  |
| $\mathcal{L}_{\text{msg}}^-(j)$ | leaf $j$ | $\sigma_{\text{msg}}^-$ , non-final leaf message block        |
| $\mathcal{L}_{\text{tag}}(j)$   | leaf $j$ | $\sigma_{\text{msg}}^+$ , final leaf message block / leaf tag |

The table is a transcript-lookup table, not an observation-length table. A class is counted even when the construction's requested output length at that call is zero, including root initialization, non-final AD and aggregation calls, a final AD call whose following root chunk requests no keystream bytes, and the final root message call. A direct query triggers the same decoder regardless of  $\ell$ : short finite-output queries and zero-length queries  $(Y, 0)$  still count as direct queries to the address  $Y$ . Leaves have no AD or aggregation classes;

a leaf transcript enters the table only through leaf initialization, non-final leaf message blocks, or the final leaf tag point. When the associated data is empty the root transcript likewise has no  $\mathcal{R}_{\text{ad}}^-$  or  $\mathcal{R}_{\text{ad}}^+$  class (Section 3.5); the  $\mathcal{R}_{\text{init}}$  class then supplies the first root keystream block, exactly as  $\mathcal{L}_{\text{init}}(j)$  supplies the first leaf keystream block, and the decoder still ignores  $\ell$ . Symmetrically, when  $n = 0$  the root transcript has no  $\mathcal{R}_{\text{agg}}$  or  $\mathcal{R}_{\text{tag}}$  class (Section 3.5); the final root message call  $\mathcal{R}_{\text{msg}}^+$  then supplies the root tag, exactly as  $\mathcal{L}_{\text{tag}}(j)$  supplies a leaf tag, and the decoder again ignores  $\ell$ . An  $n \geq 1$  computation also reaches an  $\mathcal{R}_{\text{msg}}^+$  point when closing chunk 0, but there the construction requests zero output and continues into the aggregation phase, so the  $\mathcal{R}_{\text{msg}}^+$  value is exposed as a tag only on the  $n = 0$  path; since  $n = \text{leaves}(C)$  is fixed by  $\|C\|$ , the two uses never coincide on a single ciphertext.

The hidden-key encryption, verification, and mu-ind accounting all reindex transcript addresses by the key field. We record that reindexing once.

**Lemma 7** (Key-Indexed Transcript Renaming). *For a key  $K \in \{0, 1\}^k$ , let  $\mathcal{Y}_K$  be the set of bridged inputs  $\text{flat}(X)$  of well-formed TW128 transcript prefixes  $X$  whose first-call key field is  $K$ —the counted hidden-key transcript addresses of Table 9 for key  $K$ . Then:*

1. Disjointness. *For distinct keys  $K_0 \neq K_1$ , the sets  $\mathcal{Y}_{K_0}$  and  $\mathcal{Y}_{K_1}$  are disjoint.*
2. Renaming. *The map  $\Phi_{K_0 \rightarrow K_1}$  that replaces the first-call key field  $K_0$  by  $K_1$  and leaves every other recovered field unchanged—nonce, leaf index, phase sequence, AD blocks, ciphertext blocks, aggregated leaf tag bytes, and root/leaf instance—induces through flat a bijection  $\text{flat} \circ \Phi_{K_0 \rightarrow K_1} \circ \text{flat}^{-1} : \mathcal{Y}_{K_0} \rightarrow \mathcal{Y}_{K_1}$  preserving output-point classes and equality relations among addresses.*
3. Direct-query avoidance. *A direct query input  $Y \in \mathcal{Y}_K$  has  $\text{KeyedTWOut}(Y) = (K, \mathcal{C})$  for a counted class  $\mathcal{C}$ ; hence if no prior direct query decodes to  $K$  then  $\mathcal{Y}_K$  is disjoint from all prior direct-query inputs.*

*Proof.* By bridge injectivity (Lemma 6) and Transcript Injectivity (Lemma 4), a bridged input  $\text{flat}(X)$  determines the native flattened prefix and hence the first duplex call  $p_{\text{root}} \| K \| N$  or  $p_{\text{leaf}} \| K \| N \| \langle j \rangle_s$ , in particular its fixed-width key field  $K$ . (1) If  $\text{flat}(X) = \text{flat}(X')$  with key fields  $K_0$  and  $K_1$  then  $K_0 = K_1$ , so distinct keys give disjoint address sets. (2)  $\Phi_{K_0 \rightarrow K_1}$  rewrites only the key field, leaving the recovered duplex-call sequence otherwise intact, so it is a bijection on well-formed prefixes preserving the output-point class (Corollary 7) and the equality pattern among prefixes; flat transports it to the stated bijection on addresses. (3) By Lemma 6 a  $Y = \text{flat}(X) \in \mathcal{Y}_K$  decodes under KeyedTWOut to  $(K, \mathcal{C})$ ; the contrapositive gives the avoidance claim.  $\square$

## A.5 Adaptive Candidate Bounds

**Lemma 8** (Adaptive Candidate Collision and Preimage). *Consider any interaction with a lazy random oracle  $\text{RO}_{\text{flat}} : \{0, 1\}^* \rightarrow \{0, 1\}^\infty$  that builds a sequence of distinct candidate inputs  $Y_1, \dots, Y_m$ . The process may make arbitrary non-candidate oracle queries and may choose each candidate adaptively from the full prior transcript. Whenever a new candidate  $Y_j$  is appended, two premises hold: predictability—both the decision to append and the string  $Y_j$  are determined by the transcript preceding the append step—and unrevealed value—no oracle lookup before the append step has requested a positive number of bits of  $\text{RO}_{\text{flat}}(Y_j)$ ; zero-length lookups at  $Y_j$  are permitted. Fix a projection length  $\tau$ , and suppose the sequence has at most  $L$  candidates ( $m \leq L$ ).*

1. Collision. *For every  $s \geq 2$ , the probability  $\Pr[\text{coll}_{s, \tau}]$  that some  $s$  distinct candidates have equal  $\tau$ -bit projections,*

$$\lfloor \text{RO}_{\text{flat}}(Y_{i_1}) \rfloor_\tau = \dots = \lfloor \text{RO}_{\text{flat}}(Y_{i_s}) \rfloor_\tau \quad \text{for some } 1 \leq i_1 < \dots < i_s \leq m,$$

1983 satisfies  $\Pr[\text{coll}_{s,\tau}] \leq \binom{L}{s} \cdot 2^{-\tau(s-1)}$ .

1984 2. *Preimage.* For every target  $t \in \{0,1\}^\tau$  chosen independently of  $\text{RO}_{\text{flat}}$ , the probability  
 1985  $\Pr[\text{pre}_{\tau,t}]$  that some candidate's  $\tau$ -bit projection equals  $t$ , i.e.  $\lfloor \text{RO}_{\text{flat}}(Y_j) \rfloor_\tau = t$  for  
 1986 some  $1 \leq j \leq m$ , satisfies  $\Pr[\text{pre}_{\tau,t}] \leq L \cdot 2^{-\tau}$ .

1987 The preimage case has a second-preimage corollary: for a designated candidate position  $j_0$ ,  
 1988 the probability  $\Pr[\text{sec}_{\tau,j_0}]$  that some candidate  $j \neq j_0$  has  $\lfloor \text{RO}_{\text{flat}}(Y_j) \rfloor_\tau = \lfloor \text{RO}_{\text{flat}}(Y_{j_0}) \rfloor_\tau$ ,  
 1989 i.e. collides with the designated candidate on its  $\tau$ -bit projection, satisfies  $\Pr[\text{sec}_{\tau,j_0}] \leq$   
 1990  $(L-1) \cdot 2^{-\tau}$ .

1991 *Proof.* Both parts expose the interaction in chronological order and sample each candidate's  
 1992  $\text{RO}_{\text{flat}}$  value lazily at its append step, which the premises license: when  $Y_j$  is appended its  
 1993 string is fixed by the prior transcript (predictability) and no bit of  $\text{RO}_{\text{flat}}(Y_j)$  has yet been  
 1994 revealed (unrevealed value), so  $\text{RO}_{\text{flat}}(Y_j)$  may be drawn then, uniform and independent of  
 1995 everything so far.

1996 *Collision.* For a fixed position set  $I = \{i_1 < \dots < i_s\} \subseteq \{1, \dots, L\}$ , let  $E_I$  be the event  
 1997 that these candidate positions exist and their  $\tau$ -bit projections are all equal. Condition  
 1998 on any positive-probability prefix immediately before position  $i_r$  is appended, for  $r \geq 2$ ,  
 1999 and on the values sampled at the earlier positions in  $I$ . By the above,  $\text{RO}_{\text{flat}}(Y_{i_r})$  is  
 2000 then uniform and independent of the conditioning, so its  $\tau$ -bit projection equals the  
 2001 already fixed projection of  $Y_{i_1}$  with probability  $2^{-\tau}$ . Multiplying over  $r = 2, \dots, s$  gives  
 2002  $\Pr[E_I] \leq 2^{-\tau(s-1)}$ ; if some position in  $I$  is never appended then  $E_I$  is false and the bound  
 2003 still holds. A union bound over the  $\binom{L}{s}$  position sets gives the claim.

2004 *Preimage.* For a fixed position  $j \in \{1, \dots, L\}$ , let  $E_j$  be the event that position  $j$   
 2005 exists and  $\lfloor \text{RO}_{\text{flat}}(Y_j) \rfloor_\tau = t$ . Condition on any positive-probability prefix immediately  
 2006 before  $Y_j$  is appended; the target  $t$  is a constant. By the above,  $\text{RO}_{\text{flat}}(Y_j)$  is uniform and  
 2007 independent of the conditioning, so its  $\tau$ -bit projection equals  $t$  with probability  $2^{-\tau}$ . If  
 2008 position  $j$  is never appended,  $E_j$  is false. A union bound over the  $L$  positions gives the  
 2009 claim.

2010 *Second-preimage corollary.* Expose  $\text{RO}_{\text{flat}}(Y_{j_0})$  at the designated candidate's append  
 2011 step, which predictability and unrevealed value license; this fixes  $t = \lfloor \text{RO}_{\text{flat}}(Y_{j_0}) \rfloor_\tau$  as a  
 2012 target independent of the remaining candidates. Applying the preimage part to the  $L-1$   
 2013 candidates  $j \neq j_0$ , each still unrevealed when appended, gives  $\Pr[\text{sec}_{\tau,j_0}] \leq (L-1) \cdot 2^{-\tau}$ ;  
 2014 the designated candidate is excluded, so its trivially-equal projection does not enter the  
 2015 count.  $\square$

## 2016 A.6 Adaptive Key Tests

2017 **Lemma 9** (Adaptive Key-Test Bound). Consider a TW128 random oracle model game with  
 2018 a single hidden context component  $Z$  sampled uniformly from  $\{0,1\}^w$  for some  $w \geq k$ —the  
 2019 standard case being the hidden key,  $Z = K$  and  $w = k$ , used in the AE-style proofs—direct  
 2020 adversary access to  $\text{RO}_{\text{flat}}$ , and an immediate abort event  $\text{bad}_{\text{key}}$  that fires on a direct  
 2021 query  $(Y, \ell)$  exactly when  $Y$  decodes to some counted output-point class  $\mathcal{C}$  of Table 9 with  
 2022  $\text{KeyedTWOut}_Z(Y) = Z$ . Suppose that, before the abort, the following invariant holds for  
 2023 every direct query position  $i$ : after conditioning on the full visible execution prefix and on  
 2024 the set  $S_i$  of distinct decoded candidate  $Z$ -values from earlier direct queries, the hidden  $Z$   
 2025 is uniform on  $\{0,1\}^w \setminus S_i$ . Then

$$2026 \Pr[\text{bad}_{\text{key}}] \leq q_{\text{RO}}/2^w \leq \text{KeyGuess}_k(q_{\text{RO}}) = q_{\text{RO}}/2^k,$$

2027 where  $q_{\text{RO}}$  is the execution-uniform count of direct  $\text{RO}_{\text{flat}}$  queries.

2028 *Proof.* If  $q_{\text{RO}} \geq 2^w$  the right-hand side  $q_{\text{RO}}/2^w$  is at least one, so the claim is trivial.  
 2029 Assume  $q_{\text{RO}} < 2^w$ .

Order the direct  $\text{RO}_{\text{flat}}$  queries. By the definition of  $\text{KeyedTWOut}_Z$  and Lemma 6, each query input either decodes to  $\perp$  or decodes to a unique candidate  $Z$ -value and counted output-point class. Queries that decode to  $\perp$ , and repeated queries whose decoded  $Z$ -value already lies in the current set  $S_i$ , do not change the set of possible first successful tests. Removing them can only shorten the sequence, so it suffices to analyze the subsequence of distinct fresh decoded candidate  $Z$ -values  $Z'_1, \dots, Z'_m$ , where  $m \leq q_{\text{RO}}$ .

Let  $F_i$  be the event that the first  $i$  fresh decoded candidate values all miss the hidden  $Z$ . The invariant says that, conditioned on the visible prefix before the  $(i+1)$ -st fresh test and on  $F_i$ , the hidden  $Z$  is uniform over the  $2^w - i$  values not yet tested. The next fresh candidate value  $Z'_{i+1}$  is one of those remaining values and is determined by the adversary's next direct query, so

$$\Pr[Z = Z'_{i+1} \mid F_i] = \frac{1}{2^w - i}, \quad \Pr[F_{i+1} \mid F_i] = 1 - \frac{1}{2^w - i}.$$

Multiplying the miss probabilities gives

$$\Pr[F_m] = \prod_{i=0}^{m-1} \left(1 - \frac{1}{2^w - i}\right) = \prod_{i=0}^{m-1} \frac{2^w - i - 1}{2^w - i} = \frac{2^w - m}{2^w}.$$

Thus the probability that some fresh decoded candidate value equals the hidden  $Z$  is  $m/2^w \leq q_{\text{RO}}/2^w \leq q_{\text{RO}}/2^k$ , the last step by  $w \geq k$ . This event is exactly the first  $\text{bad}_{\text{key}}$  abort, because  $\text{KeyedTWOut}_Z$  returns either  $\perp$  or a unique candidate  $Z$ -value.  $\square$

## A.7 Leaf Tag Collision Bound

**Lemma 10** (Leaf Tag Collision Bound). *Consider any TW128 random oracle model game with an adversary  $\mathcal{B}$  that has direct access to  $\text{RO}_{\text{flat}}$  and construction computations that may generate final leaf transcripts—by encryption or by verification computations, whether they accept or reject. Let  $\text{bad}_{\text{leaf}}$  be the event that two distinct construction-generated final leaf transcript prefixes  $X \neq X'$  in leaf tag output-point classes receive the same  $\tau_{\text{leaf}}$ -bit projection from  $\text{RF}(\cdot)$ , and let  $n_{\text{leaf}}(\mathcal{B})$  be the execution-uniform count of distinct construction-generated final leaf transcripts (Section 4.3). Both bounds below apply the collision case ( $s = 2$ ,  $\tau = \tau_{\text{leaf}}$ ) of Lemma 8 to a chronological leaf tag candidate sequence, in two regimes.*

1. Adversary-supplied keys. When the game's keys are adversary-supplied—so direct queries may themselves land on leaf tag output points—

$$\Pr[\text{bad}_{\text{leaf}}] \leq \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})) = \binom{q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})}{2} / 2^{\tau_{\text{leaf}}}.$$

2. Hidden key. When the game has a hidden key, let  $\text{hit}_{\text{leaf}}$  be the event that a direct adversary query hits the bridged input of a construction-generated hidden-key final leaf transcript before that transcript is first created. Then

$$\Pr[\text{bad}_{\text{leaf}} \text{ before } \text{hit}_{\text{leaf}}] \leq \binom{n_{\text{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\text{leaf}}} = \frac{n_{\text{leaf}}(\mathcal{B})(n_{\text{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\text{leaf}} + 1}}.$$

*Proof.* Build a chronological leaf tag candidate sequence: a candidate is the bridged input of a leaf tag output point ( $\mathcal{L}_{\text{tag}}(j)$ ), appended the first time it is reached—by a direct query in  $\mathcal{B}$ 's oracle phase or by a construction computation generating a final leaf transcript—and not appended again. By Lemma 6, Corollary 7, and Lemma 4, two reaches of the same bridged input come from the same final leaf transcript, so repeated evaluations add no

candidate—the lazy  $\text{RO}_{\text{flat}}$  value is reused—and distinct final leaf transcripts give distinct candidates. No candidate’s  $\text{RO}_{\text{flat}}$  value is read before its append step, so the sequence meets the predictability and unrevealed-value premises of Lemma 8, whose collision case with  $s = 2$  and  $\tau = \tau_{\text{leaf}}$  bounds the probability that two candidates share a  $\tau_{\text{leaf}}$ -bit projection by  $\binom{L}{2} \cdot 2^{-\tau_{\text{leaf}}}$  for a length- $L$  sequence. Every  $\text{bad}_{\text{leaf}}$  pair is a pair of construction-generated candidates.

(1) *Adversary-supplied keys.* Direct queries may decode to a leaf tag class  $\mathcal{L}_{\text{tag}}(j)$  and are appended as candidates; together with the  $n_{\text{leaf}}(\mathcal{B})$  construction-generated transcripts the sequence has length at most  $q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})$ , so the case with  $L = q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})$  gives  $\Pr[\text{bad}_{\text{leaf}}] \leq \binom{q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\text{leaf}}} = \text{LeafColl}_{\tau_{\text{leaf}}}(q_{\text{RO}}(\mathcal{B}) + n_{\text{leaf}}(\mathcal{B}))$ .

(2) *Hidden key.* A direct query may still decode under  $\text{KeyedTWOut}$  to a  $\mathcal{L}_{\text{tag}}(j)$  class for an adversary-chosen key other than the hidden key, and is appended as a candidate. But every  $\text{bad}_{\text{leaf}}$  pair is a pair of construction-generated transcripts, and under a hidden key these are all generated with that key, so it suffices to count collisions within the subsequence of construction-generated hidden-key final leaf transcripts, of length at most  $n_{\text{leaf}}(\mathcal{B})$ . Before  $\text{hit}_{\text{leaf}}$  no direct query has hit the bridged input of any such transcript: by Lemma 6 such a query would decode under  $\text{KeyedTWOut}$  to the hidden key and its  $\mathcal{L}_{\text{tag}}(j)$  class, which is exactly  $\text{hit}_{\text{leaf}}$ . Each leaf tag in the subsequence is therefore fresh when its transcript is first created, so the case applied to this length- $n_{\text{leaf}}(\mathcal{B})$  subsequence gives  $\Pr[\text{bad}_{\text{leaf}} \text{ before } \text{hit}_{\text{leaf}}] \leq \binom{n_{\text{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\text{leaf}}}$ . By Leaf-Transcript Recovery (Lemma 5), conditioned on no leaf tag collision, equal flattened final-root inputs identify the same leaf transcript sequence.  $\square$

## A.8 Transcript Freshness

**Lemma 11** (Transcript Freshness). *For any fixed nonce-respecting encryption query in the TW128 random oracle experiment, let  $\text{hit}_{\text{fresh}}$  be the event that, before this query reaches some counted transcript point, a prior direct adversary query to  $\text{RO}_{\text{flat}}$  has already hit the bridged input of that hidden-key point. On  $\neg \text{hit}_{\text{fresh}}$ , every transcript output of the query that contributes to a returned ciphertext block, to an internal leaf tag, or to the final root tag is fresh at the moment its  $\text{RO}_{\text{flat}}$  value is sampled.*

*Proof.* Work on an execution outside  $\text{hit}_{\text{fresh}}$  for the fixed query. This rules out prior adversary reads of the hidden-key construction inputs considered below. It remains to rule out repeats from previous construction interface outputs and from the current encryption query.

**First root keystream block.** The first root keystream block, when present, is produced by the root duplex after the initialization call  $p_{\text{root}} \parallel K \parallel N$  and any AD absorption (elided when  $A = \varepsilon$ , in which case the  $\sigma_{\text{init}}$  call itself emits the first root keystream block). Since the nonce-respecting adversary never repeats a nonce, no previous encryption query can have used the same root initialization transcript. Within the current query, Lemma 4 separates initialization, AD, message, and aggregation calls by their suffixes and by the duplex-call encoding, so the first root keystream transcript is fresh. If the requested keystream length is zero, no ciphertext block is returned at that point and there is nothing to prove for that output.

**First leaf keystream block.** For each leaf chunk  $j \geq 1$ , the first leaf keystream block is produced after the initialization call  $p_{\text{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$ . The prefix separates leaves from the root, the nonce separates this encryption query from all previous encryption queries, and  $\langle j \rangle_8$  separates leaves within this query. Hence every first leaf keystream transcript is fresh.

**Later message outputs.** Argue inductively within each root or leaf message stream. Suppose the current message-output transcript is fresh. If its output is used as a keystream block, its  $\text{RO}_{\text{flat}}$  value  $Z_i$  is, by freshness of the lazy sample, uniform conditioned on the full prior view; since the adaptive plaintext block  $M_i$  is a deterministic function of that prior view,

$$C_i = M_i \oplus Z_i$$

is uniform conditioned on the prior view. TW128 then absorbs  $C_i$  with the appropriate message phase suffix ( $\sigma_{\text{msg}}^-$  or  $\sigma_{\text{msg}}^+$ ) before requesting the next keystream block. Once  $C_i$  is sampled, the next transcript is fixed; Lemma 4's injectivity and domain separation imply it has not appeared earlier unless the same duplex-call prefix has appeared earlier. Nonce uniqueness excludes this across encryption queries. By Corollary 7, the next message-keystream output point can coincide with a within-query earlier output point only if both belong to the same output-point class on the same duplex instance (root or leaf- $j$  with matching  $j$ ) and have the same duplex-call sequence; the current message-stream prefix is strictly longer than every earlier prefix in that stream, so this is excluded. Across distinct instances within the same query (root vs leaf, or two leaves with distinct  $j$ ), the prefixes already differ in the first call's  $\sigma$  by  $p_{\text{root}}/p_{\text{leaf}}$  or  $\langle j \rangle_8$  respectively. Each later message-output transcript that emits keystream is therefore fresh when sampled. The terminal message-output transcript of a leaf emits the leaf tag rather than a following keystream block; the same separation argument applies to that transcript, so each internal leaf tag is sampled at a fresh point. Empty inputs still induce a duplex call, but with zero keystream output they contribute no ciphertext block.

**Final root tag transcript.** The final root tag is sampled at the root tag output point selected by  $\|C\|$ : the  $\mathcal{R}_{\text{tag}}$  point after the root duplex absorbs the aggregation transcript built from the leaf tags and  $\langle n \rangle_8$  when  $n \geq 1$ , and the  $\mathcal{R}_{\text{msg}}^+$  point closing the root message phase when  $n = 0$  (Section 3.5). When  $n \geq 1$ , the leaf tags are themselves  $\text{RO}_{\text{flat}}$  outputs on final leaf transcripts; condition on their sampled values, on the sampled ciphertext, and on the adversary-visible prior view and  $\text{RO}_{\text{flat}}$  table state, so that the root aggregation transcript is fixed. When  $n = 0$  there is no aggregation transcript; condition on the sampled ciphertext and the adversary-visible prior view alone. By Corollary 7, the selected root tag output-point class is separated from every other output-point class in the same query — including root initialization (suffix  $\sigma_{\text{init}}$ ), AD-phase outputs, the remaining message phase outputs, leaf-side output points (separated by  $p_{\text{leaf}}$ ), and, in the  $n \geq 1$  case, earlier  $\mathcal{R}_{\text{agg}}$ -prefix output points (separated by the  $\sigma_{\text{agg}}^-$  vs  $\sigma_{\text{agg}}^+$  suffix). Even if two encryption queries realize the same aggregation transcript (or, for  $n = 0$ , the same root ciphertext), the full root transcript also contains the root initialization  $p_{\text{root}} \| K \| N$ , and nonce uniqueness separates that initialization from all previous encryption queries. The final root tag transcript is therefore fresh, and its  $\tau_{\text{root}}$ -bit  $\text{RO}_{\text{flat}}$  output is uniform independently of the conditioned information.  $\square$

## A.9 Mu-Ind Verification Coupling

The direct mu-ind adjacent-hybrid proof uses the generic key-test and leaf-collision accounting lemmas together with the verification-specific facts below. The *root tag output point* of a computation is the output point that emits its root tag:  $\mathcal{R}_{\text{tag}}$  (suffix  $\sigma_{\text{agg}}^+$ ) when  $n \geq 1$ , and the closing root message point  $\mathcal{R}_{\text{msg}}^+$  (suffix  $\sigma_{\text{msg}}^+$ ) when  $n = 0$  and the aggregation phase is elided (Section 3.5). Because  $n = \text{leaves}(C)$  is fixed by  $\|C\|$ , all computations on a common ciphertext share the same root tag output point. A *final root transcript class* is the set of construction computations whose encryption or verification computation realizes the same bridged final root input at this root tag output point. Such

a class is *encryption-first* if its first construction generation is by an encryption query, and *verification-first* if its first construction generation is by a non-replay verification query.

**Lemma 12** (Verification Visible-Prefix Key Uniformity). *In the stopped single-key TW128 random oracle encryption/verification interaction with hidden key  $K$ , direct access to  $\text{RO}_{\text{flat}}$ , and immediate abort on  $\text{bad}_{\text{key}}$ , order the direct  $\text{RO}_{\text{flat}}$  queries. For a direct query position  $i$ , let  $S_i$  be the set of distinct candidate keys decoded by  $\text{KeyedTWOut}$  from earlier direct-query inputs. Condition on any positive-probability visible execution prefix immediately before query  $i$ , on no earlier  $\text{bad}_{\text{key}}$ , and on  $S_i$ . Then the hidden key  $K$  is uniform on  $\{0, 1\}^k \setminus S_i$ . Consequently the uniformity invariant required by Lemma 9 holds in the stopped interaction.*

*Proof.* The visible prefix contains the adversary’s oracle queries and replies: direct  $\text{RO}_{\text{flat}}$  outputs, encryption outputs and the resulting replay table, verification bits, and the adversary state determined by these values. Fix such a prefix and two keys  $K_0, K_1 \notin S_i$ .

Let  $\Phi_{0 \rightarrow 1} = \Phi_{K_0 \rightarrow K_1}$  be the key-renaming map of Lemma 7 and  $\Phi_{1 \rightarrow 0}$  its inverse. By that lemma,  $\text{flat} \circ \Phi_{0 \rightarrow 1} \circ \text{flat}^{-1}$  is a bijection  $\mathcal{Y}_{K_0} \rightarrow \mathcal{Y}_{K_1}$  between the bridged hidden-key construction addresses for  $K_0$  and for  $K_1$ , preserving output-point classes and equality relations among addresses.

This bijection is applied online, not to a pre-existing fixed address set. Whenever a construction computation under  $K_0$  would first query a hidden-key address  $Y$ , the coupled computation under  $K_1$  queries  $\Phi_{0 \rightarrow 1}(Y)$  and receives the same lazy-table value; repeated queries are coupled consistently through the same map. If the queried address is a root aggregation address containing leaf tag bytes, those bytes are already coupled equal because the corresponding leaf tag lookups were either repeated or first sampled earlier under the same renaming rule. Thus adaptively generated message transcripts and root aggregation transcripts are renamed step by step with the same visible ciphertext, leaf tag, and root tag bytes.

Prior direct-query inputs are not renamed: they are adversary-chosen visible strings. Since  $K_0, K_1 \notin S_i$ , neither  $\mathcal{Y}_{K_0}$  nor  $\mathcal{Y}_{K_1}$  contains a prior direct-query input (Lemma 7, direct-query avoidance). Direct-query lazy-table values are therefore coupled identically and remain disjoint from the renamed hidden construction addresses.

Use the same adversary coins, the same direct-query lazy-table values, and the online renamed construction-internal lazy-table values. Encryption outputs have the same distribution under the coupling, because every construction lookup that determines ciphertext, leaf tags, or the final root tag is paired with a distinct renamed lookup carrying the same sampled value. Replay verification decisions depend only on the visible replay table. Non-replay verification computations are renamed in the same online way as encryption computations, and the final root tag comparison therefore gives the same accept/reject bit. Verification does not reveal plaintext on rejection or acceptance; it reveals only that bit.

Thus every visible prefix with hidden key  $K_0 \notin S_i$  has the same conditional probability as the corresponding execution with hidden key  $K_1 \notin S_i$ . Since  $K$  is initially uniform and the condition “no earlier  $\text{bad}_{\text{key}}$ ” removes exactly the keys in  $S_i$ , all keys in  $\{0, 1\}^k \setminus S_i$  remain equally likely.  $\square$

**Lemma 13** (Verification-First Root Guessing). *In the stopped single-key TW128 random oracle encryption/verification interaction before  $\text{bad}_{\text{key}} \cup \text{bad}_{\text{leaf}}$ , let  $\text{win}_{\text{vf}}$  be the event that some accepting non-replay verification submission belongs to a verification-first final root transcript class. Then*

$$\Pr[\text{win}_{\text{vf}}] \leq q_v(\mathcal{B})/2^{\tau_{\text{root}}}.$$

*Proof.* Throughout, a *non-replay verification submission* means a valid non-replay verification query; by the validity convention, an invalid verification query returns 0 and performs no construction computation, so it realizes no bridged final root input, belongs to no final root transcript class, and contributes to no  $G_{\mathcal{C}}$ .

Work in the game stopped at the first occurrence of  $\text{bad}_{\text{key}}$  or  $\text{bad}_{\text{leaf}}$ . For accounting, define an all-reject continuation: encryption queries are answered normally, replay verification queries accept normally, and every non-replay verification submission performs the same construction computation but returns reject to the adversary. Lazy sampling is deferred for each verification-first class's final root tag value: the continuation records the class address and the submitted tag, but while the class remains verification-first it does not expose that root tag value to the adversary. The execution-uniform count  $q_v(\mathcal{B})$  applies to this continuation.

Fix a verification-first final root transcript class  $\mathcal{C}$  and follow the all-reject path until the first later encryption query generating the same class, if any. Let  $G_{\mathcal{C}}$  be the set of distinct candidate tags submitted by non-replay verification queries in class  $\mathcal{C}$  before that cutoff. Repeated tags only shrink this set. The class's correct root tag  $U_{\mathcal{C}}$  is the  $\tau_{\text{root}}$ -bit projection of  $\text{RO}_{\text{flat}}$  at the class's bridged root tag input. Before the stop, no direct query has hit this hidden-key input: such a query would be decoded by  $\text{KeyedTWOut}$  to the hidden key and would trigger  $\text{bad}_{\text{key}}$ . Before the cutoff, no encryption query has revealed  $U_{\mathcal{C}}$  as an encryption tag. (When the class's root tag point is  $\mathcal{R}_{\text{msg}}^+$ , i.e.  $n = 0$ , an encryption query on a longer message that shares chunk 0 may touch this same address while closing its chunk-0 message phase, but it requests zero output there and continues into its aggregation phase, so it does not reveal  $U_{\mathcal{C}}$ ; its own tag is emitted at a later  $\mathcal{R}_{\text{tag}}$  point.) The all-reject path and the set  $G_{\mathcal{C}}$  are therefore functions only of the adversary's coins, visible oracle replies, and lazy-table values at inputs other than this class's root tag address. By lazy sampling, conditioned on those values and on the fixed path,  $U_{\mathcal{C}}$  may be sampled after  $G_{\mathcal{C}}$  is fixed and is uniform independently of  $G_{\mathcal{C}}$ .

After the cutoff, if it exists, class  $\mathcal{C}$  contributes no accepting non-replay submission. Let the cutoff encryption query return  $(N_e, A_e, C_e, T_e)$ . Any later verification submission in the same final root transcript class has, by the final-root determinacy of Lemma 4 and Leaf-Transcript Recovery (Lemma 5) under the absence of  $\text{bad}_{\text{leaf}}$ , the same  $(N, A, C)$  as that encryption computation. If the submitted tag is  $T_e$ , the tuple is a recorded encryption tuple and the verification is a replay; if the submitted tag is not  $T_e$ , the final tag comparison fails.

Consider the event that the first accepting non-replay verification submission in the stopped game belongs to class  $\mathcal{C}$ . Up to that global first accept, the adversary's visible answers match the all-reject continuation: every earlier non-replay submission rejects, replay queries accept in both executions, and encryption answers are unchanged. The construction computations also agree at every address except that the all-reject continuation defers the final root tag value for class  $\mathcal{C}$  until the comparison is needed. Therefore the first accepting non-replay submission can belong to class  $\mathcal{C}$  only before the cutoff, and only if  $U_{\mathcal{C}} \in G_{\mathcal{C}}$ , which has probability at most  $|G_{\mathcal{C}}|/2^{\tau_{\text{root}}}$ .

Each non-replay verification submission deterministically reconstructs one final root transcript at the root tag output point. If the class is verification-first and the submission occurs before that class's encryption cutoff, the submitted tag contributes to at most one set  $G_{\mathcal{C}}$ ; otherwise it contributes to none. Thus  $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \leq q_v(\mathcal{B})$ , and a union bound over the verification-first classes gives the claim.  $\square$

## A.10 Mu-Ind Adjacent-Hybrid Lemma

The multi-user indistinguishability theorem (Theorem 2) telescopes a hybrid over the  $D$  users, replacing one real user by its ideal ( $\text{Rand}, \text{Replay}$ ) interface at each step. The following lemma bounds one such step directly inside the shared random oracle environment. It couples the real and ideal user- $d$  interfaces until one of three local events occurs: a direct hidden-key transcript hit, a hidden leaf tag collision, or an accepting non-replay verification query.

**Lemma 14** (Mu-Ind Adjacent-Hybrid Bound). *Let  $\mathcal{B}$  be a flat-RO mu-ind adversary in the pre-sampled-key formulation of Section 4.4, with keys  $K_1, \dots, K_D$  sampled at the start of the experiment. Fix  $d \in \{1, \dots, D\}$ . Let  $H_{d-1}^*$  and  $H_d^*$  be the adjacent stopped hybrids that output a fixed bit if any two sampled keys collide, and otherwise run  $\mathcal{B}$  with one shared lazy  $\text{RO}_{\text{flat}}$  table as follows: users  $e < d$  are answered by  $(\text{Rand}, \text{Replay})$ , users  $e > d$  are answered by  $(\text{Enc}_{K_e}^{\text{RO}}, \text{Ver}_{K_e}^{\text{RO}})$ , and user  $d$  is answered by  $(\text{Enc}_{K_d}^{\text{RO}}, \text{Ver}_{K_d}^{\text{RO}})$  in  $H_{d-1}^*$  and by  $(\text{Rand}, \text{Replay})$  in  $H_d^*$ . If  $n_{\text{leaf}}^{(d)}$  and  $q_v^{(d)}$  are the user- $d$  resource caps, then*

$$|\Pr[\mathcal{B}^{H_{d-1}^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^*} \Rightarrow 1]| \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)}{2^{\tau_{\text{leaf}} + 1}} + \frac{q_v^{(d)}}{2^{\tau_{\text{root}}}}.$$

*Proof.* The fixed-output branch on a key collision is identical in  $H_{d-1}^*$  and  $H_d^*$ , so only the branch with pairwise distinct keys contributes. On that branch, the two hybrids differ only in user  $d$ : users  $e < d$  are ideal in both hybrids, users  $e > d$  are real in both hybrids, and user  $d$  is answered by  $(\text{Enc}_{K_d}^{\text{RO}}, \text{Ver}_{K_d}^{\text{RO}})$  in  $H_{d-1}^*$  and by  $(\text{Rand}, \text{Replay})$  in  $H_d^*$ .

Let  $\mathcal{E}_d$  denote the common surrounding environment. It runs  $\mathcal{B}$ , answers users  $e < d$  by their  $(\text{Rand}, \text{Replay})$  interfaces, answers users  $e > d$  by  $\text{Enc}_{K_e}^{\text{RO}}, \text{Ver}_{K_e}^{\text{RO}}$ , forwards  $\mathcal{B}$ 's direct  $\text{RO}_{\text{flat}}$  queries unchanged, and shares one lazy  $\text{RO}_{\text{flat}}$  table with user  $d$ . The coupling treats user  $d$ 's encryption and verification interfaces together. Direct  $\text{RO}_{\text{flat}}$  queries that decode to  $K_d$  can affect both interfaces, so the local key-test event is charged once in the adjacent-hybrid bound.

**Coupling until divergence.** Run the two hybrids on shared adversary coins and one shared lazy  $\text{RO}_{\text{flat}}$  table. Introduce the user- $d$  local events  $\text{bad}_{\text{key}}^{(d)}$ , that a direct  $\text{RO}_{\text{flat}}$  query decodes under  $\text{KeyedTWO}_{\text{out}}$  to  $K_d$  and a counted output-point class;  $\text{bad}_{\text{leaf}}^{(d)}$ , that two distinct user- $d$  construction-generated final leaf transcripts receive the same  $\tau_{\text{leaf}}$ -bit projection; and  $\text{forge}^{(d)}$ , that a non-replay user- $d$  verification query is accepted by  $\text{Ver}_{K_d}^{\text{RO}}$ . Couple the two hybrids on shared adversary coins and one shared lazy  $\text{RO}_{\text{flat}}$  table, with user  $d$ 's  $\text{Rand}$  answers in  $H_d^*$  set equal to user  $d$ 's real encryption answers in  $H_{d-1}^*$ .

We show the two hybrids' visible transcripts agree until  $\text{bad}_{\text{key}}^{(d)} \cup \text{forge}^{(d)}$ .

*The coupled  $H_d^*$  has the true  $H_d^*$  law.* Each real user- $d$  encryption output is a projection of  $\text{RO}_{\text{flat}}$  at the distinct transcript points of its computation—distinct within a computation and, by encryption nonce-uniqueness and Transcript Injectivity (Lemma 4), across distinct encryptions—so until a direct query reads such a point the outputs are jointly uniform of the prescribed lengths; this is the freshness property of Lemma 11. The present environment adds a real user- $d$  verification oracle, absent from a pure encryption-replacement game. Because only encryption queries are nonce-unique, a non-replay verification may reuse an encryption nonce and lazily read a keystream or root tag point the same-nonce encryption later reaches; but lazy sampling draws each point uniformly when first read, so its value is unchanged and stays marginally uniform. In  $H_d^*$  the user- $d$  verification oracle is  $\text{Replay}$ , which answers from the recorded ciphertexts without reading  $\text{RO}_{\text{flat}}$ . Hence the coupled  $H_d^*$  transcript has the law of the true  $H_d^*$  until  $\text{bad}_{\text{key}}^{(d)}$ : its encryption answers are the marginally uniform strings of  $\text{Rand}$  and its verification answers are the replay decisions, with no further  $\text{RO}_{\text{flat}}$  dependence. (Conditioning on a particular  $H_{d-1}^*$  verification outcome can constrain a coupled tag, but that is a property of the joint coupling; the coupled  $H_d^*$  marginal is what must match the true  $H_d^*$ , and it does.)

*Agreement until  $\text{bad}_{\text{key}}^{(d)} \cup \text{forge}^{(d)}$ .* Encryption answers are equal by construction. A verification query whose tuple is recorded returns accept in both hybrids through the shared replay-table membership check, which precedes the real-versus-ideal decryption branch (Section 4.4), so real decryption is never run on a recorded tuple. A non-replay query returns reject in  $H_d^*$  and the bit  $[\text{Ver}_{K_d}^{\text{RO}} \text{ accepts}]$  in  $H_{d-1}^*$ ; the two agree unless that bit is accept, the event  $\text{forge}^{(d)}$ . A real verification reads hidden-key points but exposes

only this bit, which is independent of the keystream values it reads—decryption absorbs the submitted ciphertext, not the recovered plaintext (Lemma 3)—and depends on an encryption output only through the root tag it recomputes. In the reused-nonce case above this correlates the  $H_{d-1}^*$  bit with that encryption’s tag, but the  $H_d^*$  verification is **Replay** and the coupled encryption answers coincide, so the two coordinates still agree on such a query unless it accepts. Direct-query answers are read from the shared table and agree until a direct query reads a user- $d$  hidden-key point ( $\text{bad}_{\text{key}}^{(d)}$ ). Therefore the visible transcripts agree until  $\text{bad}_{\text{key}}^{(d)} \cup \text{forge}^{(d)}$ , and a union bound ordered by first occurrence (inserting the leaf abort to split  $\Pr[\text{forge}^{(d)}$  before  $\text{bad}_{\text{key}}^{(d)}$ ]) gives

$$\begin{aligned} & |\Pr[\mathcal{B}^{H_{d-1}^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^*} \Rightarrow 1]| \\ & \leq \Pr[\text{bad}_{\text{key}}^{(d)}] + \Pr[\text{bad}_{\text{leaf}}^{(d)} \text{ before } \text{bad}_{\text{key}}^{(d)}] \\ & \quad + \Pr[\text{forge}^{(d)} \text{ before } \text{bad}_{\text{key}}^{(d)} \cup \text{bad}_{\text{leaf}}^{(d)}]. \end{aligned}$$

**The environment does not enlarge the user- $d$  events.** The real users  $e > d$  perform construction-internal  $\text{RO}_{\text{flat}}$  lookups while simulating  $\text{Enc}_{K_e}^{\text{RO}}$  and  $\text{Ver}_{K_e}^{\text{RO}}$ . Whenever such a lookup lies in a counted keyed transcript class, Lemma 6 decodes its candidate key as  $K_e$ , which on the non-collision branch differs from  $K_d$ ; a lookup outside the counted keyed language is irrelevant to  $\text{bad}_{\text{key}}^{(d)}$ . Foreign-user lookups therefore neither trigger user  $d$ ’s key-test event nor pre-read a user- $d$  hidden transcript point, and they create no user- $d$  leaf tag collision or user- $d$  non-replay verification submission, which are counted by  $n_{\text{leaf}}^{(d)}$  and  $q_v^{(d)}$ . The three accounting events are thus local to user  $d$ ’s keyed transcript language.

**Bounding the three terms.** For  $\Pr[\text{bad}_{\text{key}}^{(d)}]$ , adapt the single-key visible-prefix renaming coupling of Lemma 12 to the environment  $\mathcal{E}_d$ . Its conclusion does not transfer verbatim: the foreign users  $e > d$  share the lazy  $\text{RO}_{\text{flat}}$  table, so renaming  $K_d$  onto a foreign key would alter their visible transcripts, and the renaming orbit must exclude the foreign keys rather than range over all of  $\{0,1\}^k \setminus S_i$ . What the coupling still delivers in  $\mathcal{E}_d$  is the marginal per-candidate bound, which is all the key test needs. The key-test contribution is taken in the original pre-sampled-key experiment, not in one renormalized to the non-collision branch: fix the foreign keys and run the visible-prefix coupling with foreign-key transcript addresses left unchanged; before the first user- $d$  key hit, the visible prefix is invariant under renaming  $K_d$  among keys not already tested and not equal to a fixed foreign key. For each fresh candidate key tested by one of  $\mathcal{B}$ ’s direct  $\text{RO}_{\text{flat}}$  queries, the contributing event is contained in  $\{K_d \text{ equals that candidate}\}$ , costing at most  $2^{-k}$ , and a foreign candidate contributes nothing; with at most  $q_{\text{RO}}(\mathcal{B})$  such tests, the union bound of Lemma 9 packages these into  $\Pr[\text{bad}_{\text{key}}^{(d)}] \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B}))$ . For the leaf term, before  $\text{bad}_{\text{key}}^{(d)}$  no direct query has pre-read the bridged input of a user- $d$  hidden-key final leaf transcript—such a query would decode under **KeyedTWO** to  $K_d$  and its  $\mathcal{L}_{\text{tag}}(j)$  class and would itself be  $\text{bad}_{\text{key}}^{(d)}$ —so the hidden-key instance of Lemma 10 gives  $\Pr[\text{bad}_{\text{leaf}}^{(d)} \text{ before } \text{bad}_{\text{key}}^{(d)}] \leq n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)/2^{\tau_{\text{leaf}}+1}$ . For the forgery term, before  $\text{bad}_{\text{key}}^{(d)} \cup \text{bad}_{\text{leaf}}^{(d)}$  every encryption-first final root transcript class admits no accepting non-replay submission—by the final-root determinacy of Lemma 4 and Leaf-Transcript Recovery (Lemma 5), a non-replay submission in such a class reaches the already-returned root tag and fails the tag comparison—so every accepting non-replay submission is verification-first, and Lemma 13 gives  $\Pr[\text{forge}^{(d)} \text{ before } \text{bad}_{\text{key}}^{(d)} \cup \text{bad}_{\text{leaf}}^{(d)}] \leq q_v^{(d)}/2^{\tau_{\text{root}}}$ .

Summing the three bounds gives

$$|\Pr[\mathcal{B}^{H_{d-1}^*} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^*} \Rightarrow 1]| \leq \text{KeyGuess}_k(q_{\text{RO}}(\mathcal{B})) + \frac{n_{\text{leaf}}^{(d)}(n_{\text{leaf}}^{(d)} - 1)}{2^{\tau_{\text{leaf}}+1}} + \frac{q_v^{(d)}}{2^{\tau_{\text{root}}}},$$

2355 with the user- $d$  key-test event  $\text{bad}_{\text{key}}^{(d)}$  charged a single time for the adjacent hybrid.  $\square$

## 2356 B The Flat Sponge Lift

2357 This appendix expands the sketch of Section 5.2 into the full argument: it supplies the  
2358 flat sponge lift machinery (Lemmas 16 and 17 and Corollary 8) invoked by the headline  
2359 theorems of Section 5.2 and by Corollary 6. The machinery applies uniformly to the games  
2360 named once here.

2361 **Definition 3** (Single-Stage TW128 Games). The *single-stage TW128 games* are the games  
2362 of Sections 4.1 and 4.4: all-in-one AE, mu-ind, CMTDs-4, the H collision, second-preimage,  
2363 and preimage games,  $\text{CDY}^*[\text{ts}]$ , and CMTs- $\ell$  for  $\ell \in \{1, 3, 4\}$ . In  $\text{CDY}^*[\text{ts}]$  the selector  
2364 encryption is construction-internal, counted in  $N$  like a challenger check.

### 2365 B.1 Simulator Convention

2366 Throughout this appendix,  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$  denotes the public primitive simulator sup-  
2367 plied by [BDPVA08, Theorem 2] — the indistinguishability theorem for the simple padded  
2368 sponge over a random permutation — after applying the [BDPVA11, Theorem 3] bridge  
2369 of Lemma 6 to TW128’s native  $\text{pad10}^*1$  transcript inputs. The superscript records that  
2370 both simulator directions share the same internal lazy  $\text{RO}_{\text{flat}}$  table. This is the simulator  
2371 referenced by the Sections 4.1 and 4.4 definitions of  $\text{Adv}_{\text{TW128}}^{\text{ae}}$  and  $\text{Adv}_{\text{TW128}}^{\text{mu-ind}}$ : the ideal  
2372 side of each AE-style experiment exposes the construction interface specified by the game  
2373  $((\text{Rand}, \text{Replay})$ , instantiated per user in mu-ind) together with  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$  as the  
2374 public primitive interface. The  $\text{RO}_{\text{flat}}$  table in this notation is internal to the ideal system  
2375 and is not an extra interface exposed to the application adversary; the committing security  
2376 advantages remain the real public permutation success probabilities of Section 4.1, with  
2377 the simulator appearing only inside the reduction to the corresponding random oracle  
2378 model adversary.

2379 **Lemma 15** (Simulator RO-Call Filter). *For the simulator  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ , every inverse*  
2380 *primitive-interface query and every forward primitive-interface query violating one of the*  
2381 *following conditions induces no simulator  $\text{RO}_{\text{flat}}$  call. The symbols  $s$ ,  $p$ ,  $\mathcal{R}$ ,  $\mathcal{O}$ , and  $\mathcal{C}$*   
2382 *are the local internal notation of the BDPV simple-padded-sponge simulator, not TW128*  
2383 *resource notation:*

2384 **(C1) New edge.** *The simulator’s internal graph has no outgoing edge from the input node*  
2385  *$s$ .*

2386 **(C2) Unsaturated.**  $\mathcal{R} \cup \mathcal{O} \neq \mathcal{C}$ .

2387 **(C3) Rooted input.** *The input node is rooted.*

2388 **(C4) Paddable path.** *The constructed path  $p$  decomposes uniquely as  $p = p' \parallel 0^r$  with*  
2389  *$p'$  not ending in  $0^r$ , and the prefix  $p'$  is a valid padded sponge input: equivalently,*  
2390  *$p' = x \parallel \text{pad10}^*[r](|x|)$  for an unpadded BDPV08 random oracle input  $x$ .*

2391 *When all four conditions hold, the forward primitive-interface query induces one simulator*  
2392  *$\text{RO}_{\text{flat}}$  call. Consequently, each primitive-interface query induces at most one simulator-side*  
2393  *$\text{RO}_{\text{flat}}$  call.*

2394 *Proof.* This is the branch structure of the [BDPVA08] simple-padded permutation simulator.  
2395 The simulator invokes the random oracle only on the forward-query branch where the input  
2396 node is rooted, unsaturated, paddable to an unpadded random oracle input  $x$ , and not  
2397 already extended by an outgoing edge in the simulator graph. Forward queries violating

any of these conditions and all inverse queries return outputs sampled or read from the simulator's internal graph without invoking the random oracle. Under the [BDPVA11, Theorem 3] bridge fixed in Section B.1, that simulator random oracle invocation is the simulator-side  $\text{RO}_{\text{flat}}$  call used throughout this appendix.  $\square$

**Lemma 16** (Flat Sponge Lift Bound). *Let  $\mathsf{G}$  be a single-stage TW128 game (Definition 3). Let  $\mathcal{A}$  be a public permutation  $\mathsf{G}$  adversary with construction interface cost at most  $N$  and at most  $N_{\text{prim}}$  direct primitive-interface queries. In the [BDPVA08] single-stage replacement, the combined construction-and-primitive transcript has length at most*

$$N_{\text{tot}} = N + N_{\text{prim}}.$$

Consequently replacing the real public permutation and TW128 construction by the simulator-world primitive interface  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$  and the corresponding flat-RO construction interface costs at most  $\text{Lift}_c(N_{\text{tot}})$ .

*Proof.* The [BDPVA08] replacement is applied before the derived-adversary construction, so its transcript contains two kinds of public primitive activity. First, each construction interface computation contributes the TW128 duplex calls it performs. By Lemmas 3 and 6, each such root or leaf output point is a typed restriction of the bridged flat sponge construction, and by the definition of  $N$  in Section 4.3 these calls are counted at the same per-block granularity as the underlying KECCAK- $p[1600, 12]$  calls. This includes verification computations whether they accept or reject, the per-user sum of construction calls in mu-ind, and the deterministic challenger decryption or encryption checks actually performed in the committing and discoverability games.

Second, each direct adversary query to  $(\pi, \pi^{-1})$  contributes one primitive-interface query and is counted in  $N_{\text{prim}}$ . These two counts are disjoint in the public permutation experiment:  $N$  counts construction-internal use of the primitive, while  $N_{\text{prim}}$  counts only the adversary's public primitive interface queries. The execution-uniform caps ensure that every adaptive transcript seen by the single-stage replacement has length at most  $N + N_{\text{prim}} = N_{\text{tot}}$ . If  $N_{\text{tot}} \geq 2^c$  then  $\text{Lift}_c(N_{\text{tot}}) \geq 1$  and the claimed bound holds trivially, so assume  $N_{\text{tot}} < 2^c$ , satisfying the hypothesis of [BDPVA08, Theorem 2]. That theorem gives distinguishing or success-probability cost at most the random-permutation differentiating bound  $f_P(N_{\text{tot}})$  of [BDPVA08, Lemma 5]. The [BDPVA08, Lemmas 4 and 5] product bounds give the random-transformation bound  $f_T(N_{\text{tot}}) = 1 - \prod_{i=1}^{N_{\text{tot}}} (1 - i/2^c)$ , and the factor of  $f_P(N_{\text{tot}})$  corresponding to the  $f_T$  factor at index  $i$  is that factor divided by a quantity  $1 - (i-1)/2^{r+c} \in (0, 1]$ , hence at least as large, so  $f_P(N_{\text{tot}}) \leq f_T(N_{\text{tot}})$ . With  $N_{\text{tot}} < 2^c$  every factor lies in  $[0, 1]$  and

$$f_P(N_{\text{tot}}) \leq f_T(N_{\text{tot}}) = 1 - \prod_{i=1}^{N_{\text{tot}}} \left(1 - \frac{i}{2^c}\right) \leq \sum_{i=1}^{N_{\text{tot}}} \frac{i}{2^c} = \frac{N_{\text{tot}}(N_{\text{tot}} + 1)}{2^{c+1}} = \text{Lift}_c(N_{\text{tot}}),$$

the second inequality by the Weierstrass product inequality  $\prod_i (1 - x_i) \geq 1 - \sum_i x_i$  for  $x_i \in [0, 1]$ .  $\square$

**Lemma 17** (Derived-Adversary Execution Contract). *Let  $\mathsf{G}$  be a single-stage TW128 game (Definition 3). Let  $\mathcal{A}$  be a simulator-world  $\mathsf{G}$  adversary whose public primitive interface is answered by  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ , with at most  $N_{\text{prim}}$  direct primitive-interface queries. Define  $\mathcal{B}_{\text{derived}}(\mathcal{A})$  to run  $\mathcal{A}$  internally, answer each primitive-interface query by running the same simulator against direct finite-output queries to its own  $\text{RO}_{\text{flat}}$  oracle, and forward construction interface queries (or, in the committing and discoverability games, the final structured output) unchanged to the corresponding random oracle experiment. Then the simulator-world execution of  $\mathcal{A}$  and the random oracle execution of  $\mathcal{B}_{\text{derived}}(\mathcal{A})$  have identical joint distributions, and*

$$q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}}.$$

2445 *In this coupling, simulator-induced  $\text{RO}_{\text{flat}}$  calls and construction-internal RF lookups use*  
 2446 *the same lazy table. Therefore, if a simulator-induced query is the bridged input  $\text{flat}(X)$  of*  
 2447 *a well-formed TW128 transcript prefix  $X$  and a forwarded construction computation later*  
 2448 *reaches the same prefix  $X$ , both interfaces read the same  $\text{RO}_{\text{flat}}(\text{flat}(X))$  value, projected to*  
 2449 *the requested output length. Moreover,  $\mathcal{B}_{\text{derived}}$  inherits the construction-side caps of  $\mathcal{A}$ :*

$$\begin{aligned} 2450 \quad q_v(\mathcal{B}_{\text{derived}}) &= q_v(\mathcal{A}) \quad (\text{AE, mu-ind}), \\ 2451 \quad n_{\text{leaf}}(\mathcal{B}_{\text{derived}}) &\leq n_{\text{leaf}}(\mathcal{A}) \quad (\text{all games above}). \end{aligned}$$

2452 *In the mu-ind game these resource inheritances hold per user, and the New-query transcript*  
 2453 *is unchanged.*

2454 *Proof.* Couple the two executions by using the same lazy  $\text{RO}_{\text{flat}}$  table for the simulator  
 2455 and for all construction-internal  $\text{RF}(\cdot)$  lookups. In the simulator-world execution,  $\mathcal{A}$  sees  
 2456 construction answers from the selected game interface and public primitive answers from  
 2457  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$ . In the derived random oracle execution,  $\mathcal{B}_{\text{derived}}$  runs the same  $\mathcal{A}$ ,  
 2458 forwards construction interface queries (or the final committing- or discoverability-game  
 2459 output) unchanged, and answers primitive-interface queries by invoking the same simulator  
 2460 code with the same lazy  $\text{RO}_{\text{flat}}$  table. The construction interface or committing challenger  
 2461 therefore receives the same forwarded inputs, the simulator receives the same primitive  
 2462 queries in the same order, and the lazy table supplies the same values. The full views  
 2463 of  $\mathcal{A}$  in the simulator-world execution and of  $\mathcal{B}_{\text{derived}}$  in the random oracle execution are  
 2464 identically distributed.

2465 The simulator's random oracle calls may be on arbitrary [BDPVA08]  $H$ -input strings,  
 2466 not only bridged TW128 transcript inputs. These calls are the direct  $\text{RO}_{\text{flat}}$  queries of  
 2467  $\mathcal{B}_{\text{derived}}$  and are counted in  $q_{\text{RO}}(\mathcal{B}_{\text{derived}})$ . Construction-internal  $\text{RF}(\cdot)$  lookups made by  
 2468  $\text{Enc}_K^{\text{RO}}$ ,  $\text{Dec}_K^{\text{RO}}$ ,  $\text{Ver}_K^{\text{RO}}$ , or a committing or discoverability challenger are forwarded to the  
 2469 same  $\text{RO}_{\text{flat}}$  table but are not counted in  $q_{\text{RO}}$ , exactly as in Section 4.3.

2470 By Lemma 15, each primitive-interface query induces at most one direct  $\text{RO}_{\text{flat}}$  query  
 2471 by  $\mathcal{B}_{\text{derived}}$ , and inverse queries or non-RO-touching forward queries induce none. Since  $\mathcal{A}$   
 2472 makes at most  $N_{\text{prim}}$  direct primitive-interface queries, this gives  $q_{\text{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\text{prim}}$ .

2473 Lemma 6's bridge places TW128 construction restrictions and simulator-induced  $\text{RO}_{\text{flat}}$   
 2474 calls in a single lazy table. If the simulator invokes  $\text{RO}_{\text{flat}}$  on  $\text{flat}(X)$  for a well-formed  
 2475 TW128 transcript prefix  $X$ , and a forwarded construction computation later reaches  $X$ ,  
 2476 the construction lookup  $\text{RF}(X) = \text{RO}_{\text{flat}}(\text{flat}(X))$  reads the same table entry and returns  
 2477 the requested projection. Since TW128 uses  $r = r_{\text{max}}$ , the [BDPVA11, Theorem 3] output  
 2478 post-processing does not alter TW128 output bits. Thus simulator-induced direct queries  
 2479 and construction-internal lookups at the same bridged input are transcript-consistent. In  
 2480 the AE-style games, a simulator call whose input is accepted by  $\text{KeyedTWOut}$  for the  
 2481 hidden key is still a direct  $\text{RO}_{\text{flat}}$  query and is covered by the same  $\text{bad}_{\text{key}}$  accounting used  
 2482 by the direct random oracle mu-ind proof. The derivation step itself does not prove a  
 2483 key-guessing statement; it only supplies the derived adversary and the query bound at  
 2484 which the random oracle bounds of Section 5.2 are instantiated.

2485 Finally, the execution-uniform caps of Section 4.3 hold along every oracle-answer trace  
 2486 of  $\mathcal{A}$ , including traces generated by the simulator on the ideal public primitive interface.  
 2487 The derived-adversary construction does not add construction interface encryption or  
 2488 verification queries and forwards  $\mathcal{A}$ 's construction interface queries unchanged along each  
 2489 trace.  $\mathcal{B}_{\text{derived}}$  therefore inherits the relevant construction resources:  $q_v(\mathcal{B}_{\text{derived}}) = q_v(\mathcal{A})$   
 2490 in the AE and mu-ind games, and  $n_{\text{leaf}}(\mathcal{B}_{\text{derived}}) \leq n_{\text{leaf}}(\mathcal{A})$  for the distinct construction-  
 2491 generated final-leaf-transcript cap. In mu-ind, construction interface forwarding preserves  
 2492 the New calls and these caps user by user.  $\square$

## B.2 Lift Application

The lift bound of Lemma 16 and the derived-adversary construction of Lemma 17 combine into a single template, stated once and then instantiated by the headline theorems of Section 5.2 and the committing-notion bounds of Corollary 6.

**Corollary 8** (Lift Application). *Let  $\mathsf{G}$  be a single-stage TW128 game (Definition 3). For every public permutation  $\mathsf{G}$ -adversary  $\mathcal{A}$  against TW128 with the resources of Section 4.3, the derived adversary  $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$  of Lemma 17 is a random oracle  $\mathsf{G}$ -adversary with  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$  and the construction-side caps of Lemma 17—in particular  $q_v(\mathcal{B}) \leq Q_v$  and  $n_{\text{leaf}}(\mathcal{B}) \leq N_{\text{leaf}}$ , where these apply—such that*

$$\text{Adv}_{\text{TW128}}^{\mathsf{G}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \text{Adv}_{\text{RO}}^{\mathsf{G}}(\mathcal{B}).$$

If moreover  $\text{Adv}_{\text{RO}}^{\mathsf{G}}(\mathcal{B}) \leq \varepsilon_{\mathsf{G}}(q_{\text{RO}}(\mathcal{B}), n_{\text{leaf}}(\mathcal{B}))$  for some  $\varepsilon_{\mathsf{G}}$  nondecreasing in each argument, then

$$\text{Adv}_{\text{TW128}}^{\mathsf{G}}(\mathcal{A}) \leq \text{Lift}_c(N_{\text{tot}}) + \varepsilon_{\mathsf{G}}(N_{\text{prim}}, N_{\text{leaf}}).$$

*Proof.* By the flat sponge lift bound (Lemma 16), replacing the real public permutation and TW128 construction by  $(\mathcal{S}^{\text{RO}_{\text{flat}}}, \mathcal{S}^{-1, \text{RO}_{\text{flat}}})$  and the flat-RO construction interface costs at most  $\text{Lift}_c(N_{\text{tot}})$ , where  $N_{\text{tot}} = N + N_{\text{prim}}$  counts both the construction calls of  $N$ —including any construction oracle exposed during  $\mathcal{A}$ ’s run and, in the win-event games, the challenger’s post-output decryption or encryption checks—and the  $N_{\text{prim}}$  direct primitive queries. The derived-adversary contract (Lemma 17) expresses the resulting simulator-world execution as the identically distributed random oracle execution of  $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ , with  $q_{\text{RO}}(\mathcal{B}) \leq N_{\text{prim}}$  and the stated construction-side inheritances, giving the first bound. The second follows by the monotonicity of  $\varepsilon_{\mathsf{G}}$ . When  $\mathsf{G}$  is parameterized by a target fixed before  $\mathcal{A}$  runs—the discoverability target  $T^*$ —the coupling holds for each fixed target and the target-independent bound holds for the maximum defining  $\text{Adv}_{\text{TW128}}^{\mathsf{G}}(\mathcal{A})$ .  $\square$

## C Vectorized Kernel Details

This appendix details the architecture-specific cores summarized in Section 7.1: the instruction selection, register layout, and remainder handling of the `amd64` and `arm64` kernels.

**amd64.** The `amd64` implementation provides AVX-512 and AVX2 cores and selects between them once at startup from the `AVX512VL` CPUID feature bit. The single-duplex permutation has two variants: a general-purpose-register core adapted from the standard library’s `KECCAK-p` permutation (using the lane-complement trick so that  $\chi$  requires no separate negation), and an AVX-512 variant; the root duplex uses whichever matches the selected core. The eight-way AVX-512 core packs the lane-major state into 25 ZMM registers, one per lane with all eight instances in the register’s eight 64-bit slots, and performs the permutation with no cross-lane shuffles: rotations use `VPROLQ`, and `VPTERNLOGQ` fuses the  $\theta$   $D$ -term formation and the  $\chi$  and-not into single three-input logic instructions. The eight-way AVX2 core, lacking 512-bit registers, runs the eight instances as two groups of four (four 64-bit lanes per YMM register), rotating by shift-or and computing  $\chi$  with `VPANDN`. The fused chunk kernels combine absorption with the permutation. The AVX-512 steady-state kernel reads the eight chunks with contiguous loads, transposes each rate block into the lane-major layout in registers (`VPUNPCKLQDQ/VPUNPCKHQDQ` and `VSHUFI64X2`), absorbs on the lane-major state, and transposes each output block back for a contiguous store, leaving only the 7-byte partial rate lane to a masked gather; these sequential accesses are prefetcher-friendly, whereas per-lane gathers and scatters at the chunk stride  $\beta$  (`VPGATHERQQ/VPSCATTERQQ`) would stall on cold memory and cap the long-message

rate. The register-resident kernel that drains a two-to-seven-chunk remainder instead uses that per-lane gather/scatter form under a mask, reading the active chunks in place: the transpose’s shuffle work is constant regardless of the chunk count and would be spent partly on inactive lanes, whereas the gathers scale with the active element count. The AVX2 path drains the same remainders with a variant of its grouped-by-four kernels that aims each inactive instance’s loads and stores at a dummy block inside the kernel frame with a zero advance stride, so inactive instances touch no memory beyond the remainder and the hot loop stays branch-free.

**arm64.** The **arm64** path uses NEON plus the Armv8.2 SHA-3 extension (**FEAT\_SHA3**), available on Apple Silicon and on Neoverse V1/V2 and N2. The permutation is expressed with the extension’s fused instructions: **EOR3** (three-input XOR) for the  $\theta$  column parities, **RAX1** (rotate-by-one and XOR) for the  $\theta$   $D$ -terms, **XAR** (XOR then rotate) to fuse the  $\theta$  accumulation with the  $\rho$  rotation, and **BCAX** (bit-clear and XOR) for  $\chi$ . The single-duplex core places one instance per 64-bit  $D$  lane across the 25 vector registers; the eight-way core packs two instances per 128-bit register and processes the batch of eight as four such pairs, using **ZIP1** and duplicating moves to interleave and extract the two instances of each lane in the fused chunk kernel. The same pair machinery backs a 2-wide kernel that drains a leaf-chunk remainder two chunks at a time—one instance per 64-bit half of a 128-bit register—at roughly twice the single-duplex per-chunk rate. Both batched kernels store the permutation state back into the batch after the closing block—the  $\times 8$  kernel for its first pair, the 2-wide kernel for both instances—which is the writeback that permits the root duplex to occupy lane 0.

## D Test Vectors

The following test vectors cover the corner cases of TW128 encryption, as specified in § 3.

All byte strings are written in hexadecimal. For each vector we list the key  $K$ , nonce  $N$ , associated data  $A$ , message  $M$ , ciphertext  $C$ , and tag  $T$ ; the sealed output of encryption is  $C \parallel T$ . The empty string is written  $\varepsilon$ . The leaf index is encoded as a little-endian 64-bit integer and the phase suffixes are appended least significant bit first.

To keep the longer vectors compact, any value exceeding 32 bytes is given by its length in bytes together with its SHA-256 digest rather than in full. Every message and associated data byte is defined by  $m_i = (17i + 31) \bmod 256$  for  $i = 0, 1, \dots$ , so each such input is determined by its length alone and can be reconstructed and checked against the stated digest; a ciphertext given by its length and digest can likewise be checked by encrypting the corresponding message and hashing the result.

---

**Vector 1.** Empty associated data and empty message: the associated data and aggregation phases are both elided, and the root tag is emitted by the single closing message call with no leaves.

---

|     |                                                                  |
|-----|------------------------------------------------------------------|
| $K$ | 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f |
| $N$ | 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f |
| $A$ | $\varepsilon$                                                    |
| $M$ | $\varepsilon$                                                    |
| $C$ | $\varepsilon$                                                    |
| $T$ | da65bbda0b20b1e936f5326458166633d1e5a3f7aebc0b318d24b051a4efa697 |

---

**Vector 2.** Multi-block associated data with an empty message: the associated data spans two rate blocks and no message bytes are processed.

---

|     |                                                                                        |
|-----|----------------------------------------------------------------------------------------|
| $K$ | 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f                       |
| $N$ | 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f                       |
| $A$ | 168 bytes, SHA-256<br>4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a |
| $M$ | $\varepsilon$                                                                          |
| $C$ | $\varepsilon$                                                                          |
| $T$ | 2093878418f3fca96ad3a8d1748a9be4d2a8b518e917d988fd703dcb7bf72173                       |

---

**Vector 3.** Empty associated data and a single-block message: the root message phase only, no leaves.

---

2575 *K* 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f  
*N* 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f  
*A*  $\epsilon$   
*M* 1f30415263748596a7b8c9daebfc0d1e2f405162738495a6b7c8d9eafb0c1d2e  
*C* 676a1441e20daea0f1ee4571ee4543842c946d59e0d3df6f1dd2f92dc4f876aa  
*T* 1873332d359ff2fe5197488224af7bf6ad821875fdbd964100de424aa77342be

---

**Vector 4.** Empty associated data and a two-block message: the root keystream is chained across two rate blocks.

---

2576 *K* 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f  
*N* 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f  
*A*  $\epsilon$   
*M* 168 bytes, SHA-256  
4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a  
*C* 168 bytes, SHA-256  
df3f0a98a91a69ed474edbc52e0dc1d8e54bfd6fc514966ce7b878d48be0bf9a  
*T* aab2413074a202cfadd5eb784663b1831eb330025972d163d03e9ed5a964b765

---

**Vector 5.** A message of exactly one chunk: the root is full, there is still no leaf, and the aggregation phase is elided so the closing root message call emits the tag.

---

2577 *K* 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f  
*N* 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f  
*A*  $\epsilon$   
*M* 8183 bytes, SHA-256  
6532a370a4ab6b5f4094dd8a6016512ab8e8416abe910bc1a0b532979224151d  
*C* 8183 bytes, SHA-256  
8d58f8aa3413ea0ba1b8cc3ed4e9a334521d8dfe9694a0661fb8251049907e9  
*T* c2201637f1116c1303b8982622756770d23acb85384f4658fcb63c37e4b629dd

---

**Vector 6.** A message of one chunk and one further byte: one root chunk and one leaf, exercising leaf tag aggregation.

---

2578 *K* 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f  
*N* 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f  
*A*  $\epsilon$   
*M* 8184 bytes, SHA-256  
c4c383bca0808e2ce44d481ddfe56efdc7a147dcd5a14c898ef5423f2604cd7  
*C* 8184 bytes, SHA-256  
ef1b2f7607ade503ab497fc61e62cef2b44c6328fa06e499718eb7da5fdd04d3  
*T* c96bea48e07a8431fd19721ffbe8afe6a73047f96d953e3281f10126eeca06a1

---

**Vector 7.** A multi-leaf message of two chunks and one further byte: the full tree path with two leaves.

---

2579 *K* 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f  
*N* 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f  
*A*  $\epsilon$   
*M* 16367 bytes, SHA-256  
17e31f3700d0602aab8a686024ecc05b8192e80a531483c67dd90d5a26557c5e  
*C* 16367 bytes, SHA-256  
6d83d8fbdca2a7c2ef590d29a36527a45019e50d1f8c8db037959081685a46908  
*T* cdc2e50ac5380419f06772a2f3af2146050bd0340b35f61b67f839351767804e

---

**Vector 8.** Maximum associated data with a short message: the associated data blocks dominate the cost.

---

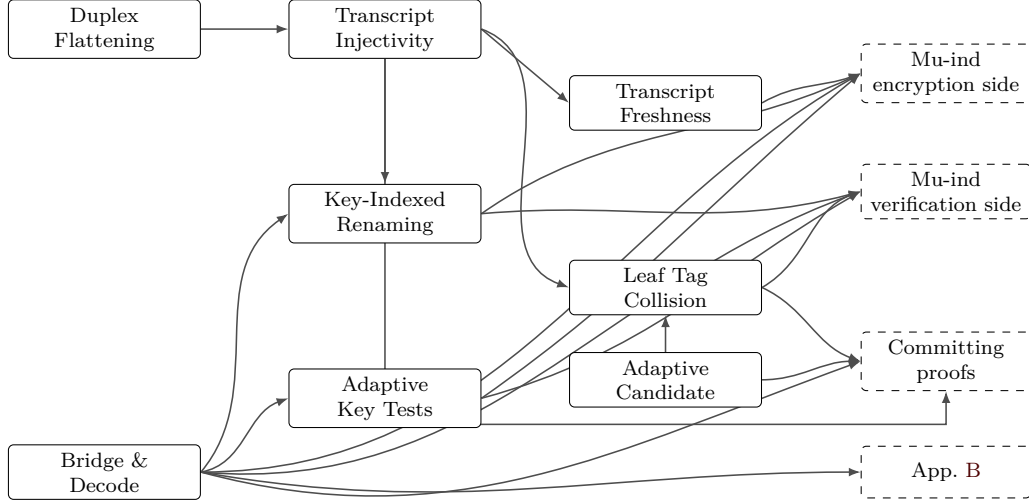
2580 *K* 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f  
*N* 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f  
*A* 337 bytes, SHA-256  
04ca7b24930bc6b5fe8b95ff1eb0933907657c1283cc3a7e69c5f7f02220f281  
*M* 1f30415263748596a7b8c9daebfc0d1e  
*C* Odd615a0a05ed82a78c3999b9e80cbcd  
*T* 22e4e6725f548ad0660041d66cf2f18dc45c69b447cdaae1f147d33a3b6aa1a9

---

|                                                                        |                                                                                         |
|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Vector 9.</b> An all-ones key and nonce with a multi-chunk message. |                                                                                         |
| <i>K</i>                                                               | ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff                        |
| <i>N</i>                                                               | ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff                        |
| <i>A</i>                                                               | 168 bytes, SHA-256<br>4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a  |
| <i>M</i>                                                               | 8184 bytes, SHA-256<br>c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7 |
| <i>C</i>                                                               | 8184 bytes, SHA-256<br>49818c320ab282296639798d840cbd60751e1623f449dcbf9423371d44145dfb |
| <i>T</i>                                                               | 310c9fff20e9651f8ed47eed9bb182a086ed1af1b3f1799e35307a4139865fc7                        |

---

2581



**Figure 2:** Dependency graph of the supporting lemmas in this appendix and their consumers. Solid boxes are lemmas; dashed boxes are the proof components that consume them. The Bridge & Decode box is Lemma 6 together with the exact keyed decoder it specifies (Definition 2); the Key-Indexed Renaming box (Lemma 7) draws on it and on Lemma 4 to supply the key-field disjointness and renaming used by the hidden-key encryption and verification accounting. The Adaptive Candidate box (Lemma 8) is the single probabilistic engine: it bounds the Leaf Tag Collision lemma (Lemma 10) and, through the Root Tag Hash Bound (Lemma 2) of Section 6, the CMTDs-4, collision, preimage, and second-preimage proofs. The edge from Lemma 4 into the committing proofs in Section 6 and the proof of Corollary 6 is mediated by Opening Triple Separation (Proposition 1) for distinct contexts and Final-Root Input Separation (Proposition 2)—which adds the full-ciphertext recovery of Lemma 5 for distinct full inputs absent a leaf tag collision—together with Corollary 7 for candidate-sequence filtering. The Leaf Tag Collision box feeds the verification side of mu-ind through its hidden-key bound and the committing collision and second-preimage proofs through its adversary-supplied-key bound. The edge from Lemma 4 into Lemma 11 uses Corollary 7; the verification-first root tag guessing accounting reached through the same separation is carried out in Section A.9 (Lemma 13) and consumed by Lemma 14. The Lemma 9 edge records the shared hidden-key  $\text{bad}_{\text{key}}$  accounting used by the mu-ind proof.